INFORMATION AND COMPUTATION 105, 159–267 (1993)

Full Abstraction in the Lazy Lambda Calculus

SAMSON ABRAMSKY

Department of Computing, Imperial College of Science, Technology and Medicine, 180, Queen's Gate, London SW7 2BZ, England

AND

C.-H. LUKE ONG

Department of Information Systems and Computer Science, National University of Singapore, Lower Kent Ridge Road, Singapore 0511, Republic of Singapore, and Trinity College, Cambridge University, Cambridge CB2 1TQ, England

A theory of lazy λ-calculus is developed as a basis for lazy functional programming languages. This is motivated by a mismatch between the "standard" (i.e., sensible) theory of the classical λ-calculus and the practice of lazy functional programming. Part I sets up the fundamentals of the lazy λ-calculus starting from the pure lazy language λl. We develop the rudiments of weakly sensible λ-theory and characterize λl as the maximal such theory. The axiomatic framework of our theory is based on the notion of applicative transition systems which gives rise to lazy λ-models. The subclass of lazy λ-models which are operationally extensional are called lambda transition systems (lts's). The canonical model of the lazy λ-calculus is the initial solution to the domain equation $D \cong [D \to D]_\perp$. We set up the domain logic corresponding to the same type expression. In part II, we recast the full abstraction problem in the lazy λ-calculus. The pure lazy language λl is not equationally fully abstract w.r.t. D. In fact, λl, λl_ω (a labelled version of λl), and λl_c (λl augmented with a convergence testing constant) are all not equationally fully abstract w.r.t. D. We illustrate two approaches to full abstraction:

- The expansive approach consists in enriching the language by augmenting it with appropriate constants, thereby enabling all finite semantic information to be represented syntactically as program phrases using the machinery of domain logic developed in Part I. We show that λl_p, which is λl augmented with a parallel convergence testing constant, is conditionally fully abstract *w.r.t. D*.

- The restrictive approach regards the language as prescribed and tailors the model to fit by "cutting down" (i.e., "quotienting out" by a bisimulation logical relation) the existing "over-generous" canonical model D to an appropriate substructure. We establish a general method to construct inequationally fully abstract lazy λ-models which are retracts of D for a class of sufficiently expressive lts's in which the n-projection maps are internally definable.

0890-5401/93 $5.00

Copyright © 1993 by Academic Press, Inc.
All rights of reproduction in any form reserved.

The restrictive approach solves the full abstraction problem for λI_c and λI_ω but not λI. The full abstraction problem for λI is reduced to a conjecture of the conservativity of λI_ω over λI. Finally, we study lazy languages with ground data and show that it is wrong to conflate parallel or with parallel convergence testing.
© 1993 Academic Press, Inc.

Contents

1. *Introduction.* 1.1. Motivations. 1.2. Organization of the paper. 1.3. Notation.
2. *The lazy lambda calculus: Applicative bisimulation.* 2.1. Pure lazy language. 2.2. Applicative bisimulation. 2.3. Weakly sensible λ-theory.
3. *Applicative transition systems and lazy lambda models.* 3.1. Applicative transition systems. 3.2. Lazy λ-models. 3.3. Lambda transition systems.
4. *Basic properties of the canonical model D.* 4.1. Introduction. 4.2. Construction of the initial solution. 4.3. Index calculation and extensionality properties. 4.4. λC-definability of the projection maps ψ_n. 4.5. Application preserves arbitrary joins.
5. *A Domain logic for applicative transition systems.* 5.1. Introduction. 5.2. Domain logic $\mathscr{L}$. 5.3. Proof system for $\mathscr{L}$. 5.4. Stone duality. 5.5. Categories of ATS and their final algebra.
6. *The Full abstraction problem—A brief introduction.* 6.1. Introduction. 6.2. Full abstraction in the lazy lambda calculus.
7. *Full abstraction I: An expansive approach.* 7.1. Expansive full abstraction. 7.2. Operational extensionality results.
8. *Non-full-abstraction results.* 8.1. λI is not equationally fully abstract w.r.t. D. 8.2. λI_c is not equationally fully abstract w.r.t. D.
9. *Full abstraction II: A restrictive approach.* 9.1. Overview of the section. 9.2. Approximants of the language: $\mathscr{K}_i = \langle \Lambda(K)_i^o, \cdot^*, \Downarrow_{\mathscr{K}} \rangle$. 9.3. Relationships between λI, λI_ω, λI_c, and λI_p. 9.4. Bisimulation logical relations and their properties. 9.5. Approximants of the model: Q_i^*. 9.6. Proof of full abstraction.
10. *Variations.* 10.1 Lazy lambda calculus with ground data. 10.2. Varieties of function space.

1. Introduction

1.1. *Motivations*

The commonly accepted basis for functional programming is the λ-calculus; and it is folklore that the λ-calculus *is* the prototypical functional language in purified form. But what is the λ-calculus? The syntax is simple and classical; variables, abstraction, and application in the pure calculus, with applied calculi obtained by adding constants. The further elaboration of the theory, covering conversion, reduction, theories and models, is laid out in Barendregt's already classical treatise (Bar84). It is instructive to recall the following crux, which occurs rather early in that work (p. 39):

Meaning of λ-Terms: First Attempt

- The meaning of a λ-term is its normal form (if it exists).
- All terms without normal forms are identified.

This proposal incorporates such a simple and natural interpretation of the λ-calculus as a programming language, that if it worked there would surely be no doubt that it was the right one. However, it gives rise to an inconsistent theory! (See the above reference.)

Second Attempt: Sensible Theory

- The meaning of λ-terms is based on *head normal forms* via the notion of *Böhm tree.*
- All *unsolvable* terms (no head normal form) are identified.

This second attempt forms the central theme of Barendregt's book, and gives rise to a very beautiful and successful theory (henceforth referred to as the "standard theory"), as that work shows.

This, then, is the commonly accepted foundation for functional programming; more precisely, for the *lazy* functional languages (FW76, HM76), which represent the mainstream of current functional programming practice. Examples: MIRANDA (Tur 85), LML (Aug84), LISPKIT (Hen80), ORWELL (Wad85), PONDER (Fai85), TALE (BvL86). But do these languages as defined and implemented actually evaluate terms to head normal form? To the best of our knowledge, *not a single one of them does so.* Instead, they evaluate to *weak head normal form*; i.e., they do not evaluate under abstractions (see (PJ87) for a comprehensive survey of the pragmatics of functional programming languages).

Example

$\lambda x.(\lambda y.y)M$ is in weak head normal form, but not in head normal form, since it contains the head redex $(\lambda y.y)M$.

So we have a fundamental *mismatch* between theory and practice. Since current practice is well motivated by efficiency considerations and is unlikely to be abandoned readily, it makes sense to see if a good modified theory can be developed for it. To see that the theory really does need to be modified, we consider the following example.

Example

Let $\Omega \equiv (\lambda x.xx)(\lambda x.xx)$ be the standard unsolvable term. Then

$$\lambda x.\Omega = \Omega$$

in the standard theory, since $\lambda x.\Omega$ is also unsolvable; but $\lambda x.\Omega$ is in weak

head normal form, hence should be distinguished from Ω in our "lazy" theory.

We now turn to a second point at which the standard theory is not completely satisfactory.

Is the λ-Calculus a Programming Language?

In the standard theory, the λ-calculus may be regarded as being characterized by the type equation

$$D = [D \to D]$$

(for justification of this in a general categorical framework, see, e.g., (Sco80, Koy82, LS86)).

It is one of the most remarkable features of the various categories of domains used in denotational semantics that they admit non-trivial solutions of this equation. However, there is no *canonical* solution in any of these categories (in particular, the initial solution is trivial—the one-point domain).

We regard this as a symptom of the fact that the pure λ-calculus in the standard theory *is not a programming language.* Of course, this is to some extent a matter of terminology, but we feel that the expression "programming language" should be reserved for a formalism with a definite computational interpretation (an operational semantics). The pure λ-calculus as ordinarily conceived is too schematic to qualify.

A further indication of the same point is that studies on full abstraction carried out in (PLO77, Mil77) and more recently (Mul86) have not been carried over to the pure λ-calculus, for lack of any convincing way of doing so in the standard theory. This in turn impedes the development of a theory which integrates the λ-calculus with concurrency and other computational notions.

We shall see that by contrast with this situation, the lazy λ-calculus we develop does have a canonical model; that the ideas of Plotkin, Milner and Mulmuley can be carried over to it in a very natural way; and that the theory we develop runs quite strikingly in parallel with the first author's treatment of concurrency in (Abr91a).

1.2. *Organization of the Paper*

The rest of the paper may be divided into two parts. Part I sets up the theory of the *lazy lambda calculus* and comprises Sections 2 to 5. Part II, consisting of Sections 6 to 10, applies the theory to the problem of *full abstraction.*

Part I: *The Lazy Lambda Calculus*

In Section 2, we introduce the intuitions on which our theory is based in the concrete setting of a "minimal" functional programming language called λl, consisting of the closed λ-terms equipped with a reduction relation that captures lazy evaluation. We introduce an operational preorder $\precsim^B$ on λ-terms, called *applicative bisimulation.* A characterization of the λ-theory induced by it is then given in the class of *weakly sensible* λ-theories.

In Section 3, we set up the axiomatic framework for our theory based on the notion of *applicative transition systems* which forms a bridge both to the standard theory, and to concurrency and other computational notions. We propose the notion of *lazy λ-model* and give three essentially equivalent formulations as in the case of the classical λ-model. We identify a subclass of the lazy λ-models which satisfy a strong (bisimulation) extensionality principle. They are called *lambda transition systems* (lts's) and are the main objects of our study for the rest of the paper.

Section 4 focuses on the canonical model of the lazy λ-calculus D which is the initial solution of the domain equation

$$D \cong [D \to D]_\perp,$$

and on its extensionality properties. We show that the canonical projection maps $\psi_n : D \to D_n$ are λC-*definable* (but *not* λ-definable) where C is a *convergence testing* constant.

The same domain equation (for applicative transition systems) seen as a type expression is then given a *logical* interpretation in Section 5—in addition to the by now standard denotational interpretation—in which types are interpreted as propositional theories and terms are interpreted via a program logic as set out in (Abr87c) (full version: (Abr91b)). We then apply the theoretical results in *op. cit.* to show that the two interpretations are *Stone duals* of each other. Hence, semantics and logic are guaranteed to be in harmony with each other. This opens the way to a wide range of applications. One such pleasing application is a strikingly simple proof of a Computational Adequacy Theorem, which asserts that a term converges operationally if and only if it denotes a non-bottom element in our domain.

Part II: *The Full Abstraction Problem*

Section 6 reviews the *full abstraction* problem and formulates the property of full abstraction in three variants of increasing strength: namely, *equational*, *inequational*, and *conditional* full abstraction.

The pure lazy language λI is not (equationally) fully abstract for D[1] (a main non-full-abstraction result in Section 8). This is because the model is in some sense "over generous" w.r.t. the language, or dually, the language lacks expressiveness *vis-à-vis* the model. This "poorness of fit" may be rectified by either of the two following approaches depending on whether the language or the model is taken as prior:

- The *expansive* approach consists in enriching the language by augmenting it with appropriate constants, thereby enabling all *finite* semantic information to be represented syntactically as program phrases.
- The *restrictive* approach regards the language as prescribed and tailors the model to fit by "cutting down" the existing "over-generous" model to an appropriate sub-structure.

There is another approach to full abstraction which regards the language rather than the model as prior. This approach, followed by Berry, Curien, and their colleagues seeks to synthesize fully abstract models by natural or semantic means. We direct the reader to the survey article (BCL85) and Curien's book (Cur86) for more details.

Section 7 studies the *expansive* approach to full abstraction. We show how the ideas of Plotkin (Plo77) can be formulated in the setting of the pure untyped λ-calculus. A distinctive feature of this approach is the use of the machinery of domain logic set up in Section 5 in studying the equational (or equations conditional on the existence of certain terms) theory over our "programs" (λ-terms). Two functional languages, λI_c and λI_p, which are essentially λI augmented with *convergence testing* C and *parallel convergence testing* P, respectively, are introduced. We apply the main result in the section (Expansive Full Abstraction Theorem) to show that λI_p is *conditionally fully abstract* w.r.t. D. A second application of domain logic is the proof of a general *operational extensionality* result for a class of lazy λ-models.

Two non-full-abstraction results are presented in Section 8: both λI_c and λI are equationally not fully abstract w.r.t. D. The former establishes a *topological incompleteness* result (in the sense of (HdR88)) as a corollary.

Section 9 studies the *restrictive* approach to full abstraction. We follow the ideas of Mulmuley (Mul86) and Milner (Mil77) to construct inequationally fully abstract lazy λ-models which are *retracts* of the canonical model D for a sufficiently expressive class of lts's in which the canonical

[1] The notion of *full abstraction*, which first appeared in (Mil75), normally qualifies a model with reference to an operational equivalence, rather than the other way round as we have stated above. While taking the language or the model as the reference point in the definition of full abstraction may reflect one's perspective, it does not change the mathematical meaning of the definition; i.e., conincidence of the operational and the semantic sense of equivalence.

projection maps $\Psi_n: D \to D_n$ which send elements in D to their respective nth approximants, are definable. This class includes λI_c but not λI. As an attempt to apply the construction to λI, we introduce a new language λI_ω, which is λI augmented with formal constants Ψ_n which simulate the projection maps. The full abstraction problem for λI is then reduced to a conjecture on the *conservativity* of the inequational theory of λI_ω over λI. We also show in this section that λI_ω is not equationally fully abstract w.r.t. D and establish the relationship between the λ-theories induced by λI_p, λI_c, λI_ω and λI. We summarize the full abstraction results obtained in the paper as follows:

Languages	Fully abstract models	Fully abstract w.r.t. D
λI_p	D (Thm 7.1.3)	Yes (Prop 7.2.10)
λI_c	$Q^{\lambda I_c}$ (Thm 9.6.5)	No (Thm 8.1.2)
λI_ω	$Q^{\lambda I_\omega}$ (Thm 9.6.5)	No (Prop 9.3.4)
λI	? (Conj 9.6.6)	No (Prop 8.2.5)

In Section 10, we consider the extension of lazy languages with *ground constants* and show that once *ground constants* are added to a language, *parallel or* and *parallel convergence* are two distinct notions and it is wrong to conflate them.

1.3. *Notation*

Most of the notation from elementary set theory and logic which we use is standard and should cause no problems to the reader. Let A, B be sets. We denote A^B as the set of all functions from B to A. $f: A \rightharpoonup B$ means that f is a partial function from A to B. We use $=^{\text{def}}$ for *definitional equality*; thus $M =^{\text{def}} N$ means "the expression M is by definition equal to N" (or just: "is defined to be N"). Syntactic equality is denoted by $\equiv$. We use ω to denote the natural numbers $\{0, 1, \ldots\}$ (thought of sometimes as an ordinal, and sometimes as just a set). Given a set X, we write $\wp X$ for the powerset of X.

Our notational convention in λ-calculus largely follows that of (Bar84). In particular, for M ranging over λ-terms, $\vec{M}$ is an abbreviation for a possibly empty finite sequence of λ-terms $M_1, \ldots, M_n$ whose length $|\vec{M}|$ is n, where $n \geqslant 0$. Note that we write substitution of N for x in M, where M, N are expressions and x is a variable, as $M[N/x]$. We assume the usual notions of free and bound variables, as expounded, e.g., in (Bar84). We always take expressions modulo α-conversion, and treat substitution as a *total* operation in which variable capture is avoided by suitable renaming of bound variables.

Our notations for semantics follow those standardly used in denotational semantics as in (Bat84). One operation we frequently need is *updating* of

environments. Let $\mathsf{Env} = \mathsf{Var} \to \mathcal{V}$, where Var is a set of variables, and $\mathcal{V}$ some value space. Then for $\rho \in \mathsf{Env}$, $x \in \mathsf{Var}$, $v \in \mathcal{V}$, the expression $\rho[x \mapsto v]$ denotes the environment defined by

$$(\rho[x \mapsto v])\, y = \begin{cases} v, & \text{if } x = y, \\ \rho y, & \text{otherwise.} \end{cases}$$

We assume such standard notions as cpo's and continuous functions. Our main reference to domain theory is (Plo81). We denote the standard *step function* $f: D \to E$ between cpo's such that

$$f(x) \stackrel{\text{def}}{=} \begin{cases} e & \text{if } d \sqsubseteq x, \\ \bot & \text{otherwise} \end{cases}$$

as $(d \searrow e)$.

2. The Lazy Lambda Calculus: Applicative Bisimulation

In this section, we introduce a functional programming language called the *pure lazy language* which has closed λ-terms as *programs* and *abstractions* as *values*. The evaluation mechanism is represented by a binary reduction relation over Λ^o simulating a normal order (or leftmost) reduction strategy that terminates whenever the reduction reaches a weak head normal form (whnf). An operational preorder on λ-terms $\lesssim^B$ which we call *applicative bisimulation* is then introduced and its relationship to *observational* and *contextual preorders* investigated. These lead up to λl (called the *Abramsky lazy λ-theory* in (Ong88b, Ong88a) which is shown to be *fully lazy* as a λ-theory. We show that λl is *Hilbert–Post complete* (i.e., maximally consistent) with respect to the class of *fully lazy λ-theories*.

2.1. *Pure Lazy Language*

Definition 2.1.1. The relation $M \Downarrow \mathcal{N} \mathbin{@} n$ ("M converges to *principal weak head normal form* N in n steps") is defined inductively over Λ^o as follows:

$$\frac{}{\lambda x.M \Downarrow \lambda x.M \mathbin{@} 0} \qquad \text{(abs)}$$

$$\frac{M \Downarrow \lambda x.P \mathbin{@} m \quad P[Q/x] \Downarrow N \mathbin{@} n}{MQ \Downarrow N \mathbin{@} m+n+1}. \qquad (\beta_N)$$

The (unlabelled) transition system $\lambda l = ^{\text{def}} \langle \Lambda^o, \Downarrow \rangle$ is called the *pure lazy language*.

The reduction strategy captured by the above formulation is precisely the *leftmost* reduction with the proviso that all reductions terminate upon reaching abstractions. The relation $\Downarrow$ is essentially the same as the "eval" relation in Plotkin's *call-by-name* programming language defined in (Plo75); hence the subscript "N" in the label (β_N) of the second inference rule. We write $M \Downarrow N @ n$ simply as $M \Downarrow N$ unless there is a need to keep track of the number of steps it takes for M to converge, for example, in induction proofs. We invite the reader to show, by induction on the length of $\vec{L}$, that

$$(\lambda x.M)\, N\vec{L} \Downarrow P @ n+1 \Leftrightarrow M[N/x]\vec{L} \Downarrow P @ n,$$

a fact which we need later in the proof of Proposition 2.2.4.

Notation 2.1.2.

$$M\Downarrow \stackrel{\text{def}}{=} \exists N.M \Downarrow N \qquad (\text{"}M \text{ converges"});$$

$$M\Downarrow \stackrel{\text{def}}{=} \neg(M\Downarrow) \qquad (\text{"}M \text{ diverges"}).$$

Plainly, the reduction relation $\Downarrow$ defines a partial function from Λ^o to the class of principal weak head normal forms (the "range" of $\Downarrow$ is the collection of all closed abstractions, to be precise).

2.2. *Applicative Bisimulation*

Under the reduction strategy $\Downarrow$, the possible "results" are of a particularly simple, indeed *atomic* kind. That is to say, either a term M converges to an abstraction (and according to this strategy, we have no clue as to the structure "under" the abstraction), or it diverges. The relation $\Downarrow$ by itself is too "shallow" to yield information about the behaviour of a term under all experiments.

Inspired by the work of Robin Milner (Mil80) and David Park (Par81) on concurrency, we use the reduction relation $\Downarrow$ as a building block to yield a deeper relation which we call *applicative bisimulation.*[2] To motivate this relation, let us spell out the observational scenario we have in mind: Given a closed term M, the only experiment of depth 1 we can do is to evaluate M and see if it converges to some abstraction (weak head normal form) $\lambda x.M_1$. If it does so, we can continue the experiment to depth 2 by supplying a term N_1 as input to $\lambda x.M_1$, and so on. Note that what the

[2] In keeping with the usual distinction between *simulation* and *bisimulation* in the theory of concurrency, the operational preorder should probably be named more accurately as *applicative simulation.*

experimenter can observe at each stage is only the *fact* of convergence, not which lies under the abstraction. We can picture matters thus:

Stage 1 of experiment:	$M \Downarrow \lambda x.M_1$;
	environment "consumes" λ,
	produces N_1 as input
Stage 2 of experiment:	$M_1[N_1/x] \Downarrow \ldots$
$\vdots$	

DEFINITION 2.2.1. We define a sequence of binary relations $\langle \lesssim_k^B : k \in \omega \rangle$ on Λ^o as follows:

- $\forall M, N. M \lesssim_0^B N$.
- $M \lesssim_{k+1}^B N =^{\text{def}} M \Downarrow \lambda x.P \Rightarrow [N \Downarrow \lambda x.Q \quad \& \quad \forall R \in \Lambda^o. P[R/x] \lesssim_k^B Q[R/x]]$.
- $M \lesssim^B N =^{\text{def}} \forall k \in \omega. M \lesssim_k^B N$.

(Throughout this work, we assume that all α-convertible terms are implicitly identified.) The definition is then extended to all λ-terms by considering closures in the usual way; i.e., for $M, N \in \Lambda$,

$$M \lesssim^B N \stackrel{\text{def}}{=} \forall \sigma : \mathsf{Var} \to \Lambda^o. M_\sigma \lesssim^B N_\sigma$$

(where, e.g., M_σ means the result of simultaneously substituting $\sigma(x)$ for each occurrence of x in M with x ranging over Var, the collection of denumerable variables in the λ-calculus). We abbreviate $M \lesssim^B N \,\&\, N \lesssim^B M$ as $M \sim^B N$.

It is easy to see that an equivalent definition of applicative bisimulation is the following:

$$M \lesssim^B N \Leftrightarrow \forall \vec{P} \subseteq \Lambda^o. M\vec{P}\Downarrow \Rightarrow N\vec{P}\Downarrow.$$

For an alternative description of $\lesssim^B$, recall that $\mathsf{Rel}(\Lambda^o) =^{\text{def}} \wp(\Lambda^o \times \Lambda^o)$, the set of all binary relations over Λ^o, is a complete lattice under set inclusion. Now, define $F: \mathsf{Rel}(\Lambda^o) \to \mathsf{Rel}(\Lambda^o)$ by

$$F(R) \stackrel{\text{def}}{=} \{(M, N): M \Downarrow \lambda x.P \Rightarrow [N \Downarrow \lambda x.Q \,\& \, \forall L \in \Lambda^o. (P[L/x], Q[L/x]) \in R]\}.$$

Since F is monotone, by Tarski's Theorem (Tar55), it has a maximal fixpoint given by $\bigcup \{R: R \subseteq F(R)\}$ which is precisely $\lesssim^B$ since the *closure*

ordinal (Mos74) of $\langle \precsim_k^B : k \in \omega \rangle$ is ω. $\precsim^B$ satisfies the following recursive specification:

$$M \precsim^B N \Leftrightarrow M \Downarrow \lambda x.P \Rightarrow [N \Downarrow \lambda x.Q \,\&\, \forall R \in \Lambda^o.P[R/x] \precsim^B Q[R/x]].$$

DEFINITION 2.2.2. We define an (in)equational theory $\lambda I =^{\text{def}} \langle \Lambda^o, \sqsubseteq, = \rangle$ where:

$$\lambda I \vdash M \sqsubseteq N \stackrel{\text{def}}{=} M \precsim^B N,$$

$$\lambda I \vdash M = N \stackrel{\text{def}}{=} M \sim^B N.$$

Of course, we need to verify that λI is indeed a λ-theory (in fact, it is a *fully lazy λ-theory*); and this we do in the following subsection.

Note that we refer to the pure lazy language and the λ-theory induced by applicative bisimulation by the same name: λI.

Properties of $\precsim^B$ and λI

In this subsection, we prove the *operational extensionality* of $\precsim^B$. First, we introduce a definition.

DEFINITION 2.2.3. The binary relation $\precsim^{\text{cxt}}$ over Λ^o is defined as

$$M \precsim^{\text{cxt}} N \stackrel{\text{def}}{=} \forall C[\,] \in \Lambda^o.C[M]\Downarrow \Rightarrow C[N]\Downarrow.$$

$\precsim^{\text{cxt}}$ is extended to a relation over Λ in the same way as $\precsim^B$. By virtue of the following result, we will not mention $\precsim^{\text{cxt}}$ any more in the sequel.

PROPOSITION 2.2.4 (Operational Extensionality). $\precsim^B = \precsim^{\text{cxt}}$.

Proof. This is a variation of Berry's proof of a *Context Lemma* in (Ber81) communicated to us by Allen Stoughton (Sto88b).

It suffices to prove the following: Let M, N range over Λ^o. Then

$$M \precsim^B N \Rightarrow \forall l \in \omega.\forall C[\,] \in \Lambda^o.C[M]\Downarrow @\, l \Rightarrow C[N]\Downarrow.$$

We prove the assertion by induction on l. The base case is obvious. Without loss of generality, consider the following two cases of closed contexts:

(1) $C[\,] \equiv (\lambda x.P[\,])(Q[\,])\,\vec{R}[\,]$,

(2) $C[\,] \equiv [\,](P[\,])\,\vec{Q}[\,]$.

(1) Suppose $C[M]\Downarrow @\, l+1$. Define $D[\,] = (P[\,])[Q[\,]/x]\,\vec{R}[\,]$. Then it is easy to see that

$$C[M] > D[M]\Downarrow @\, l,$$

where $>$ is the one-step leftmost reduction relation. Invoking the induction hypothesis, we have $D[N]\Downarrow$, which implies that $C[N]\Downarrow$.

(2) Let $M \equiv (\lambda x.U)\vec{V}$. Suppose $C[M]\Downarrow @ l+1$. Define $D[\ ] = (\lambda x.U)\ \vec{V}(P[\ ])\ \vec{Q}[\ ]$, a context of case (1). Note that $C[M] \equiv D[M]$. By an appeal to (1), we have $D[N]\Downarrow$. But $D[N] \equiv MP[N]\ \vec{Q}[N]$, and so by $M \precsim^B N$, we have $NP[N]\ \vec{Q}[N]\Downarrow$, i.e. $C[N]\Downarrow$. ∎

In Section 7, we present another proof which makes essential use of domain logic, despite the fact that the *statement* of the result does not mention domains at all.

Remark 2.2.5. (i) The above result says that any two programs whose respective computational behaviours are distinguishable by some program context may be distinguished just by considering *applicative contexts*. In order words, the computational behaviour of any program in the language λI is characterized by considering it as a *function* and observing the computational output when it is applied to all arguments. This is what one would expect of a functional programming language. This property is called *operational extensionality* in (Blo88). Milner [Mil77) proved a similar result in the case of simply typed combinatory algebra which he referred to as the *Context Lemma*. See also (Ber81, Mul86, Blo88).

(ii) It follows immediately from the definition of $\precsim^B$ that the application operation in Λ^o is monotone in the *left* argument with respect to $\precsim^B$. Operational extensionality is equivalent to the monotonicity of the application operation in the *right* argument; i.e.,

$$M \precsim^B N \Rightarrow \forall P \in \Lambda^o.\, PM \precsim^B PN,$$

which is the same as saying that $\precsim^B$ is *substitutive*, i.e.,

$$M \precsim^B N \,\&\, P \precsim^B Q \Rightarrow MP \precsim^B NQ.$$

In view of Proposition 2.2.4, it should now be clear that λI is a λ-theory.

λI admits a nice λ-theoretic characterization. First, let us introduce some notions in *weakly sensible* λ-theory (see (Ong88b, Chap. 2) for a more detailed account).

2.3. *Weakly Sensible λ-Theory*

We introduce the notion of the *functionality order* of a λ-term (see also (Lon83)). Informally, the functionality order of a term M expresses

how "higher order" M is, which corresponds roughly to the (maximal) number of nested λ-abstractions M has or is convertible to. A (closed) term of functionality order 0, say, is not β-convertible to an abstraction; or equivalently, when applied to another term, does not "consume" it.

DEFINITION 2.3.1. (i) *A* term M has *functionality order* (or simply *order*) 0, denoted $M \in \mathbf{O}_0$, if $\neg[\exists N.\lambda\beta \vdash M = \lambda x.N]$.

(ii) M has *proper order* 0, denoted $M \in \mathbf{PO}_0$, if

$$M \in \mathbf{O}_0 \ \& \ \neg[\exists \vec{N}.\lambda\beta \vdash M = x\vec{N}].$$

We say that M is *strongly unsolvable* if M is of proper order 0.

(iii) Let $n \geqslant 1$. M has *order* n, denoted $M \in \mathbf{O}_n$, if n is the largest i such that

$$\exists N.\lambda\beta \vdash M = \lambda x_1 \cdots x_i.N.$$

(iv) M has *order* ∞, denoted $M \in \mathbf{O}_\infty$, if $\forall n \in \omega.M \notin \mathbf{O}_n$.

(v) M is a *functional term*, denoted $M \in \mathscr{F}$, if $\exists N.\lambda\beta \vdash M = \lambda x.N$; further,

$$M \in \mathscr{F}_n \overset{\text{def}}{=} \exists N.\lambda\beta \vdash M = \lambda x_1 \cdots x_n.N.$$

Remark 2.3.2. It should not be difficult to see that $\{\mathbf{O}_i : i \in \omega + 1\}$ is a partition of Λ. Also,

$$\begin{array}{ccccccc} \mathscr{F} \equiv \mathscr{F}_1 & \supset & \mathscr{F}_2 & \supset & \mathscr{F}_3 & \cdots \\ \cup & & \cup & & \cup & \\ \mathbf{O}_1 & & \mathbf{O}_2 & & \mathbf{O}_3 & \cdots \end{array}$$

EXAMPLE 2.3.3. (i) $x\vec{M} \in \mathbf{O}_0$, $\Omega\vec{M} \in \mathbf{PO}_0$, where $\Omega \equiv (\lambda x.xx)(\lambda x.xx)$ is syntactically the minimal strongly unsolvable term and will be important as a prime example of the class of $\mathbf{PO}_0$-terms in the sequel. Writing $\Delta_m \equiv \underbrace{x \cdots x}_{m}$, we have

$$\forall m, n \geqslant 2.\forall \vec{Q} \subseteq \Lambda.\Delta_m \Delta_n \vec{Q} \in \mathbf{PO}_0.$$

(ii) Intuitively, an $\mathbf{O}_\infty$-term is an unboundedly deeply nested λ-term; e.g., $(\lambda xy.xx)(\lambda xy.xx)$. Let F be any fixed-point operator in Λ; i.e., $\forall M \in \Lambda.M(FM) = FM$; e.g.,

$$\mathbf{Y} \equiv \lambda f.(\lambda x.f(xx))(\lambda x.f(xx)) \quad \text{or} \quad \Theta \equiv (\lambda xy.y(xxy))(\lambda xy.y(xxy)).$$

Let $\mathbf{K} \equiv \lambda xy.x$ as usual. Since $\lambda\beta \vdash F\mathbf{K} = \mathbf{K}(F\mathbf{K}) = \cdots = \underbrace{\mathbf{K}(\cdots(\mathbf{K}}_{n}(F\mathbf{K}))) = \lambda x_1 \cdots x_n.F\mathbf{K}$, we have $F\mathbf{K} \in \mathbf{O}_\varkappa$. We can think of any $\mathbf{O}_\varkappa$-term as the infinite process



solving the equation $\xi = \lambda x.\xi$. This is actually a top element in our applicative bisimulation ordering (see Proposition 3.3.6 for a general result) because M converges under all finite stages of evaluation for all arguments—the experimenter can always observe convergence (or "consume an infinite λ-stream").

We claim that $F\mathbf{I} \in \mathbf{PO}_0$, the proof of which follows from the next lemma.

There is a nice operational characterization of the $\mathbf{PO}_0$-terms. First, we introduce the notion of a (one-step) *lazy reduction* which is essentially those instances of one-step *leftmost* reduction that do not occur "under" a λ-abstraction.

DEFINITION 2.3.4. (i) The *one-step lazy reduction* relation $>$ over terms in Λ is the least satisfying the following rules:

$$\frac{}{(\lambda x.M)N > M[N/x]} \qquad \frac{M > M'}{MN > M'N}.$$

(ii) An infinite reduction sequence

$$M_0 \to M_1 \to \cdots M_i \to M_{i+1} \to \cdots$$

is *quasy lazy* if

$$\exists\{n_i : i \in \omega, n_i < n_{i+1}\}.\forall i \in \omega.M_{n_i} \to M_{n_i+1} \text{ is a one-step lazy reduction.}$$

Roughly speaking, an infinite reduction sequence is quasi lazy if there is an infinite strictly increasing "subsequence" of one-step lazy reductions.

LEMMA 2.3.5. *Let $M \in \Lambda$. Then $M \in \mathbf{PO}_0$ iff there is an infinite quasi lazy reduction starting from M. In particular, for $M \in \Lambda^o$,*

$$M \in \mathbf{PO}_0 \Leftrightarrow M\Uparrow.$$

Proof. The proof, which depends on an *Advancement of One-Step Lazy Reduction Lemma* is straightforward. A version can be found in (Ong88a, Chap. 2). ∎

We prove a classification result for the class of unsolvable terms which shows that the $\mathbf{PO}_0$ terms are a proper subclass of the unsolvables. This justifies calling $\mathbf{PO}_0$ the class of strongly unsolvable terms.

PROPOSITION 2.3.6 (Classification of the Unsolvables). *M is unsolvable iff*

(i) $M \in \mathbf{O}_\infty$ *or*

(ii) $\exists n \geqslant 0. \exists N \in \mathbf{PO}_0. \lambda\beta \vdash \lambda x_1 \cdots x_n. N = M$

Then, n is called the order of unsolvability *of M. In the case of* (i), *we say that M has order of unsolvability ∞.*

Proof. $\Leftarrow$: Suppose (i) or (ii) holds. Then, it is easy to see that the head reduction on M diverges. The result then follows from Wadsworth's operational characterization of unsolvability (see, e.g., (Bar84, Corollary 11.4.8)).

$\Rightarrow$: Suppose $M \notin \mathbf{Q}_\infty$. Assume n is the largest such that $M = \lambda x_1 \cdots x_n. N$. Then $\lambda\beta \not\vdash N = x\vec{P}$ since M is unsolvable. By the maximality of n, $N \in \mathbf{O}_0$. Hence, $N \in \mathbf{PO}_0$. ∎

$\mathbf{PO}_n$: *Unsolvables of Order n*

By abuse of language and for consistency with the case $n = 0$, an unsolvable term of order n, M will be called a $\mathbf{PO}_n$-term, denoted $M \in \mathbf{PO}_n$; also we decree that $\mathbf{PO}_\infty \overset{\text{def}}{=} \boldsymbol{O}_\infty$. Hence, we have

$$\{\text{unsolvable terms}\} = \bigcup_{n \in \omega + 1} \mathbf{PO}_n.$$

Note that $\forall n \in \omega. \mathbf{PO}_n \subset \mathbf{O}_n$. In view of the above results, we call a term that is not strongly unsolvable a *weakly solvable term.*

Lazy Lambda Theory and λl

The theory of *sensible* λ-calculus identifies all unsolvable terms and regards them as the meaningless terms. This theory is beautifully presented in Barendregt's book on λ-calculus (Bar84). We have argued earlier on that the sensible theory does not match up well with the lazy semantics of most functional programming languages. In this subsection, we develop a theory of *weakly sensible* λ-theories, i.e., identifying only all the *strongly unsolvable terms.* At this point, the reader should refresh his or her memory of the basic notions of λ-theories as introduced, e.g. , in (Bar84, Chap. 4).

DEFINITION 2.3.7. Define, for $n \in \omega + 1$,

$$\mathscr{PO}_n \overset{\text{def}}{=} \{M = N : M, N \in \mathbf{PO}_n\},$$

$$\mathscr{PO} \overset{\text{def}}{=} \mathsf{Th}\left(\bigcup_{n \in \omega + 1} \mathscr{PO}_n\right),$$

where $\mathsf{Th}(-)$ is the closure operation with respect to provability in $\lambda\beta$. Let $\mathscr{T}$ be a λ-theory. We write $\mathscr{T} + (P = Q)$ for $\mathsf{Th}(\mathscr{T} \cup \{P = Q\})$.

Remark 2.3.8. Strictly speaking, we should consider only closed terms, i.e., $M \in \mathbf{PO}^o_m =^{\text{def}} \mathbf{PO}_m \cap \Lambda^o$ in the preceding definition, as a λ-theory is usually understood to be a collection of equations between closed terms closed under provability. Our extension to open terms is natural in the light of the *invariance* of $\mathbf{PO}_n$-terms under substitution; i.e.,

$$\forall m \in \omega + 1 . M \in \mathbf{PO}_m \Rightarrow \forall P \in \Lambda . M[P/x] \in \mathbf{PO}_n.$$

DEFINITION 2.3.9. (i) Let $\mathscr{T}$ be a λ-theory. $\mathscr{T}$ is *zero sensible* if $\mathsf{Th}(\mathscr{P}\mathscr{O}_0) \subseteq \mathscr{T}$; i.e., $\mathscr{T}$ equates all the strongly unsolvable terms.

(ii) $\mathscr{T}$ is *finitely sensible* if $\mathscr{T}$ equates all unsolvable λ-terms of *finite* orders.

(iii) $\mathscr{T}$ is *pre-lazy* if $\mathscr{P}\mathscr{O} \subseteq \mathscr{T}$.

(iv) A *fully lazy λ-theory* $\mathscr{T}$ is a pre-lazy λ-theory which equates any unsolvable terms iff they have the same order; i.e.,

$$\forall m, n \in \omega + 1 . \forall M \in \mathbf{PO}_m . \forall N \in \mathbf{PO}_n . \mathscr{T} \vdash M = N \Leftrightarrow m = n.$$

EXAMPLE 2.3.10. Denote the λ-theories induced by Longo trees (Lon83) and Böhm trees (Böh68, Bar84) by $\mathscr{L}\mathscr{T}$ and $\mathscr{B}\mathscr{T}$ respectively; then

Properties	$\mathscr{B}\mathscr{T}$	$\mathscr{L}\mathscr{T}$	λI
sensible	√		
semi-sensible	√	√	√
zero-sensible	√	√	√
finitely-sensible	√		
pre-lay	√	√	√
fully-lazy		√	√

We shall see later $\mathscr{L}\mathscr{T} \subseteq \lambda I$. The inclusion is in fact *strict*. Consider the equations $xx = x(\lambda y . xy)$, $x(x(\mathbf{K}\Omega)\Omega)(\mathbf{K}\Omega) = x(\lambda y . x(\mathbf{K}\Omega)\Omega y)(\mathbf{K}\Omega)$; they are valid in λI but not in $\mathscr{L}\mathscr{T}$. The proof that the second equation is valid in λI is a non-trivial exercise.

$\mathscr{P}\mathscr{O}$ is consistent because $\bigcup_{n \in \omega + 1} \mathscr{P}\mathscr{O}_n \subset \mathscr{H}$—the least λ-theory obtained by identifying all unsolvables—and that $\mathscr{H}$ is consistent. See (Bar84) for proofs of the consistency of $\mathscr{H}$.

LEMMA 2.3.11. *Let $\mathscr{T}$ be a zero sensible λ-theory which equates all $\mathbf{O}_\infty$-terms; then $\mathscr{T}$ is pre-lazy.*

Proof. It remains to show that $\mathscr{T}$ equates any two terms of the same non-zero finite order of unsolvability. Let $M, N \in \mathbf{PO}_{n+1}$. Then

$$M =_\beta \lambda x_1 \cdots x_{n+1}.M', \qquad N =_\beta \lambda x_1 \cdots x_{n+1}.N',$$

and $M', N' \in \mathbf{PO}_0$. But $M' =_{\mathscr{T}} N'$; the result then follows from the fact that $\mathscr{T}$ is a congruence. ∎

PROPOSITION 2.3.12. *A pre-lazy λ-theory $\mathscr{T}$ is either fully lazy or finitely sensible. Clearly, a sensible λ-theory is trivially finitely sensible.*

Proof. Let Ω_n be the representative $\mathbf{PO}_n$-term for each $n \in \omega + 1$.

(I) Suppose $\exists m,\ n \in \omega$ and $m \neq n$ such that $\Omega_m = \Omega_n$. Then $\exists k > 0.\Omega_0 = \Omega_k$, since $\forall C \in \Lambda^o.\Omega_{n+1}C = \Omega_n$. This implies that $\forall l.l = ak + b$, where $0 \leqslant b < k$, $\Omega_l = \Omega_b$, since $\lambda x.\Omega_n = \Omega_{n+1}$. Now, because $\Omega_0 = \Omega_0 C = \Omega_k C = \Omega_{k-1}$, we have $\Omega_0 = \Omega_1 = \cdots = \Omega_k$. Hence, $\forall m,\ n \in \omega.\Omega_m = \Omega_n$; i.e., $\mathscr{T}$ is finitely sensible.

(II) Suppose $\neg[\exists m,\ n \in \omega,\ m \neq n.\mathscr{T} \vdash \Omega_m = \Omega_n]$. It remains to show that $\forall n \in \omega.\mathscr{T} \nvdash \Omega_n = \Omega_\infty$; then we are done. Suppose, for a contradiction, that for some $n \geqslant 0.\mathscr{T} \vdash \Omega_n = \Omega_\infty$. In particular, $\mathscr{T} \vdash \mathbf{YK} = \Omega_n$, since $\mathbf{YK} \in \mathbf{PO}_\infty$. But $\mathscr{T} \vdash \mathbf{YK} = \lambda x.\mathbf{YK} = \lambda x.\Omega_n = \Omega_{n+1}$, i.e., $\mathscr{T} \vdash \Omega_n = \Omega_{n+1}$, which contradicts the assumption. ∎

COROLLARY 2.3.13. *Let $\mathscr{T}$ be a pre-lazy λ-theory. If $\mathscr{T} \nvdash \Omega_0 = \Omega_1$ then $\mathscr{T}$ is fully lazy.* ∎

We are now in a position to characterize λl as a λ-theory. λl is *Hilbert–Post complete* w.r.t. fully lazy λ-theories; i.e., it is the *maximal consistent* fully lazy λ-theory.

LEMMA 2.3.14. *λl is a* fully lazy *λ-theory; i.e., for $m, n \in \omega + 1$,*

$$\forall M \in \mathbf{PO}_m.\forall N \in \mathbf{PO}_n.[\lambda l \vdash M = N \Leftrightarrow m = n].$$

Proof. It is easy to see that all strongly unsolvable terms are identified in λl; similarly for $\mathbf{PO}_\infty$-terms. Hence, by Lemma 2.3.11, λl is pre-lazy. Since $\mathbf{PO}_0$ and $\mathbf{PO}_1$ are not provably equal in λl, applying Corollary 2.3.13, we conclude that λl is fully lazy. ∎

The following proposition says that λl is the *maximal* fully lazy λ-theory.

PROPOSITION 2.3.15. *λl is Hilbert–Post complete with respect to the class of fully lazy λ-theories; i.e., for $M, N \in \Lambda^o$ such that $\lambda l \nvdash M = N$,* either *$\lambda l + (M = N)$ is inconsistent or it is not fully lazy.*

Proof. Suppose $\lambda I \not\vdash M = N$. Then, by Proposition 2.2.4,

$$\exists C[\ ] \in \Lambda^{o}.\, C[M]{\downarrow} \,\&\, C[N]{\uparrow}.$$

Suppose $C[M]$ is solvable. Then, by definition, $(\lambda \vec{x}.C\lfloor M \rfloor)\vec{P} = \mathbf{I}$ for some $\vec{x}$, $\vec{P}$; but $(\lambda \vec{x}.\, C[N])\vec{P} \equiv L$ is unsolvable of finite order. Without loss of generality, we consider only the case where the order of unsolvability of L is 0. Suppose $L \in \mathbf{PO}_0$; then

$$\begin{aligned} \lambda I + (M = N) \vdash \mathbf{I} &= (\lambda \vec{x}.C[M])\vec{P} = (\lambda \vec{x}.C[N])\vec{P} \\ &= (\lambda x.xxx)(\lambda x.xxx). \end{aligned}$$

Now, every λ-theory which equates $\mathbf{I}$ and $(\lambda x.xxx)(\lambda x.xxx)$ is inconsistent, for

$$\begin{aligned} \mathbf{I} &= (\lambda x.xxx)(\lambda x.xxx) \\ &= (\lambda x.xxx)(\lambda x.xxx)(\lambda x.xxx) \\ &= \mathbf{I}(\lambda x.xxx) \\ &= (\lambda x.xxx); \end{aligned}$$

i.e., two distinct λI-terms in $\beta\eta$-nf are equated, and so, by Böhm's Theorem (see (Bar84, Theorem 10.5.31)), the theory is inconsistent.

Suppose $C[M]$ is unsolvable. Then $C[M] \in \mathbf{PO}_n$, where $0 \neq n \in \omega + 1$. We have

$$\lambda I + (M = N) \vdash \mathbf{PO}_n \ni C[M] = C[N] \in \mathbf{PO}_0;$$

i.e., $\lambda I + (M = N)$ is not a fully lazy λ-theory. ∎

3. Applicative Transition Systems and Lazy Lambda Models

In this section, we set up the axiomatic framework of our theory based on the notions of (quasi)-*applicative transition system* (q-ats) and (quasi)-*applicative structure with divergence* (q-aswd) which form a bridge both to the standard (i.e., sensible) theory, and to concurrency and other computational notions. We formulate environment, functional, and first order *lazy λ-models* based on q-ats and q-aswd. A subclass of lazy λ-models called *lambda transition systems* (lts) which satisfy a bisimulation extensionality principle will be the main objects of our study in this paper. *Adequate* lts's are shown to be the models of fully lazy λ-theories.

3.1. *Applicative Transition Systems*

The theory λI defined in the previous section was derived from a particular operational model, the transition system $\langle \Lambda^o, \Downarrow \rangle$. What is the general concept of which this is an example?

DEFINITION 3.1.1. (i) *A quasi-applicative transition system* (q-ats) is a structure $\langle A, \mathsf{eval} \rangle$ such that $\mathsf{eval}: A \rightharpoonup A^A$ and $\mathsf{dom}(\mathsf{eval}) \neq A$.

Notation.

$$a \Downarrow f \stackrel{\text{def}}{=} a \in \mathsf{dom}(\mathsf{eval}) \,\&\, \mathsf{eval}(a) = f,$$

$$a\Downarrow \stackrel{\text{def}}{=} a \in \mathsf{dom}(\mathsf{eval}),$$

$$a\Uparrow \stackrel{\text{def}}{=} \neg(a\Downarrow).$$

A q-ats is *pointed* if $\exists \bot \in A.\mathsf{dom}(\mathsf{eval}) = A \setminus \{\bot\}$.

(ii) Let $\langle A, \mathsf{eval} \rangle$ be a q-ats and $\mathsf{Rel}(A) \stackrel{\text{def}}{=} \wp(A \times A)$. Define $F: \mathsf{Rel}(A) \to \mathsf{Rel}(A)$ by

$$F(R) \stackrel{\text{def}}{=} \{(a, b): a \Downarrow f \Rightarrow [b \Downarrow g \,\&\, \forall c \in A.f(c)\, Rg(c)]\}.$$

Then $R \in \mathsf{Rel}(A)$ is called an *applicative bisimulation* if $R \subseteq F(R)$. We define an operational pre-order $\lesssim^B$ as follows. For $a, b \in A$, $a \lesssim^B b$ holds if there is an applicative bisimulation R such that aRb. It follows that

$$\lesssim^B \stackrel{\text{def}}{=} \bigcup \{R \in \mathsf{Rel}(A): R \subseteq F(R)\}$$

and hence is the maximal fixpoint of the monotone function F. Since $\Downarrow$ is a partial function, it is easily shown that the closure ordinal of F is less than or equal to ω. $\lesssim^B$ can thus be described more explicitly as follows:

$$a \lesssim^B b \stackrel{\text{def}}{=} \forall k \in \omega \cdot a \lesssim^B_k b,$$

$$a \lesssim^B_0 b \stackrel{\text{def}}{=} \mathsf{true},$$

$$a \lesssim^B_{k+1} b \stackrel{\text{def}}{=} a \Downarrow f \Rightarrow [b \Downarrow g \,\&\, \forall c \in A.f(c) \lesssim^B_k g(c)].$$

We abbreviate $a \lesssim^B b \,\&\, b \lesssim^B a$ as $a \sim^B b$. It is easily checked that $\lesssim^B$ is a preorder; hence, $\sim^B$ is an equivalence relation. Denote the equivalence class of a as $[a]$.

(iii) An *applicative transition system* (ats) is a q-ats $\mathscr{A} = \langle A, \mathsf{eval} \rangle$ satisfying

$$\text{(ats)} \qquad \forall a, b, c \in A.\, [a \Downarrow f \;\&\; b \precsim^B c] \Rightarrow f(b) \precsim^B f(c).$$

An ats has a quotient $\mathscr{A}/\sim^B =^{\text{def}} \langle A/\sim^B, \mathsf{eval}/\sim^B \rangle$, where

$$(\mathsf{eval}/\sim^B)[a] \stackrel{\text{def}}{=} \begin{cases} [b] \mapsto [f(b)] & \text{if } a \Downarrow f, \\ \text{undefined} & \text{otherwise.} \end{cases}$$

Recall that for any (total) applicative structure $\langle D, \cdot \rangle$, there is an associated structure, $\langle D, \mathsf{Fun} \rangle$, where $\mathsf{Fun}: D \to D^D$ such that $\forall d, e \in D.\, \mathsf{Fun}(d)e = d \cdot e$. In fact, the two structures are inter-definable. In the same way, we show how applicative transition systems are related to the class of structures known as *applicative structure with divergence.*

DEFINITION 3.1.2. (i) A *quasi-applicative structure with divergence* (q-aswd) is a structure $\langle A, \cdot, \Uparrow \rangle$ such that $\langle A, \cdot \rangle$ is an applicative structure, and $\varnothing \neq \Uparrow \subseteq A$ is a *divergence predicate* satisfying

$$\text{(left-strictness)} \qquad \forall x \in A.\, x\Uparrow \Rightarrow \forall y \in A.\, x \cdot y\Uparrow.$$

A q-aswd is *pointed* if $\Uparrow = \{\bot\}$, a singleton set.

Given $\langle A, \cdot, \Uparrow \rangle$ we can define a preorder $\precsim^A$ with the recursive specification

$$a \precsim^A b \stackrel{\text{def}}{=} a\Downarrow \Rightarrow [b\Downarrow \;\&\; \forall c \in A.\, a \cdot c \precsim^A b \cdot c]$$

as the maximal fixpoint of a monotone function as before, where $\Downarrow =^{\text{def}} \neg(\Uparrow)$.

(ii) An *applicative structure with divergence* (aswd)$\langle A, \cdot, \Uparrow \rangle$ is a q-aswd that satisfies

$$\text{(aswd)} \qquad \forall a, b, c \in A.\, b \precsim^A c \Rightarrow a \cdot b \precsim^A a \cdot c.$$

The associated quotient is denoted as $\mathscr{A}/\sim^A$. Since $\Downarrow$ is just the complement in A of $\Uparrow$, we sometimes denote a q-aswd as $\langle A, \cdot, \Downarrow \rangle$ instead of $\langle A, \cdot, \Uparrow \rangle$.

Equivalence of ATS and ASWD

Aswd and ats are essentially equivalent in a sense which we make precise in the following.

Given a q-aswd (aswd) $\mathscr{A} = \langle A, \cdot, \Uparrow \rangle$, define the associated evaluation function $\mathsf{eval}: A \rightharpoonup A^A$ by

$$\mathsf{eval}(a) \stackrel{\text{def}}{=} \begin{cases} x \mapsto a \cdot x & \text{if } a{\downarrow}; \\ \text{undefined} & \text{otherwise.} \end{cases}$$

Clearly, the structure $\mathscr{A}^{\text{q-ats}} =^{\text{def}} \langle A, \mathsf{eval} \rangle$ thus defined is a q-ats and associated bisimulation preorders of $\mathscr{A}$ and $\mathscr{A}^{\text{q-ats}}$ coincide. Also, the axiom (aswd) implies (ats); hence $\mathscr{A}^{\text{ats}} =^{\text{def}} \langle A, \mathsf{eval} \rangle$ is an ats if $\mathscr{A}$ is an aswd.

Conversely, given a q-ats $\mathscr{A} = \langle A, \mathsf{eval} \rangle$, define an associated binary operation $(-) \cdot (-): A \times A \to A$ by

$$a \cdot b \stackrel{\text{def}}{=} \begin{cases} f(b) & \text{if } \mathsf{eval}(a) = f, \\ \text{some } d \text{ such that } d \notin \mathsf{dom}(\mathsf{eval}) & \text{otherwise;} \end{cases}$$

and define $\Uparrow =^{\text{def}} A \setminus \mathsf{dom}(\mathsf{eval})$. Observe that the respective bisimulation preorders coincide and that the axiom (ats) implies (aswd). It is clear that $\mathscr{A}^{\text{q-aswd}} =^{\text{def}} \langle A, \cdot, \Uparrow \rangle$ is a q-aswd; and $\mathscr{A}^{\text{aswd}} =^{\text{def}} \langle A, \cdot, \Uparrow \rangle$ is an aswd if $\mathscr{A}$ is an ats. In fact, it is easy to see $(\mathscr{A}^{\text{q-aswd}})^{\text{q-ats}} = \mathscr{A}$, and $(\mathscr{A}^{\text{aswd}})^{\text{ats}} = \mathscr{A}$ if $\mathscr{A}$ is a q-ats and an ats, respectively.

However, for a q-aswd $\mathscr{A} = \langle A, \cdot, \Uparrow \rangle$, we only have $(\mathscr{A}^{\text{q-ats}})^{\text{q-aswd}} = \mathscr{A}$ and $(\mathscr{A}^{\text{ats}})^{\text{aswd}} = \mathscr{A}$ *provided* $\Uparrow = \{\bot\}$; i.e., if $\mathscr{A}$ is pointed. To summarize,

PROPOSITION 3.1.3. (i) *Given a q-ats* $\mathscr{A}$, $\mathscr{A}^{\text{q-aswd}}$ *as defined above is a q-ats; similarly, if* $\mathscr{A}$ *is an ats.*

(ii) *Given a q-aswd* $\mathscr{A}'$, $\mathscr{A}'^{\text{q-ats}}$ *as defined above is a q-ats. Similarly, if* $\mathscr{A}'$ *is an aswd.*

(iii) *Moreover, we always have* $(\mathscr{A}^{\text{q-aswd}})^{\text{q-ats}} = \mathscr{A}$; *but* $(\mathscr{A}'^{\text{q-ats}})^{\text{q-aswd}} = \mathscr{A}'$ *holds* provided $\Uparrow = \{\bot\}$, *i.e.,* $\mathscr{A}'$ *is pointed.*

(iv) *Let* $\mathscr{A}$ *be an ats and* $\mathscr{A}'$ *be an aswd. Then*

$$([\mathscr{A}/\sim]^{\text{aswd}})^{\text{ats}} = [\mathscr{A}/\sim]; \qquad ([\mathscr{A}'/\sim]^{\text{ats}})^{\text{aswd}} = [\mathscr{A}'/\sim]. \quad \blacksquare$$

From now on, we denote the bisimulation preorder associated with a q-aswd as $\precsim^B$.

3.2. *Lazy λ-Models*

Now, we are in a position to formulate (essentially equivalent) structures which we propose as candidates for the notion of a *lazy λ-model.* Following (Bar84, Koy84, Mey82), we describe them as *environment, functional,* and *first order lazy λ-models,* respectively.

Before we present the definition, let us recall some standard notions:

• Let $\langle A, \cdot \rangle$ be an applicative structure and $X \subseteq A$. A representable function $f \in A^A$ is said to be *X-representable* if f has a representative in X; i.e., $\exists d \in X \subseteq A. \forall a \in A. d \cdot a = f(a)$. Denote $[A \to_X A]$ as the class of all X-representable functions from A to A.

• Let $[\![-]\!] : \Lambda(\underline{A})^o \to A$ be an interpretation of closed λ-terms in an applicative structure A. A function $f \in A^A$ is *λ-definable algebraic* if $\exists M \in \Lambda(\underline{A})^o. \forall a \in A. f(a) = [\![M]\!] \cdot a$.

• Recall that for $M \in \Lambda^o$, $M\Downarrow \Leftrightarrow \exists N \in \Lambda. \lambda\beta \vdash M = \lambda x.N$.

DEFINITION 3.2.1. We present three formulations of *lazy λ-model*.

(I) Environment Lazy λ-Model $\mathscr{A} = \langle A, \cdot^{\mathscr{A}}, \Downarrow_{\mathscr{A}}, [\![-]\!]^{\mathscr{A}}_{-} \rangle$.

(1) $\langle A, \cdot^{\mathscr{A}}, \Downarrow_{\mathscr{A}} \rangle$ is a q-aswd.

(2) $[\![-]\!]^{\mathscr{A}}_{-}$ *preserves* application; i.e., $\forall M, N \in \Lambda(\underline{A}), \forall a \in A$,

$$[\![\underline{a}]\!]^{\mathscr{A}}_{\rho} = a,$$
$$[\![x]\!]^{\mathscr{A}}_{\rho} = \rho(x),$$
$$[\![MN]\!]^{\mathscr{A}}_{\rho} = [\![M]\!]^{\mathscr{A}}_{\rho} \cdot^{\mathscr{A}} [\![N]\!]^{\mathscr{A}}_{\rho}.$$

(3) $\mathscr{A} \models (\beta)$; i.e., $\lambda\beta \vdash M = N \Rightarrow \mathscr{A} \models M = N$.

(4) $\mathscr{A} \models (\xi)$, where (ξ) is

$$\forall M, N \in \Lambda(\underline{A}). \forall \rho. \forall a \in A. [\![M]\!]^{\mathscr{A}}_{\rho[x \mapsto a]} = [\![N]\!]^{\mathscr{A}}_{\rho[x \mapsto a]}]$$
$$\Rightarrow [\![\lambda x.M]\!]^{\mathscr{A}}_{\rho} = [\![\lambda x.N]\!]^{\mathscr{A}}_{\rho}.$$

(5) $\forall M \in \Lambda^o. M\Downarrow \Rightarrow \mathscr{A} \models M\Downarrow \overset{\text{def}}{=} [\![M]\!]^{\mathscr{A}}\Downarrow_{\mathscr{A}}$.

$\mathscr{A}$ is *pointed* if the underlying q-aswd is pointed; and *monotone* if it is an aswd.

(II) Functional Lazy λ-Model $\mathscr{A} = \langle A, \mathsf{Fun}, \mathsf{Gr}, [\![-]\!]^{\mathscr{A}}_{-} \rangle$.

(1) $\langle A, \mathsf{Fun} \rangle$ is a q-ats;

(2) $A \xrightarrow{\mathsf{Fun}} [A \to_{\Downarrow} A] \xrightarrow{\mathsf{Gr}} A$ with $\mathsf{Fun} \circ \mathsf{Gr} = \mathsf{id}_{[A \to_{\Downarrow} A]}$, such that $\mathsf{range}(\mathsf{Gr}) \cap \Uparrow_{\mathscr{A}} = \varnothing$, where the representable function space is defined with respect to $\langle A, \mathsf{Fun} \rangle^{\text{q-aswd}}$ (note that (2) implies that $[A \to_{\Downarrow} A]_{\perp} \lhd A$; i.e., $[A \to_{\Downarrow} A]_{\perp}$ is a retract of A.)

(3) $[\![-]\!]^{\mathscr{A}}_{-} : \Lambda(\underline{A}) \times A^{\mathsf{Var}} \to A$ is defined as follows: $\forall M, N \in \Lambda(\underline{A})$, $\forall a \in A$,

(a) $[\![-]\!]^{\mathscr{A}}$ *preserves application*:

$$[\![\underline{a}]\!]^{\mathscr{A}}_{\rho} \stackrel{\text{def}}{=} a,$$

$$[\![x]\!]^{\mathscr{A}}_{\rho} \stackrel{\text{def}}{=} \rho(x),$$

$$[\![MN]\!]^{\mathscr{A}}_{\rho} \stackrel{\text{def}}{=} [\![M]\!]^{\mathscr{A}}_{\rho} \cdot^{\mathscr{A}} [\![N]\!]^{\mathscr{A}}_{\rho},$$

where the application "$\cdot^{\mathscr{A}}$" is as defined in $\langle A, \mathsf{Fun}\rangle^{\text{q-aswd}}$;

(b) (weak-extensionality) $[\![\lambda x.M]\!]^{\mathscr{A}}_{\rho} \stackrel{\text{def}}{=} \mathsf{Gr}(d \mapsto [\![M]\!]^{\mathscr{A}}_{\rho[x \mapsto d]})$ such that $[A \to_{\Downarrow} A]$ is closed under λ-*definable algebraic* functions.

$\mathscr{A}$ is *pointed* if the underlying q-ats is pointed; and *monotone* if it is an ats.

(III) First Order Lazy λ-Model $\mathscr{A} = \langle A, \cdot^{\mathscr{A}}, \Downarrow_{\mathscr{A}}, \varepsilon, [\![-]\!]^{\mathscr{A}}\rangle$.

(1) $\langle A, \cdot^{\mathscr{A}}, \Downarrow_{\mathscr{A}}\rangle$ is a q-aswd.

(2) $\langle A, \cdot^{\mathscr{A}}\rangle$ is a combinatory algebra such that for $x, y \in A$:

$$\mathscr{A} \models \mathbf{K}x\Downarrow, \mathbf{S}xy\Downarrow.$$

(3) $\mathscr{A}$ satisfies the following axioms:

(ε1) $\quad \forall x, y.(\varepsilon \cdot^{\mathscr{A}} x) \cdot^{\mathscr{A}} y = x \cdot^{\mathscr{A}} y;$

(MS) $\quad \forall x, y.[\forall z.x \cdot^{\mathscr{A}} z = y \cdot^{\mathscr{A}} z] \Rightarrow \varepsilon \cdot^{\mathscr{A}} x = \varepsilon \cdot^{\mathscr{A}} y;$

(stability) $\quad \varepsilon \cdot^{\mathscr{A}} \varepsilon = \varepsilon.$

$\mathscr{A}$ is *pointed* if the underlying q-aswd is pointed; and *monotone* if it is an aswd.

PROPOSITION 3.2.2. *The three formulations of lazy λ-models are essentially equivalent.*

(i) *There are bijective translations between the following classes of structures*:

(a) *pointed environment lazy λ-models,*

(b) *pointed functional lazy λ-models,*

(c) *pointed first order lazy λ-models.*

(ii) *There are bijective translations between the following classes of quotient (w.r.t. bisimulation equivalence) structures*:

(a) *monotone environment lazy λ-models,*

(b) *monotone functional lazy λ-models,*

(c) *monotone first order lazy λ-models.*

Proof. The proof of the Proposition is tedious but straightforward. A version is contained in (Ong88b, Chap. 3). ▮

3.3. *Lambda Transition Systems*

In this subsection, we study the properties of a class of lazy λ-models called *lambda transition systems* (lts's). Lts's are lazy λ-models in which bisimulation equivalent elements are identified. Some rather strong conditional extensionality principles are valid in an lts. We show that lts's are models of the fully lazy λ-theories.

For the rest of the paper, we develop our theory in the axiomatic framework of lts. This is in keeping with the view that two elements are distinguished in the model only if their respective applicative behaviours are distinguishable by some (finite) experiment. As we will see in the sequel, the class of lts's is general; it includes not just models of the lazy λ-calculus, but also programming languages (cast as "syntax-free" operational transition systems) such as the pure lazy language λl.

Let $\mathscr{A} = \langle A, \cdot, \Uparrow \rangle$ be a q-aswd. Define a binary relation $\sim_{\Downarrow} \subseteq A \times A$ as follows:

$$x \sim_{\Downarrow} y \stackrel{\text{def}}{=} [x\Downarrow \Leftrightarrow y\Downarrow] \,\&\, \forall z \in A.\, x \cdot z = y \cdot z.$$

Clearly, $\sim_{\Downarrow}$ is an equivalence relation that relates any two elements either if they are both divergent or if they are both convergent and have the same applicative behaviour. There is a 1–1 correspondence between $[A \to_{\Downarrow} A]_{\perp}$ and $A/\sim_{\Downarrow}$ as follows:

$$\perp \leftrightarrow [\perp]_{\sim_{\Downarrow}},$$
$$\langle 0, f \rangle \leftrightarrow [a]_{\sim_{\Downarrow}} \quad \text{where} \quad \forall z \in A.\, f(z) = a \cdot z.$$

We define a few axioms.

DEFINITION 3.3.1. Let $\mathscr{A}$ be a lazy λ-model:

$$\text{adequacy} \stackrel{\text{def}}{=} \forall M \in \Lambda^o.\, \mathscr{A} \models M\Downarrow \Rightarrow M\Downarrow;$$
$$\eta_{\text{cond}} \stackrel{\text{def}}{=} \forall a \in A.\, \mathscr{A} \models [a\Downarrow \Rightarrow \lambda x.ax = a];$$
$$\text{Ext}_{\text{cond}} \stackrel{\text{def}}{=} \forall x, y \in A.\, x \sim_{\Downarrow} y \Rightarrow x = y.$$

We refer to the last two axioms as the *conditional* η and *conditional* (*strong*) *extensionality axiom* respectively.

LEMMA 3.3.2. *Let* $\mathscr{A} = \langle A, \mathsf{Fun}, \mathsf{Gr}, [\![-]\!]_- \rangle$ *be a functional lazy* λ*-model. Then the following are equivalent*:

(1) $\mathscr{A} \models (\mathrm{Ext}_{\mathrm{cond}})$,

(2) $\mathsf{Gr} \circ \mathsf{Fun} = \mathsf{id}_{\Downarrow_A}$ *which implies that* $A \cong [A \to_{\Downarrow} A]_{\perp}$,

(3) $\mathscr{A} \models (\eta_{\mathrm{cond}})$.

Proof. (1) $\Leftrightarrow$ (2): First observe that

$$\forall x, y \in \Downarrow . x \sim_{\Downarrow} y \Leftrightarrow \mathsf{Fun}(x) = \mathsf{Fun}(y).$$

Suppose (1). Since $\mathsf{Fun} \circ \mathsf{Gr} = \mathsf{id}_{[A \to_{\Downarrow} A]}$—by definition of a functional lazy λ-model—for any convergent x, $\mathsf{Fun}(\mathsf{Gr}(\mathsf{Fun}(x))) = \mathsf{Fun}(x)$, and so $\mathsf{Gr}(\mathsf{Fun}(x)) \sim_{\Downarrow} x$ by the preceding observation. By $(\mathrm{Ext}_{\mathrm{cond}})$, we have $\forall x \Downarrow . \mathsf{Gr}(\mathsf{Fun}(x)) = x$; i.e., the assertion (2). Suppose (2). Then, for $x \Downarrow \,\&\, y \Downarrow$,

$$x \sim_{\Downarrow} y \Leftrightarrow \mathsf{Fun}(x) = \mathsf{Fun}(y) \Rightarrow \mathsf{Gr}(\mathsf{Fun}(x)) = \mathsf{Gr}(\mathsf{Fun}(y)).$$

Applying (2), we get $x = y$.

(2) $\Leftrightarrow$ (3) follows from the observation that for convergent a,

$$[\![\lambda x . \underline{a} x]\!] = \mathsf{Gr}(\mathsf{Fun}(a)). \quad \blacksquare$$

DEFINITION 3.3.3. An aswd $\mathscr{A} = \langle A, \cdot, \Uparrow \rangle$ (or an ats$\langle A, \mathsf{eval} \rangle$) satisfies the *bisimulation extensionality principle* if the following axiom is satisfied:

$$(\mathrm{Ext}_{\mathrm{bisim}}) \qquad \forall a, b \in A . a \sim^B b \Rightarrow a = b.$$

DEFINITION 3.3.4 (Lambda Transition System). A *lambda transition system* is a monotone functional lazy λ-model

$$\mathscr{A} = \langle A, \mathsf{Fun}, \mathsf{Gr}, [\![-]\!]_- \rangle \qquad \text{such that } \mathscr{A} \models (\mathrm{Ext}_{\mathrm{bisim}}).$$

Clearly, any monotone functional lazy λ-model $\mathscr{A}$ uniquely determines an lts, namely, $\mathscr{A}/\sim^B$.

Remark 3.3.5. (i) Let $\mathscr{A} = \langle A, \mathsf{Fun} \rangle$ be an ats. It is easy to see that

$$\mathscr{A} \models (\mathrm{Ext}_{\mathrm{bisim}}) \Rightarrow \mathscr{A} \models (\mathrm{Ext}_{\mathrm{cond}});$$

the converse is not true in general. Hence, with reference to Lemma 3.3.2, if $\mathscr{A}$ is an lts, then condition (3) above implies that

$$\mathscr{A} \models (\eta_{\mathrm{cond}}), (\mathrm{Ext}_{\mathrm{cond}}) \,\&\, \mathsf{Gr} \circ \mathsf{Fun} = \mathsf{id}_{\Downarrow_A}.$$

(ii) In view of the equivalence between monotone environment and monotone functional lazy λ-models (Proposition 3.2.2), a lambda transition system may be defined equivalently as a monotone environment lazy λ-model $\mathscr{A} = \langle A, \cdot, \Downarrow, [\![-]\!]\ _\ \rangle$ such that $\mathscr{A} \models (\mathrm{Ext}_{\mathrm{bisim}})$.

(iii) Our formulation of lts is essentially equivalent to that in (Abr87b). The only difference is that in our formulation, bisimulation equivalent elements are identified *axiomatically.*

PROPOSITION 3.3.6. *Let* $\mathscr{A} = \langle A, \mathsf{Fun}, \mathsf{Gr}, [\![-]\!]\ _\ \rangle$ *be an lts. Then, with respect to the bisimulation preorder,*

(i) $\mathscr{A}$ *has unique least and greatest elements denoted* $\bot$ *and* $\top$, *respectively. The top element* $\top$ *is the interpretation of the* $\mathbf{PO}_\infty$*-elements.*

(ii) $\bot$ *and* $\top$ *are the only solutions, divergent respectively, of the following equation*:

$$\forall a \in A.\, x \cdot a = x.$$

Proof. It is easy to see that $\bot$, the only divergent element, is the least element w.r.t. $\lesssim^B$, and satisfies the equation in (ii). The top element is that $x \in A$ such that $\forall y \in A.\, x\vec{y}\Downarrow$. By $(\mathrm{Ext}_{\mathrm{bisim}})$, such x is unique.

Claim. If x is a convergent solution to $\forall a \in A.\, x \cdot a = x$, then x is the greatest element with respect to $\lesssim^B$.

By definition, $M \in \mathbf{PO}_\infty \Rightarrow MN \in \mathbf{PO}_\infty$. Since all $\mathbf{PO}_\infty$-terms have the same denotation in $\mathscr{A}$, we have $\forall x \in A.\, lx = l$ with l being the denotation of $\mathbf{PO}_\infty$-terms. Since $l\Downarrow$, we have $\forall \vec{y} \subseteq A.\, l\vec{y}\Downarrow$; and so, $l = \top$. ∎

Under the hypothesis of *adequacy*, the least element $\bot$ in an lts is the denotation of all strongly unsolvable terms. In fact, we can say more:

COROLLARY 3.3.7. *If an lts* $\mathscr{A}$ *is* adequate, *then* $\mathscr{A}$ *is a* fully lazy λ-model, *i.e., for* $m, n \in \omega + 1$,

$$\forall M \in \mathbf{PO}_m, \forall N \in \mathbf{PO}_n.\, [\mathscr{A} \models M \sqsubseteq N \Leftrightarrow m \leqslant n].$$

Proof. It suffices to show:

(a) $\forall n \in \omega + 1.\, \forall M, N \in \mathbf{PO}_n.\, \mathscr{A} \models M = N$.

(b) $\forall M \in \mathbf{PO}_n.\, \forall N \in \mathbf{PO}_{n+1}.\, \forall L \in \mathbf{PO}_\infty.\, \mathscr{A} \models M \sqsubset N \sqsubset L$, where $\mathscr{A} \models M \sqsubset N$ means $[\![M]\!] \lesssim^B [\![N]\!]$ & $[\![M]\!] \neq [\![N]\!]$.

(a) follows from the hypothesis of adequacy, (i) of the Proposition, and Proposition 2.3.6. Now, (a) means that the λ-theory induced by its lts $\mathscr{A}$ is *pre-lazy.* (b) then follows from Corollary 2.3.13 and inductively from the

fact that the denotations of the $\mathbf{PO}_{\mathcal{X}}$-terms and the $\mathbf{PO}_0$-terms are the greatest and least elements w.r.t. $\precsim^B$, respectively. ▮

It follows that any adequate lts induces a fully lazy λ-theory. Is every fully lazy λ-theory induced by an adequate lts? In other words, are adequate lts's *complete* with respect to fully lazy λ-theories? We return to this question later in the sequel.

EXAMPLE 3.3.8. (i) D, the initial solution of the domain equation $D \cong [D \to D]_\perp$ in the category of cpo's and continuous functions, is an lts. In fact, it is *adequate* and *internally fully abstract*; i.e.,

$$\forall x, y \in D. x \precsim^B y \Leftrightarrow x \sqsubseteq y,$$

where $\precsim^B$ is the associated bisimulation preorder of the q-aswd D.

(ii) $(\Lambda^o/\sim^B, \Downarrow)$ is an lts. That the quotient is well-defined—the validity of the axiom (aswd)—follows from Proposition 2.2.4.

4. Basic Properties of the Canonical Model D

This section studies the basic properties of D—the initial solution of the domain equation $D \cong [D \to D]_\perp$—which we regard as the canonical model for the family of lazy languages (see next section for justification). D is an *internally fully abstract*, *adequate* lts whose application operation left-preserves *arbitrary* joins. Some index calculation results are presented. The projection maps $\psi_n : D \to D_n$, where D_n is the nth approximant of D, are shown to be $\lambda\mathsf{C}$-*definable*, where C is a formal constant that (semi)-tests for convergence.

4.1. *Introduction*

In the *standard* theory, λ-calculus may be regarded as being characterized by the type equation

$$D = [D \to D].$$

Regarded as a domain equation, it has non-trivial solutions in many categories including those of cpo's, Scott domains, and complete lattices (all these with continuous functions as morphisms). Note, however, that the initial solutions in each of the above categories is trivial—the one-point domains.

In the *lazy* theory, a distinction is drawn between *convergent* elements, i.e., those which evaluate to functions from D to D, and *divergent* elements

whose evaluations do not terminate, i.e., those devoid of any functional (or *operator* as opposed to *operand*) content. That is to say, the following structure map is *partial*:

$$\mathsf{eval}: D \rightharpoonup [D \to D].$$

The standard approach to partial maps in domain theory is to make them into total ones by sending undefined arguments to a "bottom element," thereby changing the type of eval to

$$\mathsf{eval}: D \to [D \to D]_\perp.$$

Some Basic Definitions

Let D, E be cpo's. We say that $\langle i, j \rangle$ is an *embedding* of D into E, denoted $D \trianglelefteq E$, if i, j are continuous maps $D \overset{i}{\rightarrowtail} E \overset{j}{\twoheadrightarrow} D$ and if

$$i \circ j \sqsubseteq \mathsf{id}_E \qquad \text{and} \qquad j \circ i = \mathsf{id}_D.$$

i is the left-adjoint of j and each uniquely determines the other.

Recall the category-theoretic characterization of lifting as the left adjoint to the forgetful functor U,

$$\mathbf{CPO} \xrightarrow{(-)_\perp} \mathbf{CPO}_\perp \xrightarrow{U} \mathbf{CPO},$$

where $\mathbf{CPO}_\perp$ is the sub-category of strict functions. As is standard, we have:

- A natural transformation: $\mathsf{up} : I_{\mathbf{CPO}} \to U \circ (-)_\perp$.
- For each continuous function $f: D \to UE$, its adjoint

$$\mathsf{lift}(f): (D)_\perp \to_\perp E.$$

Concretely, we have, for $x, y \in D$:

$$(D)_\perp \overset{\text{def}}{=} \{\perp\} \cup \{\langle 0, d \rangle : d \in D\},$$

$$x \sqsubseteq y \overset{\text{def}}{=} x = \perp \qquad \text{or} \qquad [x = \langle 0, d \rangle \ \&\ y = \langle 0, d' \rangle \ \&\ d \sqsubseteq_D d'],$$

$$\mathsf{up}_D(d) \overset{\text{def}}{=} \langle 0, d \rangle,$$

$$\mathsf{lift}(f)(\perp) \overset{\text{def}}{=} \perp_E,$$

$$\mathsf{lift}(f)\langle 0, d \rangle \overset{\text{def}}{=} f(d).$$

Let $\mathsf{dn}_D = {}^{\text{def}} \mathsf{lift}(\mathsf{id}_D) : D_\perp \to_\perp D$. Note that $\mathsf{dn}_{[D \to D]}(\langle 0, f \rangle) = f$ for $f \in [D \to D]$.

LEMMA 4.1.1. *Suppose* $\langle i, j\rangle$ *is an embedding of* D *into* D'. *Then* $\langle i^*, j^*\rangle$, *which is defined as*

$$i^* \stackrel{\text{def}}{=} \lambda f \in [D \to D]_\perp \cdot \mathsf{C}^{[D \to D]_\perp}_{[D' \to D']_\perp} f \mathsf{up}_{[D' \to D']}[\lambda y \in D'.i \circ \mathsf{dn}_{[D \to D]}(f) \circ j(y)],$$

$$j^* \stackrel{\text{def}}{=} \lambda g \in [D' \to D']_\perp \cdot \mathsf{C}^{[D' \to D']_\perp}_{[D \to D]_\perp} g \mathsf{up}_{[D \to D]}[\lambda y \in D.j \circ \mathsf{dn}_{[D' \to D']}(g) \circ (y)],$$

is an embedding of $[D \to D]_\perp$ *into* $[D' \to D']_\perp$, *where* $\mathsf{C}^X_Y: X \times Y \to Y$ *is defined as*

$$\mathsf{C}^X_Y(x, y) \stackrel{\text{def}}{=} \begin{cases} \perp_Y & \text{if } x = \perp_X; \\ y & \text{otherwise.} \end{cases}$$

Proof. Clearly i^* and j^* are continuous. Also, it is straightforward to check that $i^* \circ j^* \sqsubseteq \mathsf{id}_{[D' \to D']_\perp}$ and $j^* \circ i^* = \mathsf{id}_{[D \to D]_\perp}$. ∎

4.2. *Construction of the Initial Solution*

Define $D_0 \stackrel{\text{def}}{=} 1$ (the one-point domain) and $i_0: D_0 \to [D_0 \to D_0]_\perp$ so that $i_0(\perp) = \perp$, and inductively,

$$D_{n+1} \stackrel{\text{def}}{=} [D_n \to D_n]_\perp, \qquad D_n \overset{i_n}{\rightarrowtail} D_{n+1} \overset{j_n}{\twoheadrightarrow} D_n,$$

$$\langle i_{n+1}, j_{n+1}\rangle \stackrel{\text{def}}{=} \langle i_n^*, j_n^*\rangle.$$

Writing out the embedding pairs in full, we have

$$i_{n+1} \stackrel{\text{def}}{=} \lambda f \in D_{n+1}.\mathsf{C}^{D_{n+1}}_{D_{n+2}} f \; \mathsf{up}_{[D_{n+1} \to D_{n+1}]}[\lambda y \in D_{n+1}.i_n \circ \mathsf{dn}_{[D_n \to D_n]}(f) \circ j_n(y)],$$

$$j_{n+1} \stackrel{\text{def}}{=} \lambda g \in D_{n+2}.\mathsf{C}^{D_{n+2}}_{D_{n+1}} g \; \mathsf{up}_{[D_n \to D_n]}[\lambda y \in D_n.j_n \circ \mathsf{dn}_{[D_{n+1} \to D_{n+1}]}(g) \circ i_n(y)].$$

$\langle D_n, j_n\rangle_{n \in \omega}$ is an inverse system of cpo's which are ω-algebraic complete lattices.

We define the *inverse limit*, $D = \lim_{\leftarrow} \langle D_n, j_n\rangle_{n \in \omega}$, as follows: Define, as is standard, $\phi_{m,n}: D_n \to D_m$ by

$$\begin{aligned} m \geqslant n \qquad & \phi_{n,n} = \lambda x \in D_n.x, \\ & \phi_{m+1,n} = \phi_{m,n} \circ j_m; \\ m < n \qquad & \phi_{m,n+1} = i_n \circ \phi_{m,n}. \end{aligned}$$

Identify the initial solution $D \subseteq \prod_{n \in \omega} D_n$ as

$$D = \{\langle x_n : n \in \omega \rangle : x_n \in D_n \ \& \ j_n(x_{n+1}) = x_n\}$$

and name the isomorphism pair as

$$D \xrightarrow{\text{Fun}} [D \to D]_\perp \xrightarrow{\text{Gr}} D;$$

the precise definitions of Fun ("function") and Gr ("graph") will be spelt out in full in the sequel.

D as an Applicative Transition System

D is a quasi-applicative transition system with the associated evaluation map $\text{eval}: D \rightharpoonup D^D$ defined as

$$\text{eval}(d) \stackrel{\text{def}}{=} \begin{cases} f & \text{if} \quad \text{Fun}(d) = \langle 0, f \rangle, \\ \text{undefined} & \text{if} \quad \text{Fun}(d) = \perp. \end{cases}$$

Thus, we may write $d \Downarrow f$ and $d \Uparrow$ etc. Equivalently, we may recast the above as a quasi-applicative structure with divergence, $\langle D, \cdot, \Uparrow \rangle$ where

$$d \cdot e \stackrel{\text{def}}{=} \begin{cases} f(e) & \text{if} \quad d \Downarrow f, \\ \perp & \text{if} \quad d \Uparrow. \end{cases}$$

We shall see later that D is an ats; i.e., it satisfies the axiom (ats). This is a consequence of D being *internally fully abstract.*

In the following, we work out the definition of the application operation in terms of projections onto approximants.

Notation 4.2.1. For simplicity, henceforth, we write the *n-projection map* $\phi_{\infty,n}$ as ψ_n and the *n-injection map* $\phi_{n,\infty}$ as ϕ_n. Also, we regard each D_n as a subset of D; i.e., we identify $\phi_n(x)$, where $x \in D_n$, with x; and for $x \in D$, $\psi_n(x) = x_n \in D_n$. Hence,

$$D = \bigcup_{n \in \omega} D_n.$$

4.3. *Index Calculations and Extensionality Properties*

We unpack the structure of D somewhat and highlight some basic index-calculation results. These are used to define the Fun and Gr functions of D. Finally, we present some extensionality properties satisfied by D. These results are similar to those obtained in (Wad76).

LEMMA 4.3.1. *Let* $x \in D$. *Then*

(i) $(x_n)_m = x_{\min(n,m)}$.

(ii) *If* $n \leqslant m$, *then* $x_n \sqsubseteq x_m \sqsubseteq x$.

(iii) $x = \bigsqcup_n x_n$.

(iv) *If* $x \in D_n$, *then* $\forall m \geqslant n . x_m = x$.

Proof. We omit the largely straightforward proofs. ∎

COROLLARY 4.3.2. *Let* $d \in D$. *Then* (*note that in* D, $d\Uparrow \Leftrightarrow d = \bot$)

(i) $d\Uparrow \Leftrightarrow \forall n \geqslant 0 . d_n \Uparrow$.

(ii) $d\Downarrow \Leftrightarrow \forall n \geqslant 1 . d_n \Downarrow$. ∎

DEFINITION 4.3.3. Define, for $n \in \omega$, $\mathsf{Ap}_n^{\perp} : [D_n \to D_n]_{\perp} \times D_n \to D_n$ by

$$\mathsf{Ap}_n^{\perp}(f, d) \stackrel{\text{def}}{=} \begin{cases} \bot & \text{if } f = \bot, \\ g(d) & \text{if } f = \langle 0, g \rangle \text{ with } g \in [D_n \to D_n]. \end{cases}$$

In the following, however, we will abbreviate $\mathsf{Ap}_n^{\perp}(f, d)$ simply as $f(d)$.

LEMMA 4.3.4. *For* $n \leqslant k \in \omega$,

(i) $x_{n+1}(y_n) \sqsubseteq x_{k+1}(y_k)$.

(ii) $(x_{n+1})_{k+1}\, y_k = x_{n+1}(y_n)$.

(iii) $(x_{k+1}(y_n)_k)_n = x_{n+1}(y_n)$.

Proof. We refer the reader to (Ong88b, Chap. 4) for the largely straightforward induction proofs. ∎

The application operation $(-)\cdot(-) : D \times D \to D$ is defined as follows:

$$x \cdot y \stackrel{\text{def}}{=} \bigsqcup_{n \in \omega} \mathsf{Ap}_n^{\perp}(x_{n+1}, y_n).$$

It should be clear that "$\cdot$" is continuous. In fact, we can say more, that "$\cdot$" *left-preserves arbitrary joins*, of which more anon,

Note that if $x \in D_{n+1}$, $y \in D_n$, then

$$x \cdot y = x_{n+1} \cdot y_n = x_{n+1}(y_n).$$

PROPOSITION 4.3.5. *Let* $x, y \in D$.

(i) $x_{n+1} \cdot y = x_{n+1} \cdot y_n = (x \cdot y_n)_n$.

(ii) $x_1 \cdot y = \bot$.

(iii) $x_0 \cdot y = x_0 = \bot$.

Proof. The induction proof uses Lemma 4.3.4 and is similar to the proof of the index calculation results of the sensible λ-model D_∞ as presented, say, in (Bar84). See (Ong88b, Chap. 4) for the details. ∎

In the same way as $\mathsf{Ap}_n^\perp$, we define $\mathsf{Ap}^\perp : [D \to D]_\perp \times D \to D$ as

$$\mathsf{Ap}^\perp(f, d) \stackrel{\text{def}}{=} \begin{cases} g(d) & \text{if } f = \langle 0, g \rangle \text{ with } g \in [D \to D], \\ \perp & \text{if } f = \perp. \end{cases}$$

PROPOSITION 4.3.6. *Define, for* $f \in [D \to D]_\perp$

$$\mathsf{rep}(f) \stackrel{\text{def}}{=} \begin{cases} \bigsqcup_{n \in \omega} \mathsf{up}_{[D_n \to D_n]}[\lambda y \in D_n . (\mathsf{Ap}^\perp(f, y))_n] & \text{if } f = \langle 0, g \rangle; \\ \perp & \text{if } f = \perp. \end{cases}$$

Then $\forall y \in D . \mathsf{Ap}^\perp(f, y) = \mathsf{rep}(f) \cdot y$; *in which case, we say that* f *is representable by* $\mathsf{rep}(f)$.

Proof. Note that all joins exist in D. Without loss of generality, suppose $f \neq \perp$. Then

$$\begin{aligned} \mathsf{rep}(f) \cdot y &\stackrel{\text{def}}{=} \bigsqcup_{i \in \omega} \mathsf{rep}(f)_{i+1}(y)_i \\ &= \bigsqcup_{i \in \omega} (\mathsf{rep}(f) \cdot y_i)_i \\ &= \bigsqcup_i \left(\left(\bigsqcup_n \mathsf{up}_{[D_n \to D_n]}[\lambda y \in D_n . (g(y))_n] \right) \cdot y_i \right)_i \\ &= \bigsqcup_{i,n} (\mathsf{up}_{[D_n \to D_n]}[\lambda y \in D_n . (g(y))_n] \cdot y_i)_i \\ &= \bigsqcup_{j \in \omega} (\mathsf{up}_{[D_n \to D_n]}[\lambda y \in D_j . (g(y))_j] \cdot y_j)_j \\ &= \bigsqcup_j (g(y_j))_j \\ &= \bigsqcup_n \bigsqcup_i (g(y_i))_n \\ &= \bigsqcup_i g(y_i) \\ &= g(y). \end{aligned}$$ ∎

We can now define the maps (which determine an isomorphism as well as a homeomorphism) between D and $[D \to D]_\perp$ as

$$D \xrightarrow{\mathsf{Fun}} [D \to D]_\perp \xrightarrow{\mathsf{Gr}} D,$$

where

$$\mathsf{Fun}(d) \stackrel{\text{def}}{=} \begin{cases} \mathsf{up}_D(x \mapsto d \cdot x) & \text{if } d \neq \bot, \\ \bot & \text{otherwise,} \end{cases}$$

and $\mathsf{Gr}(f) =^{\text{def}} \mathsf{rep}(f)$. We leave the verification of $\langle \mathsf{Gr}, \mathsf{Fun} \rangle$ being an isomorphism to the interested reader.

The following corollary states that representable functions of D coincide with its continuous functions.

COROLLARY 4.3.7. *Let $f \in D^D$. Then f is continuous iff f is $\Downarrow$-representable; i.e., $\exists d \in D.d\Downarrow \;\&\; \forall x \in D.f(x) = d \cdot x$.*

Proof. Let $f \in D^D$ be continuous; then the proposition asserts that $\mathsf{rep}(\mathsf{up}_{[D \to D]}(f))$ is a representative of f. Conversely, any *convergent* $d \in D$ is a representative of a continuous map; namely, $\mathsf{dn}_{[D \to D]}(\mathsf{Fun}(d))$. ∎

Extensionality Properties of D

D does not satisfy the strong extensionality principle[3]—just consider $\bot$ and $\bot_1$, where $\bot_1 =^{\text{def}} \mathsf{Gr}(\mathsf{up}_{[D \to D]}(\lambda x.\bot))$.

However, D satisfies a weaker notion which we call

PROPOSITION 4.3.8 (Conditional Strong Extensionality). *For $d, e \in D$,*

$$d\Downarrow \;\&\; e\Downarrow \Rightarrow [\forall x \in D.d \cdot x \sqsubseteq e \cdot x \Leftrightarrow d \sqsubseteq e].$$

Proof. Suppose $d\Downarrow$ and $e\Downarrow$ and $\forall x \in D \cdot d \cdot x \sqsubseteq e \cdot x$. Let $\mathsf{Fun}(d) = \langle 0, f \rangle$ and $\mathsf{Fun}(e) = \langle 0, g \rangle$ with $f, g \in [D \to D]$. Then, $\forall x.f(x) \sqsubseteq g(x)$; and so, $f \sqsubseteq g$, which implies that $\langle 0, f \rangle \sqsubseteq \langle 0, g \rangle$. By the monotonicity of Gr and that $\mathsf{Gr} \circ \mathsf{Fun} = \mathsf{id}$,

$$d = \mathsf{Gr}(\langle 0, f \rangle) \sqsubseteq \mathsf{Gr}(\langle 0, g \rangle) = e.$$

The other direction follows direction from the monotonicity of the application operation. ∎

THEOREM 4.3.9 (Internal Full Abstraction).

$$\forall d, e \in F.d \sqsubseteq e \Leftrightarrow d \sqsubseteq\!\!\sim^{B} e.$$

Proof. Unpacking the definitions, we see that for all $d, e \in D$,

$$d \sqsubseteq e \Leftrightarrow d\Downarrow f \Rightarrow [e\Downarrow g \;\&\; \forall c \in D.f(c) \sqsubseteq g(c)].$$

[3] The strong extensionality principle is the following: $\forall x, y \in D.\forall z \in D.x \cdot z = y \cdot z \Rightarrow x = y]$.

Thus the domain ordering is an applicative bisimulation, and so is included in $\sqsubseteq^B$. For the converse, we prove a stronger statement.

Claim. $\forall d, e \in D. \forall k \in w. d \sqsubseteq^B_k e \Rightarrow d_k \sqsubseteq e_k$ which clearly implies $d \sqsubseteq^B e \Rightarrow d \sqsubseteq e$, since $d = \bigsqcup_{k \in \omega} d_k$. The base $k = 0$ is trivial. Suppose true for k. Without loss of generality, assume $d\Downarrow$ and $e\Downarrow$. Suppose $d \sqsubseteq^B_{k+1} e$. By definition of $\sqsubseteq^B_{k+1}$,

$$\forall f \in D. d \cdot f_k \sqsubseteq^B_k e \cdot f_k.$$

Invoking the induction hypothesis, we have

$$\forall f \in D. (d \cdot f_k)_k \sqsubseteq (e \cdot f_k)_k,$$

which implies $\forall f \in D. d_{k+1} \cdot f \sqsubseteq e_{k+1} \cdot f$. Now, $d_{k+1}\Downarrow$ and $e_{k+1}\Downarrow$, and the result follows from the conditional strong extensionality of D. ∎

As a corollary, we see that D satisfies the axiom (aswd), hence it is an aswd.

D Is a Lambda Transition System

Define an interpretation of λ-terms in D as follows: for $d \in D$ and $M \in \Lambda$,

$$[\![x]\!]_\rho \stackrel{\text{def}}{=} \rho(x),$$

$$[\![\underline{d}]\!]_\rho \stackrel{\text{def}}{=} d,$$

$$[\![MN]\!]_\rho \stackrel{\text{def}}{=} [\![M]\!]_\rho \cdot [\![N]\!]_\rho,$$

$$[\![\lambda x.M]\!]_\rho \stackrel{\text{def}}{=} \mathsf{Gr}(\mathsf{up}_{[D \to D]}[d \mapsto [\![M]\!]_{\rho[x \mapsto d]}]).$$

It is easy to see that D is an lts.

4.4. λC-*Definability of the Projection Maps* ψ_n

Is there a closed λ-term X that discriminates between the convergent and divergent λ-terms? That is, is there an X such that

$$\forall M \in \Lambda^o. \begin{cases} XM =_\beta \mathbf{I} & \text{if } M\Downarrow, \\ XM\Uparrow & \text{if } M\Uparrow, \end{cases}$$

where $\mathbf{I}$ is the identity? The answer is no.

LEMMA 4.4.1. *Let $X \in \Lambda^o$, Ω be any divergent (i.e. strongly unsolvable) λ-term and Ω_1 any least convergent term with respect to the bisimulation ordering, say, $\lambda x.\Omega$ for any $\Omega\Uparrow$. Then*

$$X\Omega\Uparrow \;\&\; X\Omega_1\Downarrow \Rightarrow X =_\beta \mathbf{I}.$$

Proof. Suppose X satisfies the antecedent. Then, clearly, $X \notin \mathscr{F}_2$ (see Definition 2.3.1). Whence, and because X is closed, $X =_\beta \lambda x.x\vec{P}$. But, if $|\vec{P}| \geqslant 1$, then $X\Omega_1 \Uparrow$. Hence, $X =_\beta \mathbf{I}$. ∎

More generally, we have the following definition.

DEFINITION 4.4.2. Let $\mathscr{A} = \langle A, \cdot, \Uparrow \rangle$ be a q-aswd (or equivalently, a q-ats) and let x range over A. *Convergence testing* is definable in $\mathscr{A}$ if for some $c \in A$ and for x ranging over A, $\mathscr{A}$ satisfies

- $c \Downarrow$,
- $x \Uparrow \Rightarrow cx \Uparrow$,
- $x \Downarrow \Rightarrow cx = \mathbf{I}$.

Convergence testing is definable in D by $c =_{\text{def}} \mathsf{Gr}(\mathsf{up}_{[D \to D]}(\perp_1 \searrow i))$ where $i \equiv [\![\lambda x.x]\!]$ and $\perp_1 =^{\text{def}} \mathsf{Gr}(\mathsf{up}_{[D \to D]}(\lambda x.\perp))$ as before.

DEFINITION 4.4.3. Let C be a set of constants with prescribed interpretations in D. For any function $f \in D^D$, we say that f is *λC-definable* in D if

$$\exists M \in \Lambda(C)^o . \forall x \in D . f(x) = [\![M]\!] \cdot x.$$

Observation 4.4.4. Let C be interpreted as convergence testing c in D. The embedding pairs, $\langle (i_n, j_n) \rangle_{n \in \omega}$, are $\lambda\mathsf{C}$-definable in D by the $\lambda\mathsf{C}$-terms

$$\mathbf{i}_{n+1} \stackrel{\text{def}}{=} \lambda f.\mathsf{C}f(\lambda y.\mathbf{i}_n(f(\mathbf{j}_n y))),$$

$$\mathbf{j}_{n+1} \stackrel{\text{def}}{=} \lambda g.\mathsf{C}g(\lambda y.\mathbf{j}_n(g(\mathbf{i}_n y))),$$

with $\mathbf{i}_0 =^{\text{def}} \mathbf{I}$ and $\mathbf{j}_0 =^{\text{def}} \lambda x.\perp$.

Given a sequence of functions $\langle p_n : D \to D : n \in \omega \rangle$ defined inductively by $p_0 = \lambda x.\perp$ and

$$\forall d \in D . \forall n \in \omega . j_n(p_{n+1} d) = p_n(d), \tag{†}$$

is it true that $\forall n \in \omega . p_n = \psi_n$? Recall that $\psi_n : D \to D_n$ is the canonical projection function from D to D_n.

An extra condition is needed:

$$\forall e \in D . \forall n \in \omega . p_{n+1}(d) \cdot e = p_n(d \cdot p_n(e)). \tag{‡}$$

Clearly, to show $\forall n \in \omega . p_n = \psi_n$, it suffices to show that

$$\forall n \in \omega . \forall d \in D . p_n(d) = d_n = \psi_n(d);$$

and this we do by induction on n.

The base case is valid by premise. Now, suppose for some $n \geqslant 1$, $\forall d \in D.\, p_n(d) = d_n$. If $d \Uparrow$, by the induction hypothesis $p_n = \psi_n$, hence $p_n(d) = \bot$, and so, by (†) and because j_{n+1} is bistrict (i.e., $j_{n+1}(x) = \bot \Leftrightarrow x = \bot$), we must have $\forall m > n.\, p_m(d) = \bot$. Now, suppose $d \Downarrow$. Then, $d_n \Downarrow$; by hypothesis $d_n = \psi_n d$ and hence $p_{n+1}(d) \Downarrow$ by an appeal to (†). For any $e \in D$,

$$\begin{aligned} d_{n+1} \cdot e &= (d \cdot e_n)_n \\ &= p_n(d \cdot p_n(e)) \qquad \text{induction hypothesis} \\ &= p_{n+1}(d) \cdot e. \qquad (\ddagger) \end{aligned}$$

By conditional strong extensionality of D, we have $d_{n+1} = p_{n+1}(d)$. Let us summarize the result as follows:

LEMMA 4.4.5. *Let* $\langle p_n : D \to D : n \in \omega \rangle$ *and* $d, e \in D$ *such that*

(1) $p_0 = \lambda x.\, \bot$.

(2) $\forall n \in \omega.\, j_n(p_{n+1} d) = p_n d$.

(3) $\forall n \in \omega.\, p_{n+1}(d) \cdot e = p_n(d \cdot p_n(e))$.

Then $\forall n \in \omega \cdot p_n = \psi_n$. ∎

We now apply the lemma to show that the canonical projection functions $\langle \psi_n \rangle_{n \in \omega}$ are λC-definable.

DEFINITION 4.4.6. We define inductively a sequence of λC-terms, $\langle \Psi_n \rangle_{n \in \omega}$, as

$$\Psi_0 \stackrel{\text{def}}{=} \lambda x.\, \Omega,$$

$$\Psi_{n+1} \stackrel{\text{def}}{=} \lambda x.\, \mathsf{C} x (\lambda y.\, \Psi_n (x (\Psi_n y))),$$

where Ω is any strongly unsolvable term.

LEMMA 4.4.7. (i) $\forall n \in \omega.\, \forall d \in D.\, j_n([\![\Psi_{n+1}]\!] \cdot d) = [\![\Psi_n]\!] \cdot d$.

(ii) $\forall n \in \omega.\, \forall d \in D_n.\, [\![\Psi_{n+1}]\!] \cdot (i_n d) = [\![\Psi_n]\!] \cdot d$.

(iii) $\forall n \in \omega.\, \forall d, e \in D.\, ([\![\Psi_{n+1}]\!] \cdot d) \cdot e = [\![\Psi_n]\!] \cdot (d \cdot ([\![\Psi_n]\!] \cdot e))$.

Proof. We will omit the decorated brackets in $[\![\Psi_n]\!]$ and write $d \cdot e$ simply as de in the proof for easy reading. (i) and (ii) are proved by simultaneous induction on n. Without loss of generality, suppose $d \Downarrow$, for the case of divergent d is readily seen to be true. Base cases are clearly valid. Suppose (i) and (ii) are true for $n - 1$ for some $n \geqslant 1$. For (i),

$$
\begin{aligned}
j_n(\Psi_{n+1}d) &= j_n[(\lambda x.\mathbf{C}x(\lambda y.\Psi_n(x(\Psi_n y))))d] \\
&= j_n(\lambda y.\Psi_n(d(\Psi_n y))) \\
&= \lambda x.j_{n-1}(\Psi_n(d(\Psi_n(i_{n-1}x)))) && \text{definition of } j_n \\
&= \lambda x.\Psi_{n-1}(d(\Psi_{n-1}x)) && \text{ind. hyp. (i), (ii)} \\
&= \Psi_n d. && \text{definition of } \Psi_n
\end{aligned}
$$

For (ii), let $d \in D_n$. Then

$$
\begin{aligned}
\Psi_{n+1}(i_n d) &= \lambda y.\Psi_n((i_n d)(\Psi_n y)) \\
&= \lambda y.\Psi_n(i_{n-1}(d(j_{n-1}(\Psi_n y)))) && \text{definition of } i_n \\
&= \lambda y.\Psi_{n-1}(d(j_{n-1}(\Psi_n y))) && \text{ind. hyp. (ii)} \\
&= \lambda y.\Psi_{n-1}(d(\Psi_{n-1} y)) && \text{ind. hyp. (i)} \\
&= \Psi_n d. && \text{definition of } \Psi_n
\end{aligned}
$$

(iii) is easy: $(\Psi_{n+1}d)e = (\lambda y.\Psi_n(d(\Psi_n y)))e = \Psi_n(d(\Psi_n e))$. ∎

Applying Lemma 4.4.5, we establish the following result:

PROPOSITION 4.4.8. *The canonical projection maps* $\psi_n: D \to D_n$ *are* $\lambda\mathbf{C}$*-definable; i.e.,* $\forall n \in \omega.\forall d \in D.\psi_n(d) = [\![\Psi_n]\!] \cdot d$.

4.5. *Application Preserves Arbitrary Joins*

Next, we establish an important property satisfied by the application operation of D: "$\cdot$" *left-preserves arbitrary joins*, a result which we need in Section 9. This is a consequence of the *coincidence* of representable and continuous functions of D. First, we state a technical fact.

Fact 4.5.1. *Let* $\mathcal{I}$ *and* $\mathcal{J}$ *be arbitrary indexing sets and* $\mu: \mathcal{I} \times \mathcal{J} \to D$ *and let* D *be a complete lattice. Then*

$$\bigsqcup_{i \in \mathcal{I}} \bigsqcup_{j \in \mathcal{J}} \mu(i, j) = \bigsqcup_{j \in \mathcal{J}} \bigsqcup_{i \in \mathcal{I}} \mu(i, j).$$

PROPOSITION 4.5.2. *In* D, *the application* "$\cdot$" *left-preserves arbitrary joins; i.e.,*

$$\forall X \subseteq D.\forall d \in D, \left(\bigsqcup X\right) \cdot d = \bigsqcup_{x \in X} x \cdot d.$$

Proof. Let $X \subseteq D$. Without loss of generality, assume that X contains a convergent element. Define a function $F_X : D \to D$ as: for $y \in D$,

$$F_X(y) \stackrel{\text{def}}{=} \bigsqcup_{x \in X} x \cdot y.$$

$F_X(y)$ exists for all $y \in D$ because D is a complete lattice. F_X is monotonic: for $y_1 \sqsubseteq y_2$ implies $\forall x \in X,\ x \cdot y_1 \sqsubseteq x \cdot y_2$. Hence $\bigsqcup_{x \in X} x \cdot y_1 \sqsubseteq \bigsqcup_{x \in X} x \cdot y_2$.

Further, F_X is continuous. To see this, let $Y \subseteq D$ be directed. Then

$$\begin{aligned} F_X\left(\bigsqcup Y\right) &= \bigsqcup_{x \in X} x \cdot \left(\bigsqcup_{y \in Y} y\right) \\ &= \bigsqcup_{x \in X} \left(\bigsqcup_{y \in Y} x \cdot y\right) && \text{“$\cdot$” is continuous} \\ &= \bigsqcup_{y \in Y} \left(\bigsqcup_{x \in X} x \cdot y\right) && \text{Fact 4.5.1} \\ &= \bigsqcup_{y \in Y} F_X(y). \end{aligned}$$

By the coincidence of representable and continuous function spaces of D (Corollary 4.3.7),

$$\exists \Box \in D.\, \Box \Downarrow \ \&\ [\forall y \in D.\, \Box \cdot y = F_X(y)].$$

We aim to show that $\Box = \bigsqcup X$. Now, $\forall y \in D. \forall x \in X. x \cdot y \sqsubseteq (\bigsqcup_{x \in X} x) \cdot y$ implies that

$$\forall y \in D. F_X(y) \stackrel{\text{def}}{=} \bigsqcup_{x \in X} (x \cdot y) \sqsubseteq \left(\bigsqcup_{x \in X} x\right) \cdot y.$$

Note that $\Box \Downarrow$ and $\bigsqcup X \Downarrow$; hence by conditional strong extensionality of D, we conclude that

$$\Box \sqsubseteq \bigsqcup_{x \in X} x.$$

But $\forall x \in X. \forall y \in D. x \cdot y \sqsubseteq \bigsqcup_{x \in X} (x \cdot y) = \Box \cdot y$. Hence, $\forall x \in X,\ x \sqsubseteq \Box$, from which we conclude that $\bigsqcup X \sqsubseteq \Box$. We have shown that $\Box = \bigsqcup X$ and we are done. ∎

5. A Domain Logic for Applicative Transition Systems

5.1. *Introduction*

In (Abr87b) and (Abr91b), the mathematical framework of Stone duality is used to synthesize a number of hitherto separate developments in theoretical computer science:

- Domain theory (Sco72, Plo81, Sto79), the mathematical theory of computation introduced by Scott as a foundation for denotational semantics.
- The theory of concurrency and systems behaviour developed by Milner, Hennessy, and co-workers based on operational semantics (Mil80, HM85).
- Logics of programs.

Stone duality provides a junction between semantics (spaces of points = denotations of computational processes) and logics (lattices of *properties* of processes). Moreover, the underlying logic is *geometric*, which can be computationally interpreted as the logic of *observable* properties—i.e., properties which can be determined to hold of a process on the basis of a finite amount of information about its execution.

These ideas lead to the following program:

1. A metalanguage is introduced, comprising
   - types = universes of discourse for various computational situations.
   - terms = programs = syntactic intensions for models or points.
2. A standard denotational interpretation of the metalanguage is given, assigning domains to types and domain elements to terms.
3. The metalanguage is also given a *logical* interpretation, in which types are interpreted as propositional theories and terms are interpreted *via* a program logic, which axiomatizes the properties they satisfy.
4. The two interpretations are related by showing that they are *Stone duals* of each other. Hence, semantics and logic are guaranteed to be in harmony with each other, and in fact each determines the other up to isomorphism.
5. This opens the way to a whole range of applications. Given a denotational description of a computational situation in our metalanguage, we can turn the handle to obtain a logic for that situation.

The work reported in this section as well as in Section 7 may be seen as just such an application. This program is made systematic by using the

information conveyed in the syntactic description of domains by type expressions. It can be argued that a full domain-theoretic analysis of some computational situation is only obtained when we have written down an explicit type expression, rather than using some ad hoc construction of a cpo. At any rate, the benefits which flow from having such a description are very considerable. The reader is referred to (Abr91a) for an application of this program to concurrency.

5.2. *Domain Logic* $\mathscr{L}$

Recall that the type expression describing the canonical model of the lazy λ-calculus is

$$D = [D \to D]_\perp.$$

In Sections 3 and 4, we see how this type expression hints at the class of structures known as (quasi)-applicative transition systems (q-ats). We present a domain logic for q-ats as follows.

DEFINITION 5.2.1. The syntax of our domain logic $\mathscr{L}$ is defined by

$$\phi ::= t \mid \phi \wedge \psi \mid (\phi \to \psi)_\perp.$$

DEFINITION 5.2.2 (Semantics of $\mathscr{L}$). Given a q-ats $\mathscr{A}$, we define the satisfaction relation $\models_{\mathscr{A}} \subseteq \mathscr{A} \times \mathscr{L}$:

$$a \models_{\mathscr{A}} t \stackrel{\text{def}}{=} \text{true}$$

$$a \models_{\mathscr{A}} \phi \wedge \psi \stackrel{\text{def}}{=} a \models_{\mathscr{A}} \phi \;\&\; a \models_{\mathscr{A}} \psi$$

$$a \models_{\mathscr{A}} (\phi \to \psi)_\perp \stackrel{\text{def}}{=} a \Downarrow f \;\&\; [\forall b \in A.\, b \models_{\mathscr{A}} \phi \Rightarrow f(b) \models_{\mathscr{A}} \psi].$$

Notation.

$$\mathscr{L}(a) \stackrel{\text{def}}{=} \{\phi \in \mathscr{L} : a \models_{\mathscr{A}} \phi\}$$

$$\mathscr{A} \models \phi \leqslant \psi \stackrel{\text{def}}{=} \forall a \in A.\, a \models_{\mathscr{A}} \phi \Rightarrow a \models_{\mathscr{A}} \psi$$

$$\mathscr{A} \models \phi = \psi \stackrel{\text{def}}{=} \forall a \in A.\, a \models_{\mathscr{A}} \phi \Leftrightarrow a \models_{\mathscr{A}} \psi$$

$$\models \phi \leqslant \psi \stackrel{\text{def}}{=} \forall \mathscr{A}.\, \mathscr{A} \models \phi \leqslant \psi$$

$$\lambda \stackrel{\text{def}}{=} (t \to t)_\perp$$

$$a \sqsubseteq^{\mathscr{L}} b \stackrel{\text{def}}{=} \mathscr{L}(a) \subseteq \mathscr{L}(b).$$

Note that $\forall a \in A.\, a\Downarrow \Leftrightarrow a \models_{\mathscr{A}} \lambda$.

LEMMA 5.2.3. *Let $\mathscr{A}$ be a q-ats. Then*

$$\forall a, b \in A.\, a \lesssim^{B} b \Rightarrow a \lesssim^{\mathscr{L}} b.$$

Proof. We assume that $a \lesssim^{B} b$ and prove that $\forall \phi \in \mathscr{L}.\, a \models_{\mathscr{A}} \phi \Rightarrow b \models_{\mathscr{A}} \phi$ by induction on ϕ. The non-trivial case is $(\phi \to \psi)_{\perp}$.

- $a \models_{\mathscr{A}} (\phi \to \psi)_{\perp}$
- $\Rightarrow$ $a \Downarrow f$
- $\Rightarrow$ $b \Downarrow g \;\&\; \forall c.\, f(c) \lesssim^{B} g(c)$
- $\Rightarrow$ $\forall c.\, c \models_{\mathscr{A}} \phi \Rightarrow f(c) \lesssim^{B} g(c) \;\&\; f(c) \models_{\mathscr{A}} \psi$
- $\Rightarrow$ $\forall c.\, c \models_{\mathscr{A}} \phi \Rightarrow g(c) \models_{\mathscr{A}} \psi$ ind. hyp.
- $\Rightarrow$ $b \models_{\mathscr{A}} (\phi \to \psi)_{\perp}$. ∎

To get a converse to this result, we need a condition on $\mathscr{A}$.

DEFINITION 5.2.4. A q-ats $\mathscr{A}$ is *approximable* if

$$\forall a, b_1, \ldots, b_n \in A.\, ab_1 \cdots b_n \Downarrow \Rightarrow \exists \phi_1, \ldots, \phi_n.$$
$$a \models_{\mathscr{A}} (\phi_1 \to \cdots (\phi_n \to \lambda)_{\perp} \cdots)_{\perp} \;\&\; \forall i.\, 1 \leqslant i \leqslant n.\, b_i \models_{\mathscr{A}} \phi_i.$$

This is a natural condition, which says that convergence of a function application is caused by some finite amount of information (observable properties) of its arguments.

As expected, we have

THEOREM 5.2.5 (Characterization). *Let $\mathscr{A}$ be an approximable q-ats. Then*

$$\lesssim^{B} = \lesssim^{\mathscr{L}}.$$

Proof. By Lemma 5.2.3, $\lesssim^{B} \subseteq \lesssim^{\mathscr{L}}$. For the converse, suppose $a \not\lesssim^{B} b$. Then for some k, $a \not\lesssim^{B}_{k} b$, and for some $c_1, \ldots, c_k \in A$,

$$ac_1 \cdots c_k \Downarrow \;\&\; bc_1 \cdots c_k \Uparrow.$$

By approximability, for some $\phi_1, \ldots, \phi_k \in \mathscr{L}$,

$$a \models_{\mathscr{A}} (\phi_1 \to \cdots (\phi_k \to \lambda)_{\perp} \cdots)_{\perp} \;\&\; \forall i.\, 1 \leqslant i \leqslant k.\, c_i \models_{\mathscr{A}} \phi_i.$$

Clearly $b \not\models_{A} (\phi_1 \to \cdots (\phi_k \to \lambda)_{\perp} \cdots)_{\perp}$, and so $a \not\lesssim^{\mathscr{L}} b$. ∎

As a further consequence of approximability, we have:

PROPOSITION 5.2.6. *An approximable q-ats is an ats.*

Proof. Suppose $a \Downarrow f$ and $b \sqsubseteq^B c$. We must show $f(b) \sqsubseteq^B f(c)$. It is sufficient to show that for all $k \in \omega$, $d_1, \ldots, d_k \in A$:

$$f(b)\, d_1 \cdots d_k \Downarrow \Rightarrow f(c)\, d_1 \cdots d_k \Downarrow.$$

Now $f(b)\, d_1 \cdots d_k \Downarrow$ implies $abd_1 \cdots d_k \Downarrow$; hence by approximability, for some $\phi, \phi_1, \ldots, \phi_k \in \mathscr{L}$,

$$a \models_{\mathscr{A}} (\phi \to (\phi_1 \to \cdots (\phi_k \to \lambda)_\perp \cdots)_\perp)_\perp$$

and

$$b \models_{\mathscr{A}} \phi \;\&\; \forall i.\, 1 \leqslant i \leqslant k.\, d_i \models_{\mathscr{A}} \phi_i.$$

By Theorem 5.2.5, $c \models_{\mathscr{A}} \phi$, and so $acd_1 \cdots d_k \models_{\mathscr{A}} \lambda$, and $f(c)\, d_1 \cdots d_k \Downarrow$ as required. ∎

5.3. *Proof System For* $\mathscr{L}$

We now introduce a proof system for assertions of the form $\phi \leqslant \psi$, $\phi = \psi$ ($\phi, \psi \in \mathscr{L}$).

DEFINITION 5.3.1.

$$\text{(REF)}\quad \phi \leqslant \phi \qquad \text{(TRANS)}\quad \frac{\phi \leqslant \psi \quad \psi \leqslant \xi}{\phi \leqslant \xi}$$

$$(= -I)\quad \frac{\phi \leqslant \psi \quad \psi \leqslant \phi}{\phi = \psi} \qquad (= -E)\quad \frac{\phi = \psi}{\phi \leqslant \psi \quad \psi \leqslant \phi}$$

$$(t - I)\quad \phi \leqslant t$$

$$(\wedge - I)\quad \frac{\phi \leqslant \phi_1 \quad \phi \leqslant \phi_2}{\phi \leqslant \phi_1 \wedge \phi_2} \qquad (\wedge - E)\quad \phi \wedge \psi \leqslant \phi \quad \phi \wedge \psi \leqslant \psi$$

$$((\to)_\perp - \leqslant)\quad \frac{\phi_2 \leqslant \phi_1 \quad \psi_1 \leqslant \psi_2}{(\phi_1 \to \psi_1)_\perp \leqslant (\phi_2 \to \psi_2)_\perp}$$

$$((\to)_\perp - \wedge)\quad (\phi \to \psi_1 \wedge \psi_2)_\perp = (\phi \to \psi_1)_\perp \wedge (\phi \to \psi_2)_\perp$$

$$((\to)_\perp - t)\quad (\phi \to t)_\perp \leqslant (t \to t)_\perp.$$

We write $\mathscr{L} \vdash A$ or just $\vdash A$ to indicate that an assertion A is derivable from these axioms and rules. Note that the converse of $((\to)_\perp - t)$ is derivable from $(t - I)$ and $((\to)_\perp - \leqslant)$; by abuse of notation we refer to the corresponding equation by the same name.

THEOREM 5.3.2 (Soundness). $\vdash \phi \leqslant \psi \Rightarrow \models \phi \leqslant \psi$.

Proof. By a routine induction on the length of proofs. ∎

So far, our logic has been presented in a syntax-free fashion so far as the elements of the q-ats are concerned. Now suppose we have a lazy λ-model $\mathscr{A}$. λ-terms can be interpreted in $\mathscr{A}$, and for $M \in \Lambda^o$, $\rho \in \mathsf{Env}(\mathscr{A})$, we can define

$$M, \rho \models_{\mathscr{A}} \phi \stackrel{\text{def}}{=} [\![M]\!]^{\mathscr{A}}_{\rho} \models_{\mathscr{A}} \phi.$$

We can extend this to arbitrary terms $M \in \Lambda$ in the presence of *assumptions* $\Gamma: \mathsf{Var} \to \mathscr{L}$ on the variables,

$$M, \Gamma \models_{\mathscr{A}} \phi \stackrel{\text{def}}{=} [\forall \rho \in \mathsf{Env}(\mathscr{A}). \rho \models_{\mathscr{A}} \Gamma \Rightarrow [\![M]\!]^{\mathscr{A}}_{\rho} \models_{\mathscr{A}} \phi],$$

where

$$\rho \models_{\mathscr{A}} \Gamma \stackrel{\text{def}}{=} \forall x \in \mathsf{Var}. \rho x \models_{\mathscr{A}} \Gamma x.$$

We write

$$M, \Gamma \models \phi \stackrel{\text{def}}{=} \forall \mathscr{A}. M, \Gamma \models_{\mathscr{A}} \phi.$$

We now introduce a proof system for assertions of the form $M, \Gamma \vdash \phi$.

Notation. $\Gamma \leqslant \Delta =^{\text{def}} \forall x \in \mathsf{Var}. \Gamma x \leqslant \Delta x$.

Proof System For Program Logic

$$M, \Gamma \vdash t$$

$$\frac{M, \Gamma \vdash \phi \quad M, \Gamma \vdash \psi}{M, \Gamma \vdash \phi \wedge \psi}$$

$$\frac{\Gamma \leqslant \Delta \quad M, \Delta \vdash \phi \quad \phi \leqslant \psi}{M, \Gamma \vdash \psi}$$

$$x, \Gamma[x \mapsto \phi] \vdash \phi$$

$$\frac{M, \Gamma[x \mapsto \phi] \vdash \psi}{\lambda x. M, \Gamma \vdash (\phi \to \psi)_{\perp}}$$

$$\frac{M, \Gamma \vdash (\phi \to \psi)_{\perp} \quad N, \Gamma \vdash \phi}{MN, \Gamma \vdash \psi}.$$

THEOREM 5.3.3 (Soundness of Program Logic). *For all* M, Γ, ϕ:

$$M, \Gamma \vdash \phi \Rightarrow M, \Gamma \models \phi.$$

The proof is again routine. Note the striking similarity of our program logic to type inference, in particular to the intersection type discipline and Extended Applicative Type Structures of (CDC84). The crucial *difference* lies in the entailment relation $\leqslant$, and in particular the fact that their axiom (in our notation)

$$t \leqslant (t \to t)_\perp$$

is *not* a theorem in our logic; instead, we have the weaker $((\to)_\perp - t)$. This reflects a different notion of "function space"; we discuss this further in the sequel.

5.4. *Stone Duality*

We now come to the expected connection between the domain logic $\mathscr{L}$ and the domain D. The connecting link is the domain equation used to define D, and from which $\mathscr{L}$ is derived. Since this equation corresponds to the type expression $\sigma \equiv \mathsf{rec}\, t.(t \to t)_\perp$, it falls within the scope of the general theory developed in (Abr87c); see also (Abr91a) for a detailed presentation of the theory for the more general setting of SFP domains. The logic $\mathscr{L}$ presented in this section is a streamlined version of $\mathscr{L}(\sigma)$ as defined in (Abr87c). Once we have shown that $\mathscr{L}$ is equivalent to $\mathscr{L}(\sigma)$, we can apply the results of (Abr87c) to obtain the desired relationships between $\mathscr{L} \simeq \mathscr{L}(\sigma)$ and $D \simeq D(\sigma)$.[4]

First note that $\mathscr{L}$ as presented contains no disjunctive structure, while the constructs $\to$, $(\cdot)_\perp$ appearing in σ generate no inconsistencies according to the definition of C in (Abr87c). Thus (the Lindenbaum algebra of) $\mathscr{L}_\wedge(\sigma)$, the purely conjunctive part of $\mathscr{L}(\sigma)$, is a meet-semilattice, and applying (Abr87c, Theorem 2.3.4), we obtain

$$\mathsf{Spec}(\mathscr{L}(\sigma)/=_\sigma, \leqslant_\sigma/=_\sigma) \cong \mathsf{Filt}(\mathscr{L}_\wedge(\sigma)/=_\sigma, \leqslant_\sigma/=_\sigma).$$

It remains to show that $\mathscr{L}$ is pre-isomorphic to $\mathscr{L}_\wedge(\sigma)$. We can describe the syntax of $\mathscr{L}_\wedge(\sigma)$ as follows:

- $L_\wedge(\sigma)$:

$$\phi ::= t \mid \phi \wedge \psi \mid (\phi)_\perp \quad (\phi \in L(\sigma \to \sigma))$$

- $L_\wedge(\sigma \to \sigma)$:

$$\phi ::= t \mid \phi \wedge \psi \mid (\phi \to \psi) \quad (\phi, \psi \in L(\sigma)).$$

[4] The reader unfamiliar with (Abr87c) who is prepared to take Theorems 5.4.4 and 5.4.6 on trust is advised to skip the details till after Theorem 5.4.6.

Using $(( )_\perp - \wedge)$ and $(\to -t)$ (i.e., the nullary instances of $(\to - \wedge)$) from (Abr87c), we obtain the following normal forms for $L_\wedge(\sigma)$:

$$\phi ::= t \mid \phi \wedge \psi \mid (\phi \to \psi)_\perp .$$

In this way we see that $\mathscr{L} \subseteq \mathscr{L}_\wedge(\sigma)$, and that each $\phi \in \mathscr{L}_\wedge(\sigma)$ is equivalent to one in $\mathscr{L}$. Moreover, the axioms and rules of $\mathscr{L}$ are easily seen to be derivable in $\mathscr{L}_\wedge(\sigma)$. For example, $((\to)_\perp - t)$ is derivable, since

$$\mathscr{L}_\wedge(\sigma) \vdash (\phi \to \psi)_\perp = (t)_\perp = (t \to t)_\perp .$$

It remains to show the converse; i.e., that for $\phi, \psi \in \mathscr{L}$;

$$\mathscr{L}_\wedge(\sigma) \vdash \phi \leqslant \psi \Rightarrow \mathscr{L} \vdash \phi \leqslant \psi .$$

For this purpose, we use $((\to)_\perp - \bigwedge)$ and $((\to)_\perp - t)$ to get normal forms for $\mathscr{L}$.

LEMMA 5.4.1 (Normal Forms). *Every formula in $\mathscr{L}$ is equivalent to one in $N\mathscr{L}$, where:*

- $N\mathscr{L} = \{\bigwedge_{i \in I} \phi_i : I \text{ finite}, \phi_i \in SN\mathscr{L}, i \in I\}$
- $SN\mathscr{L} = \{(\phi_1 \to \cdots (\phi_k \to \lambda)_\perp \cdots)_\perp : k \geqslant 0 \ \& \ \forall i . 1 \leqslant i \leqslant k . \phi_i \in N\mathscr{L}\}$.

Now by (Abr87c, Proposition 3.4.5 and 3.4.6), we have

LEMMA 5.4.2. *For ϕ, ψ with $\phi \equiv \bigwedge_{i \in I} (\phi_i \to \phi_i')_\perp$, $\psi \equiv \bigwedge_{j \in J} (\psi_j \to \psi_j')_\perp$:*

$$\mathscr{L}(\sigma) \vdash \phi \leqslant \psi \Leftrightarrow \forall j \in J . \mathscr{L}(\sigma) \vdash \bigwedge \{\phi_i' : \mathscr{L}(\sigma) \vdash \psi_j \leqslant \phi_i\} \leqslant \psi_j' .$$

PROPOSITION 5.4.3. *For $\phi, \psi \in N\mathscr{L}$, if $\mathscr{L}(\sigma) \vdash \phi \leqslant \psi$ then there is a proof of $\phi \leqslant \psi$ using only the meet-semilattice laws and the derived rule $((\to)_\perp)$.*

Proof. By induction on the complexity of ϕ and ψ, and the preceding lemma. ∎

We have thus shown that

$$\mathscr{L}(\sigma) \cong \mathscr{L}_\wedge(\sigma) \cong \mathscr{L},$$

and we can apply the Duality Theorem of (Abr87c) to obtain

THEOREM 5.4.4 (Stone Duality). *$\mathscr{L}$ is the Stone dual of D; that is,*

(i) $D \cong \mathsf{Filt}(\mathscr{L})$,

(ii) $(\mathscr{K}(D))^{\mathrm{op}} \cong (\mathscr{L}/=, \leqslant/=)$,

where $\mathscr{K}(D)$ *is the sub-poset of finite elements of* D *and* $\mathsf{Filt}(\mathscr{L})$ *is the collection of all* filters *on* $\mathscr{L}$ *(i.e., subsets of* $\mathscr{L}$ *closed under* $\wedge$ *and entailment) ordered by set-inclusion.*

COROLLARY 5.4.5. $D \models \phi \leqslant \psi \Leftrightarrow \mathscr{L} \vdash \phi \leqslant \psi$.

We can now deal with the program logic over λ-terms in a similar fashion. The denotational semantics for Λ in D given in the previous section can be used to define a translation map

$$(\cdot)^* : \Lambda \to \Lambda(\sigma).$$

The logic presented in this section is equivalent to the endogenous logic of (Abr87c) in the sense that

$$M, \Gamma \vdash \phi \Leftrightarrow M^*, \Gamma \vdash \phi,$$

where $M \in \Lambda$, $\Gamma : \mathsf{Var} \to \mathscr{L}$, $\phi \in \mathscr{L} \subseteq \mathscr{L}(\sigma)$. We omit the details, which by now should be routine. As a consequence of this result, we can apply the Completeness Theorem for Endogenous Logic from (Abr87c), to obtain:

THEOREM 5.4.6. D *is* $\mathscr{L}$*-complete; i.e., for all* $M \in \Lambda$, $\Gamma : \mathsf{Var} \to \mathscr{L}$, $\phi \in \mathscr{L} \subseteq \mathscr{L}(\sigma)$,

$$M, \Gamma \vdash \phi \Leftrightarrow M, \Gamma \models_D \phi.$$

In the previous section, we defined an lts over D; and we have now shown that D is isomorphic as a domain to $\mathsf{Filt}(\mathscr{L})$. We can in fact describe the lts structure over $\mathsf{Filt}(\mathscr{L})$ directly; and this shows how D, defined by a domain equation reminiscent of the D_∞ construction, can also be viewed as a "graph" model, e.g., PSE-models (Lon83).

Notation. For $X \subseteq \mathscr{L}$, $X^\dagger$ is the *filter* generated by X. This can be defined inductively by:

- $X \subseteq X^\dagger$
- $t \in X^\dagger$
- $\phi, \psi \in X^\dagger \Rightarrow \phi \wedge \psi \in X^\dagger$
- $\phi \in X^\dagger$, $\mathscr{L} \vdash \phi \leqslant \psi \Rightarrow \psi \in X^\dagger$.

DEFINITION 5.4.7. The quasi-applicative structure with divergence

$$\langle \mathsf{Filt}(\mathscr{L}), \cdot, \Uparrow \rangle$$

is defined as follows:

$$x \Uparrow \stackrel{\text{def}}{=} x = \{t\}$$

$$x \cdot y \stackrel{\text{def}}{=} \{\psi : \exists \phi . (\phi \to \psi)_\perp \in x \ \& \ \phi \in y\} \cup \{t\}.$$

It is easily verified that in this structure

$$x \sqsubseteq^B y \Leftrightarrow x \subseteq y,$$

and hence that application is monotone in each argument, and $\mathsf{Filt}(\mathscr{L})$ is an ats. Thus we have an interpretation function

$$[\![-]\!]^{\mathsf{Filt}\,\mathscr{L}} : \Lambda(\mathsf{Filt}(\mathscr{L})) \to \mathsf{Env}(\mathsf{Filt}(\mathscr{L})) \to \mathsf{Filt}(\mathscr{L})$$

with

$$[\![\lambda x . M]\!]_\rho^{\mathsf{Filt}\,\mathscr{L}} = \{(\phi \to \psi)_\perp : \psi \in [\![M]\!]_{\rho[x \mapsto \uparrow \phi]}^{\mathsf{Filt}\,\mathscr{L}}\}^\dagger,$$

where $\uparrow \phi =^{\text{def}} \{\psi \in \mathscr{L} : \phi \leqslant \psi\}$.

PROPOSITION 5.4.8. $\mathsf{Filt}(\mathscr{L})$ *is an lts. Moreover,* $\mathsf{Filt}(\mathscr{L})$ *and* D *are isomorphic as combinatory algebras.*

Proof. It is sufficient to show that the isomorphism of the Duality Theorem preserves application, divergence, and the denotation of λ-terms, since it then preserves s and k and so is a combinatory isomorphism, and $\mathsf{Filt}(\mathscr{L})$ is an lts, since D is.

First we show that application is preserved; i.e., for $d_1, d_2 \in D$,

$$(*) \quad \mathscr{L}(d_1 \cdot d_2) = \mathscr{L}(d_1) \cdot \mathscr{L}(d_2).$$

The right to left inclusion follows by the same argument as the soundness of the rule for application in Theorem 5.3.2. For the converse, suppose $\psi \in \mathscr{L}(d_1 \cdot d_2)$, $\mathscr{L} \not\vdash \psi = t$. By the Duality Theorem, each ψ in $\mathscr{L}$ corresponds to a unique $c \in \mathscr{K}(D)$ with $\mathscr{L}(c) = \uparrow \psi$. Since application is continuous in D, $c \sqsubseteq d_1 \cdot d_2$, $c \neq \perp$ implies that for some $b \in \mathscr{K}(D)$, $\mathsf{Gr}(\langle 0, b \searrow c \rangle) \sqsubseteq d_1$ and $b \sqsubseteq d_2$. (Recall that $b \searrow c$ is the one step function mapping d to c if $b \sqsubseteq d$, and to $\perp$ otherwise). Let $\mathscr{L}(b) = \uparrow \phi$; then $(\phi \to \psi)_\perp \in \mathscr{L}(d_1)$ and $\phi \in \mathscr{L}(d_2)$, as required.

Next, we show that denotations of λ-terms are preserved; i.e., for all $M \in \Lambda$, $\rho \in \mathsf{Env}(D)$,

$$(**) \quad \mathscr{L}([\![M]\!]_\rho^D) = [\![M]\!]_{\mathscr{L}\rho}^{\mathsf{Filt}\,\mathscr{L}}.$$

This is proved by induction on M. The case when M is a variable is trivial; the case for application uses $(*)$. For abstraction, we argue by structural

induction over $\mathscr{L}$. We show the non-trivial case. Let ϕ, b be paired in the isomorphism of the Duality Theorem. Then

$$\begin{aligned}
&\lambda x.M, \rho \models_D (\phi \to \psi)_\perp \\
&\quad \Leftrightarrow M, \rho[x \mapsto b] \models_D \psi \\
&\quad \Leftrightarrow M, \mathscr{L}(\ ) \circ (\rho[x \mapsto b]) \models_{\mathsf{Filt}\,\mathscr{L}} \psi \qquad \text{ind. hyp.} \\
&\quad \Leftrightarrow M, (\mathscr{L}(\ ) \circ \rho)[x \mapsto \uparrow\phi] \models_{\mathsf{Filt}\,\mathscr{L}} \psi \\
&\quad \Leftrightarrow \lambda x.M, \mathscr{L}(\ ) \circ \rho \models_{\mathsf{Filt}\,\mathscr{L}} (\phi \to \psi)_\perp .
\end{aligned}$$

Finally, divergence is trivially preserved, since the only divergent elements in D, $\mathsf{Filt}(\mathscr{L})$ are $\perp$, $\{t\}$, and these are in bi-unique correspondence under the isomorphism of the Duality Theorem.

THEOREM 5.4.9 (Computational Adequacy). *For all* $M \in \Lambda^o$,

$$M\Downarrow \Leftrightarrow D \models M\Downarrow \qquad (i.e., \llbracket M \rrbracket^D \neq \perp).$$

Proof. The direction "$\Rightarrow$" follows immediately from D being a lazy λ-model. For the converse,

$$\begin{aligned}
\llbracket M \rrbracket^D \neq \perp &\Rightarrow \llbracket M \rrbracket^{\mathsf{Filt}\,\mathscr{L}} \neq \{t\} \qquad \text{by Proposition 5.4.8} \\
&\Rightarrow M \vdash \lambda \\
&\Rightarrow M \models \lambda \qquad \text{by Theorem 5.3.3} \\
&\Rightarrow M\Downarrow . \quad \blacksquare
\end{aligned}$$

The triviality of this proof is notable, since analogous results in the literature have required lengthy arguments involving recursively defined predicates (cf. (Plo85)).

5.5. *Categories of ATS and their Final Algebra*

We can now proceed in exact analogy to (Abr87a), and use Stone Duality to convert the Characterization Theorem into a Final Algebra Theorem.

DEFINITION 5.5.1. We define a number of categories of transition systems:

qATS Objects: quasi-applicative transition systems; morphisms $\mathscr{A} \to \mathscr{B}$: maps $f: A \to B$ satisfying

$$a \models_{\mathscr{A}} \phi \Leftrightarrow f(a) \models_{\mathscr{B}} \phi.$$

ATS The full subcategory of **qATS** of ats.

LTS The subcategory of **ATS** of lts and morphisms which preserve application, s and k.

CLTS The full subcategory of **LTS** of those $\mathscr{A}$ satisfying *continuity*:

$$\psi \neq t,\ ab \models_{\mathscr{A}} \psi \Rightarrow \exists \phi.\, a \models_{\mathscr{A}} (\phi \to \psi)_{\perp}\ \&\ b \models_{\mathscr{A}} \phi,$$

and also

$$\mathscr{L}(s) = [\![s]\!]^{\mathsf{Filt}\,\mathscr{L}}, \qquad \mathscr{L}(k) = [\![k]\!]^{\mathsf{Filt}\,\mathscr{L}}.$$

Note that continuity implies approximability.

THEOREM 5.5.2 (Final Algebra). (i) *D is final in* **qATS**, *hence in* **ATS**.

(ii) *Let $\mathscr{A}$ be an approximable lts. The map*

$$\mathsf{t}_{\mathscr{A}} : \mathscr{A} \to D$$

from (i) *is an* **LTS** *morphism iff $\mathscr{A}$ is continuous.*

(iii) *D is final in* **CLTS**.

Proof. (i) Given $\mathscr{A}$ in **qATS**, define

$$\mathsf{t}_{\mathscr{A}} \to D$$

by

$$\mathsf{t}_{\mathscr{A}} \equiv \mathscr{A} \xrightarrow{\mathscr{L}(\,)} \mathsf{Filt}(\mathscr{L}) \xrightarrow{\eta} D,$$

where η is the isomorphism from the Stone Duality Theorem. For $a \in A$,

$$\mathscr{L}(a) = \mathscr{L} \circ \eta \circ \mathscr{L}(a) = \mathscr{L} \circ \mathsf{t}_{\mathscr{A}}(a),$$

and so $\mathsf{t}_{\mathscr{A}}$ is a **qATS** morphism; moreover, it is unique, since for $d, d' \in D$,

$$\mathscr{L}(d') = \mathscr{L}(d') \Rightarrow \mathscr{K}(d) = \mathscr{K}(d') \Rightarrow d = d'.$$

(ii) That $\mathscr{L}(\,)$ is a combinatory morphism iff $\mathscr{A}$ is in **CLTS** is an immediate consequence of the definitions; the result then follows from the fact that η is a combinatory isomorphism.

(iii) Immediate from (ii). ∎

Note that if $\mathscr{A}$ is approximable, we have

$$a \sqsubseteq^{B} b \Leftrightarrow \mathsf{t}_{\mathscr{A}}(a) \sqsubseteq^{B} \mathsf{t}_{\mathscr{A}}(b).$$

Thus we can regard the Final Algebra Theorem as giving a syntax-free fully abstract semantics for approximable ats. However, from the point of view of applications to programming language semantics, this is not very useful. In Section 7, we study full abstraction in a syntax-directed framework, using our domain logic as a tool.

6. The Full Abstraction Problem—A Brief Introduction

This brief section introduces the background to the *full abstraction* problem. We review some basic notions and formulate the main problem which the rest of the paper seeks to solve.

6.1. *Introduction*

A fundamental notion that arises in reasoning about programs is that of *behavioural* or *operational equivalence* of programs (or program fragments). Programmers have an intuitively well understood criterion of when two pieces of code are equivalent: it is when the two are mutually replaceable as fragments of a program without resulting in any *observable* difference to the computational outcome. It is precisely this notion of program equivalence to which one appeals when proving the correctness of an optimizing compiler or a program transformation system.

To prove properties of programs, including that of operational equivalence, based purely on an understanding of the operational semantics is usually tedious, if tractable at all. Some of the difficulties may be ameliorated by using methods of reasoning based on denotational models of the language that "fit" its operational behaviour well. This is mainly because program properties are typically behavioural or extensional in nature (as opposed to *intensional*), and so they are more attuned to the denotational formulation rather than the operational. A good denotational semantics is a very effective tool for language design, software verification, and development and this has been borne out rather satisfactorily in functional programming, e.g., the ongoing work on the language *ML* (GMW79, Pau87) or more specifically, strictness analysis (BHA86, AH87).

Consider a programming language with a class of *observables* $\mathscr{C}$ consisting typically of a collection of ground constants, and $\mathscr{L}$, the class of well-formed *terms* (which may be typed or untyped) which includes a subclass of *programs* $\mathscr{P}$, equipped with the following:

- An *evaluation* mechanism eval: $\mathscr{P} \rightharpoonup \mathscr{C}$ (capturing the *interpreter* of the language) which specifies the computational behaviour of the language.

- A denotational semantics $[\![-]\!]: \mathscr{L} \to D$, where D is the underlying semantic model satisfying the following *computational adequacy* condition: for any *program* M,

$$\mathsf{eval}(M) = c \Leftrightarrow [\![M]\!] = [\![c]\!] \neq \bot \qquad \&$$
$$\mathsf{eval}(M) \text{ undefined} \Leftrightarrow [\![M]\!] = \bot.$$

We assume that the semantic function $[\![-]\!]$ is compositional; i.e., it is a *homomorphism* from the algebra of *programs* to the algebra of the underlying *domains*.

Let **Cxt** be the class of *program contexts* and for programs M and N, we write $M \leqslant N$ to mean

"if $\mathsf{eval}(M)$ is defined, then so is $\mathsf{eval}(N)$ and that $\mathsf{eval}(M) = \mathsf{eval}(N)$."

Now, define an operational preorder $\lesssim$ on terms as follows (we read $M \lesssim N$ as "M *safely* approximates N in all contexts"):

$$M \lesssim N \stackrel{\text{def}}{=} \forall C[\ ] \in \mathbf{Cxt}.\, C[M] \leqslant C[N].$$

An important criterion of the "goodness of fit" mentioned above is the (inequational) *full abstraction* property (Mil75, Plo77, Mil77) (see also (BCL85, Mul86, Sto88a, MC88)): for all terms M, N,

$$M \lesssim N \Leftrightarrow [\![M]\!] \sqsubseteq [\![N]\!].$$

The classical discussion of full abstraction in the λ-calculus (Plo77, Mil77) is set in the simply typed λ-calculus with ground data called PCF. As remarked in Section 1, this material has not to date been transferred successfully to the pure untyped λ-calculus. To see why this is so, recall that programs in the setting of PCF are closed terms of ground type. A program either diverges, or it yields a ground constant as result. This provides a natural notion of observable behaviour for programs, and hence an operational order on them. This is extended to arbitrary terms via ground contexts. In other words, the point of view is taken that:

- only program behaviour is directly observable, and
- the meaning of a higher-type term lies in the observable behaviour of the programs into which it can be embedded.

Thus both the presence of ground data, and the fact that terms are typed, enter into the basic definitions of the theory.

6.2. *Full Abstraction in the Lazy Lambda Calculus*

By contrast, we have a notion of atomic observation for the lazy λ-calculus in the absence of types or ground data, namely convergence to weak head normal form. This leads to the applicative bisimulation relation, and hence to a natural operational ordering. D, the initial solution of the domain equation $D \cong [D \to D]_\perp$, is our canonical denotational model. We can thus develop a theory of full abstraction in the pure untyped λ-calculus.

Let $\mathscr{K} =^{\text{def}} \langle K, \cdot^{\mathscr{K}}, \Downarrow_{\mathscr{K}}, [\![-]\!]^{\mathscr{K}} \rangle$ be an lts for the rest of the section.

We define four sets of formulae over $\Lambda(K)$, namely, *equations*, *inequations*, *atomic formulae*, and *conditional formulae*:

$$\mathsf{EQ}(\mathscr{K}) \stackrel{\text{def}}{=} \{M = N : M, N \in \Lambda(K)\}$$

(Equations)

$$\mathsf{INEQ}(\mathscr{K}) \stackrel{\text{def}}{=} \{M \sqsubseteq N : M, N \in \Lambda(K)\}$$

(Inequations)

$$\mathsf{AF}(\mathscr{K}) \stackrel{\text{def}}{=} \mathsf{EQ}(\mathscr{K}) \cup \mathsf{INEQ}(\mathscr{K}) \cup \{M\Downarrow : M \in \Lambda(K)\}$$

(Atomic Formulae)

$$\mathsf{CF}(\mathscr{K}) \stackrel{\text{def}}{=} \left\{ \bigwedge_{i \in I} M_i\Downarrow \Rightarrow F : F \in \mathsf{AF}(\mathscr{K}),\ M_i \in \Lambda(K),\ I \text{ finite} \right\}.$$

(Conditional Formulae)

Note that, taking $I = \varnothing$, $\mathsf{AF}(\mathscr{K}) \subseteq \mathsf{CF}(\mathscr{K})$. Let $\mathsf{EQ}^o(\mathscr{K})$ denote the set of closed formulae, similarly for $\mathsf{INEQ}^o(\mathscr{K})$, $\mathsf{AF}^o(\mathscr{K})$ and $\mathsf{CF}^o(\mathscr{K})$.

The interpretation of the formulae over $\Lambda(K)$ in a lazy λ-model is defined as follows:

DEFINITION 6.2.1. Let $\mathscr{Q} =^{\text{def}} \{Q, \cdot^{\mathscr{Q}}, \Downarrow_{\mathscr{Q}}, [\![-]\!]^{\mathscr{Q}}\rangle$ be an (environment) lazy λ-model . A map $k : K \to Q$ is a *combinatory interpretation* (we say that there is a combinatory interpretation of the lts $\mathscr{K}$ in the lazy λ-model $\mathscr{Q}$) if k preserves

- application: $\forall d, e \in K. k(d \cdot^{\mathscr{K}} e) = k(d) \cdot^{\mathscr{Q}} k(e)$,
- λ-terms:[5] $\forall M \in \Lambda^o. k([\![M]\!]^{\mathscr{K}}) = [\![M]\!]^{\mathscr{Q}}$,
- convergence: $\forall d \in K. d\Downarrow_{\mathscr{K}} \Rightarrow k(d)\Downarrow_{\mathscr{Q}}$.

k can then be uniquely extended by structural induction to an inter-

[5] Or equivalently, **K** and **S** combinators.

pretation of $\Lambda(K)^o$ in $\mathcal{Q}$. Given a combinatory interpretation of $\mathcal{K}$ in $\mathcal{Q}$, for any $M, N \in \Lambda(K)$, we write

$$[\![M]\!]^{\mathcal{Q}} \stackrel{\text{def}}{=} k([\![M]\!]^{\mathcal{K}})$$

for the induced interpretation of $\Lambda(K)^o$ in $\mathcal{Q}$; and we define

$$\mathcal{Q} \models M\Downarrow \stackrel{\text{def}}{=} \forall \rho: \mathsf{Var} \to Q.\, [\![M]\!]^{\mathcal{Q}}_{\rho} \Downarrow_{\mathcal{Q}},$$

$$\mathcal{Q} \models M \sqsubseteq N \stackrel{\text{def}}{=} \forall \rho: \mathsf{Var} \to Q.\, [\![M]\!]^{\mathcal{Q}}_{\rho} \precsim^{\mathcal{Q}} [\![N]\!]^{\mathcal{Q}}_{\rho},$$

$$\mathcal{Q} \models M = N \stackrel{\text{def}}{=} \forall \rho: \mathsf{Var} \to Q.\, [\![M]\!]^{\mathcal{Q}}_{\rho} \sim^{\mathcal{Q}} [\![N]\!]^{\mathcal{Q}}_{\rho},$$

where $\precsim^{\mathcal{Q}}$ and $\sim^{\mathcal{Q}}$ are the associated bisimulation preorder and equivalence, respectively. The conditional formula $\bigwedge_{i \in I} M_i \Rightarrow F$ is interpreted in the usual way. Note that if $\mathcal{Q} = \mathcal{K}$, then we take the identity map as the interpretation k.

If the lazy λ-model $\mathcal{Q}$ comes equipped with a partial order, i.e., $\langle Q, \sqsubseteq^{\mathcal{Q}} \rangle$ is a poset, then we say that $\mathcal{Q}$ is *internally fully abstract* if $\sqsubseteq^{\mathcal{Q}} = \precsim^{\mathcal{Q}}$.

Now, we are in a position to formalize the notion of full abstraction in the lazy λ-calculus.

DEFINITION 6.2.2 (Full Abstraction). Let $\mathcal{K}$ be an Its and $\mathcal{Q}$ an (environment) lazy λ-model. Given that there is a *combinatory* interpretation of $\mathcal{K}$ in $\mathcal{Q}$ via $k: K \to Q$, we define

$$\mathfrak{I}^{\mathcal{K}}_{\mathsf{eq}}(\mathcal{Q}) \stackrel{\text{def}}{=} \{F \in \mathsf{EQ}(\mathcal{K}) : \mathcal{Q} \models F\},$$

$$\mathfrak{I}^{\mathcal{K}}_{\mathsf{ineq}}(\mathcal{Q}) \stackrel{\text{def}}{=} \{F \in \mathsf{INEQ}(\mathcal{K}) : \mathcal{Q} \models F\},$$

$$\mathfrak{I}^{\mathcal{K}}_{\mathsf{cond}}(\mathcal{Q}) \stackrel{\text{def}}{=} \{F \in \mathsf{CF}(\mathcal{K}) : \mathcal{Q} \models F\}.$$

We say that $\mathcal{K}$ is *inequationally fully abstract* with respect to $\mathcal{Q}$ if $\mathfrak{I}^{\mathcal{K}}_{\mathsf{ineq}}(\mathcal{K}) = \mathfrak{I}^{\mathcal{K}}_{\mathsf{ineq}}(\mathcal{Q})$; similarly for *equational* and *conditional full abstraction*. We tabulate the definitions as follows:

Full abstraction of $\mathcal{K}$ w.r.t. $\mathcal{Q}$

inequational	$\mathfrak{I}^{\mathcal{K}}_{\mathsf{ineq}}(\mathcal{K}) = \mathfrak{I}^{\mathcal{K}}_{\mathsf{ineq}}(\mathcal{Q})$
equational	$\mathfrak{I}^{\mathcal{K}}_{\mathsf{eq}}(\mathcal{K}) = \mathfrak{I}^{\mathcal{K}}_{\mathsf{eq}}(\mathcal{Q})$
conditional	$\mathfrak{I}^{\mathcal{K}}_{\mathsf{cond}}(\mathcal{K}) = \mathfrak{I}^{\mathcal{K}}_{\mathsf{cond}}(\mathcal{Q})$

Remark 6.2.3. (i) This definition is consistent with that in (Plo77, Mil77), provided we accept the applicative bisimulation on $\mathcal{K}$ as the appropriate operational preorder. The argument for doing so is made

highly plausible by the property of *operational extensionality* satisfied by all lts's (see Definition 7.2.2 and Lemma 7.2.1), which characterizes applicative bisimulation as a contextual preorder analogous to those used in (Plo77, Mil77).

For example, in the case of the "language" ($\mathscr{K}$) being λI and the "model" ($\mathscr{Q}$) being D, the condition for λI being inequationally fully abstract w.r.t. D is

$$\forall M, N \in \Lambda. M \precsim^B N \Leftrightarrow D \models M \sqsubseteq N.$$

The combinatory embedding $k: \Lambda^o \to D$ in this case is just the semantic function $[\![-]\!]^D$.

(ii) In the cases of inequational and equational full abstraction (but not so for the conditional case), no generality is lost if only *closed* formulae are considered. This is because for $M, N \in \Lambda(K)$ with $\mathsf{FV}(M, N) = \{\vec{x}\}$, say, by definition of the applicative bisimulation between open λK-terms, we have

$$\mathscr{K} \models M \sqsubseteq N \Leftrightarrow \mathscr{K} \models \lambda\vec{x}.M \sqsubseteq \lambda\vec{x}.N.$$

Similarly, and because k is a combinatory interpretation, we have

$$\mathscr{Q} \models M \sqsubseteq N \Leftrightarrow \mathscr{Q} \models \lambda\vec{x}.M \sqsubseteq \lambda\vec{x}.N.$$

DEFINITION 6.2.4. Let $\mathscr{K}$ be an lts which has a combinatory interpretation in the lazy λ-model, $\mathscr{Q}$ via $k: K \to Q$. Then the combinatory interpretation is *adequate* if

$$\forall M, N \in \Lambda(K)^o. \mathscr{K} \models M\Downarrow \Leftrightarrow \mathscr{Q} \models M\Downarrow;$$

or equivalently, $\forall d \in K. d\Downarrow_{\mathscr{K}} \Leftrightarrow k(d)\Downarrow_{\mathscr{Q}}$.

LEMMA 6.2.5. *Let $\mathscr{K}$ and $\mathscr{Q}$ be as before and $k: K \to Q$ an adequate combinatory interpretation. Then*

$$\mathfrak{I}^{\mathscr{K}}_{\mathsf{ineq}}(\mathscr{K}) \supseteq \mathfrak{I}^{\mathscr{K}}_{\mathsf{ineq}}(\mathscr{Q}),$$
$$\mathfrak{I}^{\mathscr{K}}_{\mathsf{eq}}(\mathscr{K}) \supseteq \mathfrak{I}^{\mathscr{K}}_{\mathsf{eq}}(\mathscr{Q}),$$
$$\mathfrak{I}^{\mathscr{K}}_{\mathsf{cond}}(\mathscr{K}) \supseteq \mathfrak{I}^{\mathscr{K}}_{\mathsf{cond}}(\mathscr{Q}).$$

It is easy to see that if $\mathscr{Q}$ is adequate, then so is $\mathscr{K}$.

Proof. We just prove the inequational case; the rest are similar. Let $M, N \in \Lambda(K)^o$ such that $\mathscr{K} \models M \not\sqsubseteq N$. Then, for some $\vec{P} \subseteq \Lambda(K)^o$, we have

$\mathscr{K} \models MP\Downarrow$ & $N\vec{P}\Uparrow$. By adequacy of k, we have $\mathscr{L} \models M\vec{P}\Downarrow$ & $N\vec{P}\Uparrow$ which means that $\mathscr{L} \models M \not\sqsubseteq N$. ∎

It is easy to see from the preceding proof that

COROLLARY 6.2.6. *If there is a surjective, adequate combinatory interpretation from $\mathscr{K}$ to $\mathscr{L}$, then $\mathscr{K}$ is (equationally, inequationally, and conditionally) fully abstract w.r.t. $\mathscr{L}$.* ∎

The above corollary indicates that the property of full abstraction is essentially a question of *definability*; i.e., whether there are *enough* terms in the language to denote all the elements of the underlying model.

We have formulated the property of full abstraction in the lazy λ-calculus in three different "strengths," viz. equational, inequational, and conditional full abstraction. What is the relationship between them?

- It is easy to see that conditional full abstraction implies inequational full abstraction which in turn implies equational full abstraction. Hence, the strongest kind of non full abstraction result is that of equational non-full abstraction (see Section 8).

- In the presence of a convergence testing constant, inequational formulae are as expressive as conditional formulae; whence conditional full abstraction is equivalent to inequational full abstraction. We establish this in the following Lemma.

Let $\mathscr{K}$ and $\mathscr{L}$ be as before and let there be an adequate combinatory interpretation k of $\mathscr{K}$ in $\mathscr{L}$. Suppose k is surjective. It is then easy to see that if convergence testing is definable in $\mathscr{K}$ as c say, then it is also definable in $\mathscr{L}$ and the combinatory interpretation preserves convergence testing; i.e., $k(c)$ is a convergence testing constant in $\mathscr{L}$.

LEMMA 6.2.7. *Let $\mathscr{K}$ and $\mathscr{L}$ be as before. Suppose convergence testing is definable in both $\mathscr{K}$ and $\mathscr{L}$ and that the adequate combinatory interpretation k preserves convergence testing. Then inequational full abstraction is equivalent to conditional full abstraction.*

Proof. Suppose $\mathscr{K}$ is inequationally fully abstract w.r.t. $\mathscr{L}$. By virtue of Lemma 6.2.5, it suffices to show that

$$\mathfrak{I}^{\mathscr{K}}_{\mathsf{cond}}(\mathscr{K}) \subseteq \mathfrak{I}^{\mathscr{K}}_{\mathsf{cond}}(\mathscr{L}).$$

Let c and $c' = k(c)$ be the convergence testing elements in $\mathscr{K}$ and $\mathscr{L}$, respectively. Given any conditional formula $F \equiv \bigwedge_{i \in I} P_i \Rightarrow Q$, where $I \neq \varnothing$ and $\mathsf{FV}(P_1, \ldots, P_I, Q) = \{\vec{x}\}$, say, define a corresponding inequation F' or a pair of inequations F_1, F_2 (according to the syntactic shape of F),

$$F' \stackrel{\text{def}}{=} \begin{cases} \lambda\vec{x}.cP_1(cP_2)\cdots(cP_I)\,M \sqsubseteq \lambda\vec{x}.cP_1(cP_2)\cdots(cP_I)N & \text{if } Q \equiv M \sqsubseteq N, \\ \lambda\vec{x}.cP_1(cP_2)\cdots(cP_I)(\lambda x.\bot) \sqsubseteq \lambda\vec{x}.M & \text{if } Q \equiv M\Downarrow, \end{cases}$$

where $\perp \in K$ is the divergent element. If $Q \equiv M = N$, consider the following two inequations:

$$F_1 \stackrel{\text{def}}{=} \lambda\vec{x}.cP_1(cP_2)\cdots(cP_l)M \sqsubseteq \lambda\vec{x}.cP_1(cP_2)\cdots(cP_l)N,$$

$$F_2 \stackrel{\text{def}}{=} \lambda\vec{x}.cP_1(cP_2)\cdots(cP_l)N \sqsubseteq \lambda\vec{x}.cP_1(cP_2)\cdots(cP_l)M.$$

Note that $k(\perp)\Uparrow_{\mathscr{D}}$ by adequacy of k. It suffices to note that

$$F \in \mathfrak{I}^{\mathscr{K}}_{\text{cond}}(\mathscr{K}) \Leftrightarrow F' \text{ (or } F_1, F_2) \in \mathfrak{I}^{\mathscr{K}}_{\text{ineq}}(\mathscr{K})$$

and

$$F \in \mathfrak{I}^{\mathscr{K}}_{\text{cond}}(\mathscr{D}) \Leftrightarrow F' \text{ (or } F_1, F_2) \in \mathfrak{I}^{\mathscr{K}}_{\text{ineq}}(\mathscr{D}). \quad \blacksquare$$

Two Approaches to Full Abstraction

In Section 8, we show that the *pure lazy language* λl is inequationally non-fully abstract w.r.t. D. This is because the model D is in some sense "over-generous" with respect to the language: the model is so rich in structure that it makes distinctions between terms extraneous to operational equivalence. There are two approaches to rectifying the "poorness of fit" depending on whether the language or the model is taken as prior.

- The *expansive* approach consists in enriching the language by augmenting it with appropriate constants, thereby enabling all *finite* semantic information to be represented syntactically as program phrases. This approach is exemplified in (Plo77). In Section 7, we carry out the same program for the lazy λ-calculus using the domain logic machinery set up in Section 5.

- The *restrictive* approach regards the language as prescribed and tailors the model to fit by "cutting down" (i.e., "quotienting out" by a *bisimulation logical relation*) the existing "over-generous" model to an appropriate sub-structure, as is the approach adopted in (Mil77, Mul86). In Section 9, we show how the same approach could be taken to construct lazy λ-models which are *retracts* of the canonical model and (inequationally) fully abstract for a class of "lazy" languages (lts's).

7. Full Abstraction I: An Expansive Approach

In this section, we bring the machinery of domain logic set up in Section 5 to bear on the problem of full abstraction and that of operational extensionality. Our main result is an Expansive Full Abstraction Theorem in which we establish various equivalent conditions under which a class of

"languages" $\mathscr{A}$ (lambda transition systems) are fully abstract for the canonical model D. By appealing to the Stone duality between the domain D and its logic $\mathscr{L}$, we are able to express the property of full abstraction (which is essentially a question of the *definability* of a language w.r.t. its model) in terms of equivalent properties as follows:

- *Expressiveness* of the domain logic $\mathscr{L}$,
- *Completeness* of the endogenous program logic,
- The embedding of the lts's $\mathscr{A}$ into D being both combinatory and "logic preserving".

Our full abstraction result will correspond recognizably to those in (Plo77), although the technical details contain many differences. The second application of domain logic is a general operational extensionality result. One feature of our development is that we work axiomatically with classes of lts's under various hypotheses, rather than with particular languages. (Note that operational transition systems and "programming languages" such as λI actually *are* lts's under our definitions.)

7.1. *Expansive Full Abstraction*

We now turn to the question of conditions under which D is conditionally fully abstract for $\mathscr{A}$.

DEFINITION 7.1.1. An ats $\mathscr{A}$ is *$\mathscr{L}$-expressive* if for all $\phi \in \mathscr{L}$, for some $a \in A$,

$$\mathscr{L}(a) = \uparrow\phi \stackrel{\text{def}}{=} \{\psi \in \mathscr{L} : \mathscr{L} \vdash \phi \leqslant \psi\}.$$

In the light of Stone Duality, $\mathscr{L}$-expressiveness can be read as "all finite elements of D are definable in $\mathscr{A}$."

DEFINITION 7.1.2. Let $\mathscr{A}$ be an ats. *Parallel convergence* is definable in $\mathscr{A}$ if for some $p \in A$, $\mathscr{A}$ satisfies:

- $p\Downarrow$, $px\Downarrow$,
- $x\Downarrow \Rightarrow pxy\Downarrow$,
- $y\Downarrow \Rightarrow pxy\Downarrow$,
- $x\Uparrow$ & $y\Uparrow \Rightarrow pxy\Uparrow$.

In this case, we use P to denote such a p.

Note that if C is definable, it is unique (up to bisimulation); this is not so for P.

The notion of parallel convergence is reminiscent of Plotkin's parallel or, and will play a similar role in our theory. (A sharper comparison will be made later in this section.) The notion of convergence testing is less expected. We can think of the combinator C as a sort of "1-strict" version of $\mathbf{F} \equiv \lambda x.\lambda y.y$:

$$\begin{aligned} \mathsf{C}xy &= \mathbf{F}xy = y \qquad \text{if } x\Downarrow; \\ \mathsf{C}xy &\Uparrow \qquad\qquad\quad \text{if } x\Uparrow. \end{aligned}$$

This 1-strictness allows us to test, sequentially, a number of expressions for convergence. Under the hypothesis that C is definable, we can give a very satisfactory picture of the relationship between all these notions.

THEOREM 7.1.3 (Expansive Full Abstraction). *Let $\mathscr{A}$ be an adequate, approximable lts in which convergence testing* C *is definable. The following conditions are equivalent*:

(i) *Parallel convergence is definable in $\mathscr{A}$.*

(ii) *$\mathscr{A}$ is $\mathscr{L}$-expressive.*

(iii) *$\mathscr{A}$ is $\mathscr{L}$-complete.*

(iv) *$\mathsf{t}_{\mathscr{A}}$ is a combinatory embedding with $\mathscr{K}(D) \subseteq \mathrm{Im}\, \mathsf{t}_{\mathscr{A}}$.*

(v) *D is conditionally fully abstract for $\mathscr{A}$.*

Proof. We prove a sequence of implications to establish the theorem, indicating in each case which hypotheses on $\mathscr{A}$ are used.

(i) $\Rightarrow$ (ii) ($\mathscr{A}$ adequate, C definable).

Since $\mathscr{A}$ is adequate, Ω diverges in $\mathscr{A}$.

Notation. Given a set Con of constants, $\Lambda(\mathsf{Con})$ is the set of λ-terms over Con.

For each $\phi \in N\mathscr{L}$ we define terms M_ϕ, $T_\phi \in \Lambda(\{\mathsf{P}, \mathsf{C}\})$ such that, for all $\psi \in N\mathscr{L}$:

- $M_\phi \models_{\mathscr{A}} \psi \Leftrightarrow \mathscr{L} \vdash \phi \leqslant \psi$
- $\begin{cases} T_\phi M_\psi \Downarrow & \text{if } M_\psi \models_{\mathscr{A}} \phi, \\ T_\phi M_\psi \Uparrow & \text{otherwise.} \end{cases}$

The definition is by induction on the complexity of

$$\phi \equiv \bigwedge_{i \in I} (\phi_{i,1} \to \cdots (\phi_{i,k_i} \to \lambda)_\perp \cdots)_\perp.$$

If $I = \varnothing$, $M_\phi \equiv \Omega$. Otherwise, we define $M_\phi \equiv M(\phi, k)$, where $k = \min\{k_i : i \in I\}$,

$$M(\phi, 0) \stackrel{\text{def}}{=} \mathbf{K}\Omega$$

$$M(\phi, i+1) \stackrel{\text{def}}{=} \lambda x_j.\mathsf{C}N^j M(\phi, i),$$

where

$$j \stackrel{\text{def}}{=} k - i,$$

$$N^j \stackrel{\text{def}}{=} \sum \{N_i^j : j \leqslant k_i\},$$

$$N_i^j \stackrel{\text{def}}{=} \mathsf{C}(T_{\phi_{i,1}} x_1)(\mathsf{C}(T_{\phi_{i,2}} x_2)(\cdots(\mathsf{C}(T_{\phi_{i,j}} x_j))\cdots)),$$

$$\sum \varnothing \stackrel{\text{def}}{=} \Omega,$$

$$\sum \{N\} \cup \Theta \stackrel{\text{def}}{=} \mathsf{P}N\left(\sum \Theta\right),$$

$$T_\phi \stackrel{\text{def}}{=} \lambda x.\prod \{xM_{\phi_{i,1}} \cdots M_{\phi_{i,k_i}} : i \in I\},$$

$$\prod \varnothing \stackrel{\text{def}}{=} \mathbf{K}\Omega,$$

$$\prod \{N\} \cup \Theta \stackrel{\text{def}}{=} \mathsf{C}N\left(\prod \Theta\right).$$

We must show that these definitions have the required properties. First, we prove for all $\phi \in N\mathscr{L}$ that

(1) $M_\phi \models_{\mathscr{A}} \phi$

(2) $a \models_{\mathscr{A}} \phi \Rightarrow T_\phi a\Downarrow$

by induction on ϕ:

- $\forall i \in I.\forall j.1 \leqslant j \leqslant k_i.a_j \models_{\mathscr{A}} \phi_{i,j}$

$\Rightarrow$ $M_\phi a_1 \cdots a_{k_i}\Downarrow$ by induction hypothesis (2),

$\therefore$ $M_\phi \models_{\mathscr{A}} \phi$.

- $a \models_{\mathscr{A}} \phi$

$\Rightarrow$ $T_\phi a\Downarrow$ by induction hypothesis (1).

We complete the argument by proving, for all $\phi, \psi \in N\mathscr{L}$, that

(3) $M_\phi \models_{\mathscr{A}} \psi \Rightarrow \mathscr{L} \vdash \phi \leqslant \psi$

(4) $M_\psi \models_{\mathscr{A}} \phi \Rightarrow \mathscr{L} \vdash \psi \leqslant \phi$

(5) $T_\phi M_\psi\Downarrow \Rightarrow M_\psi \models_{\mathscr{A}} \phi$

(6) $T_\psi M_\phi\Downarrow \Rightarrow M_\phi \models_{\mathscr{A}} \psi$.

The proof is by induction on $n+m$, where n, m are the number of subformulae of ϕ, ψ, respectively. Let

$$\phi \equiv \bigwedge_{i \in I} (\phi_{i,1} \to \cdots (\phi_{i,k_i} \to \lambda)_\perp \cdots)_\perp,$$

$$\psi \equiv \bigwedge_{j \in J} (\psi_{j,1} \to \cdots (\psi_{j,k_j} \to \lambda)_\perp \cdots)_\perp.$$

(3)

- $M_\phi \models_{\mathscr{A}} \psi$
- $\Rightarrow \forall j \in J. M_\phi M_{\psi_{j,1}} \cdots M_{\psi_{j,k_j}} \Downarrow$ by (1),
- $\Rightarrow \forall j \in J. \exists i \in I. \forall l. 1 \leqslant l \leqslant k_j. k_j \leqslant k_i \,\&\, T_{\phi_{i,l}} M_{\psi_{j,l}} \Downarrow$
- $\Rightarrow \forall j \in J. \exists i \in I. \forall l. 1 \leqslant l \leqslant k_j. M_{\psi_{j,l}} \models_{\mathscr{A}} \phi_{i,l}$ ind. hyp. (5)
- $\Rightarrow \forall j \in J. \exists i \in I. \forall l. 1 \leqslant l \leqslant k_j. \mathscr{L} \vdash \psi_{j,l} \leqslant \phi_{i,l}$ ind. hyp. (4)
- $\Rightarrow \mathscr{L} \vdash \phi \leqslant \psi.$

(4) Symmetrical to (3).

(5)

- $T_\phi M_\psi \Downarrow$
- $\Rightarrow \forall i \in I. M_\psi M_{\phi_{i,1}} \cdots M_{\phi_{i,k_i}} \Downarrow$
- $\Rightarrow \forall i \in I. \exists j \in J. \forall l. 1 \leqslant l \leqslant k_i. k_i \leqslant k_j \,\&\, T_{\psi_{j,l}} M_{\phi_{i,l}} \Downarrow$
- $\Rightarrow \forall i \in I. \exists j \in J. \forall l. 1 \leqslant l \leqslant k_i. M_{\phi_{i,l}} \models_{\mathscr{A}} \psi_{j,l}$ ind. hyp. (6)
- $\Rightarrow \forall i \in I. \exists j \in J. \forall l. 1 \leqslant l \leqslant k_i. \mathscr{L} \vdash \phi_{i,l} \leqslant \psi_{j,l}$ ind. hyp. (3)
- $\Rightarrow \mathscr{L} \vdash \psi \leqslant \phi$
- $\Rightarrow M_\psi \models_{\mathscr{A}} \phi$ by (1).

(6) Symmetrical to (5).

(ii) $\Rightarrow$ (iii) ($\mathscr{A}$ approximable).

Notation. For each $\phi \in \mathscr{L}$, $a_\phi \in A$ is the element representing ϕ. Given $\Gamma: \mathsf{Var} \to \mathscr{L}$, $\rho_\Gamma \in \mathsf{Env}(\mathscr{A})$ is defined by

$$\rho_\Gamma x = a_{\Gamma x}.$$

Finally, $\Gamma_t: \mathsf{Var} \to \mathscr{L}$ is the constant map $x \mapsto t$.

We begin with some preliminary results.

(1) $\mathscr{A} \models \phi \leqslant \psi \Leftrightarrow \mathscr{L} \vdash \phi \leqslant \psi.$

One half is the Soundness Theorem for $\mathscr{L}$. For the converse, note that

$$\begin{aligned}\mathscr{A} \models \phi \leqslant \psi &\Rightarrow a_\phi \models_{\mathscr{A}} \psi \\ &\Rightarrow \mathscr{L} \vdash \phi \leqslant \psi.\end{aligned}$$

(2) $\forall \psi \in N\mathscr{L}.\psi \neq t \;\&\; ab \models_{\mathscr{A}} \psi \Rightarrow \exists \phi.a \models_{\mathscr{A}} (\phi \to \psi)_\perp \;\&\; b \models_{\mathscr{A}} \phi.$

This is shown by induction on ψ:

- $ab \models_{\mathscr{A}} \bigwedge_{i \in I} \psi_i \quad (I \neq \varnothing)$

$\Rightarrow$ $\forall i \in I.ab \models_{\mathscr{A}} \psi_i$

$\Rightarrow$ $\forall i \in I.\exists \phi_i.a \models_{\mathscr{A}} (\phi_i \to \psi_i)_\perp \;\&\; b \models_{\mathscr{A}} \phi_i$ by ind. hyp.

$\Rightarrow$ $a \models_{\mathscr{A}} \left(\bigwedge_{i \in I} \phi_i \to \psi_i\right)_\perp \;\&\; b \models_{\mathscr{A}} \bigwedge_{i \in I} \phi_i$

$\Rightarrow$ $a \models_{\mathscr{A}} \left(\bigwedge_{i \in I} \phi_i \to \bigwedge_{i \in I} \psi_i\right)_\perp \;\&\; b \models_{\mathscr{A}} \bigwedge_{i \in I} \phi_i.$

- $ab \models_{\mathscr{A}} (\psi_1 \to \cdots (\psi_k \to \lambda)_\perp \cdots)_\perp$

$\Rightarrow$ $aba_{\psi_1} \cdots a_{\psi_k}\Downarrow$

$\Rightarrow$ $\exists \phi, \phi_1, \ldots, \phi_k.b \models_{\mathscr{A}} \phi \;\&\; a_{\psi_i} \models_{\mathscr{A}} \phi_i \quad (1 \leqslant i \leqslant k)$
$\&\; a \models_{\mathscr{A}} (\phi \to (\phi_1 \to \cdots (\phi_k \to \lambda)_\perp \cdots)_\perp,$
since $\mathscr{A}$ is approximable

$\Rightarrow$ $\mathscr{L} \vdash \psi_i \leqslant \phi_i \quad (1 \leqslant i \leqslant k)$

$\Rightarrow$ $\mathscr{L} \vdash (\phi \to (\phi_1 \to \cdots (\phi_k \to \lambda)_\perp \cdots)_\perp$
$\leqslant (\phi \to (\psi_1 \to \cdots (\psi_k \to \lambda)_\perp \cdots)_\perp$

$\Rightarrow$ $a \models_{\mathscr{A}} (\phi \to \psi)_\perp \;\&\; b \models_{\mathscr{A}} \phi.$

(3) $\forall M \in \Lambda.M, \Gamma \models_{\mathscr{A}} \phi \Leftrightarrow M, \rho_\Gamma \models_{\mathscr{A}} \phi.$

The right to left implication is clear, since $\rho_\Gamma \models_{\mathscr{A}} \Gamma$. We prove the converse by induction on M:

$$\begin{aligned}x, \Gamma \models_{\mathscr{A}} \phi &\Leftrightarrow \mathscr{A} \models \Gamma x \leqslant \phi \\ &\Leftrightarrow \mathscr{L} \vdash \Gamma x \leqslant \phi \qquad \text{by (1)} \\ &\Leftrightarrow a_{\Gamma x} \models_{\mathscr{A}} \phi \\ &\Leftrightarrow x, \rho_\Gamma \models_{\mathscr{A}} \phi.\end{aligned}$$

The case for $\lambda x.M$ is proved by induction on ϕ. We show the non-trivial case:

$\bullet$ $\lambda x.M, \rho_\Gamma \models_{\mathscr{A}} (\phi \to \psi)_\perp$

$\Rightarrow$ $M, \rho_\Gamma[x \mapsto a_\phi] \models_{\mathscr{A}} \psi$

$\Rightarrow$ $M, \Gamma[x \mapsto \phi] \models_{\mathscr{A}} \psi$ by (outer) ind. hyp.

$\Rightarrow$ $\lambda x.M, \Gamma \models_{\mathscr{A}} (\phi \to \psi)_\perp$.

$\bullet$ $MN, \rho_\Gamma \models_{\mathscr{A}} \psi$

$\Rightarrow$ $[\![M]\!]^{\mathscr{A}}_{\rho_\Gamma} [\![N]\!]^{\mathscr{A}}_{\rho_\Gamma} \models_{\mathscr{A}} \psi$

$\Rightarrow$ $\exists \phi. [\![M]\!]^{\mathscr{A}}_{\rho_\Gamma} \models_{\mathscr{A}} (\phi \to \psi)_\perp$ & $[\![N]\!]^{\mathscr{A}}_{\rho_\Gamma} \models_{\mathscr{A}} \phi$ by (2)

$\Rightarrow$ $M, \Gamma \models_{\mathscr{A}} (\phi \to \psi)_\perp$ & $N, \Gamma \models_{\mathscr{A}} \phi$ ind. hyp.

$\Rightarrow$ $MN, \Gamma \models_{\mathscr{A}} \psi$.

(4)

(i) $x, \Gamma[x \mapsto \phi] \models_{\mathscr{A}} \psi \Leftrightarrow \mathscr{L} \vdash \phi \leqslant \psi$

(ii) $\lambda x.M, \Gamma \models_{\mathscr{A}} (\phi \to \psi)_\perp \Leftrightarrow M, \Gamma[x \mapsto \phi] \models_{\mathscr{A}} \psi$

(iii) $MN, \Gamma \models_{\mathscr{A}} \psi \Leftrightarrow \exists \phi. M, \Gamma \models_{\mathscr{A}} (\phi \to \psi)_\perp$ & $N, \Gamma \models_{\mathscr{A}} \phi$.

4(i) is proved using (1).

4(ii)

$\bullet$ $\lambda x.M, \Gamma \models_{\mathscr{A}} (\phi \to \psi)_\perp$

$\Rightarrow$ $\forall \rho, a. \rho \models_{\mathscr{A}} \Gamma$ & $a \models_{\mathscr{A}} \phi \Rightarrow [\![\lambda x.M]\!]^{\mathscr{A}}_\rho . a \models_{\mathscr{A}} \psi$

$\Rightarrow$ $\forall \rho. \rho \models_{\mathscr{A}} \Gamma[x \mapsto \phi] \Rightarrow M, \rho \models_{\mathscr{A}} \psi$

since $[\![\lambda x.M]\!]^{\mathscr{A}}_\rho . a = [\![M]\!]^{\mathscr{A}}_{\rho[x \mapsto a]}$,

$\Rightarrow$ $M, \Gamma[x \mapsto \phi] \models_{\mathscr{A}} \psi$.

The converse follows from the soundness of $\mathscr{L}$.

4(iii)

$MN, \Gamma \models_{\mathscr{A}} \psi \Leftrightarrow MN, \rho_\Gamma \models_{\mathscr{A}} \psi$ by (3)

$\Leftrightarrow [\![M]\!]^{\mathscr{A}}_{\rho_\Gamma} [\![N]\!]^{\mathscr{A}}_{\rho_\Gamma} \models_{\mathscr{A}} \psi$

$\Leftrightarrow \exists \phi. [\![M]\!]^{\mathscr{A}}_{\rho_\Gamma} \models_{\mathscr{A}} (\phi \to \psi)_\perp$ & $[\![N]\!]^{\mathscr{A}}_{\rho_\Gamma} \models_{\mathscr{A}} \phi$ by (2)

$\Leftrightarrow \exists \phi. M, \Gamma \models_{\mathscr{A}} (\phi \to \psi)_\perp$ & $N, \Gamma \models_{\mathscr{A}} \phi$ by (3).

We can now prove

$$M, \Gamma \models_{\mathscr{A}} \phi \Rightarrow M, \Gamma \vdash \phi$$

by induction on M, using (4).

(iii) $\Rightarrow$ (i).

First, note that (iii) implies that

$$\mathscr{A} \models \phi \leqslant \psi \Leftrightarrow \mathscr{L} \vdash \phi \leqslant \psi.$$

One half is the Soundness Theorem. For the converse, suppose $\mathscr{A} \models \phi \leqslant \psi$ and $\mathscr{L} \nvdash \phi \leqslant \psi$. Then $\mathbf{I} \models_{\mathscr{A}} (\phi \to \psi)_\perp$ but $\mathbf{I} \nvdash (\phi \to \psi)_\perp$, and so $\mathscr{A}$ is not $\mathscr{L}$-complete.

Now suppose that P is not definable in $\mathscr{A}$, and consider

$$\phi \equiv (\lambda \to (t \to \lambda)_\perp)_\perp \wedge (t \to (\lambda \to \lambda)_\perp)_\perp,$$
$$\psi \equiv (t \to (t \to \lambda)_\perp)_\perp.$$

Clearly, $\mathscr{L} \nvdash \phi \leqslant \psi$. However, for $a \in \mathscr{A}$, if $a \models_{\mathscr{A}} \phi$, then $x\Downarrow$ or $y\Downarrow$ implies $axy\Downarrow$; since P is not definable in $\mathscr{A}$, and in particular, a does not define P, we must have $axy\Downarrow$ even if $x\Uparrow$ and $y\Uparrow$, and hence $a \models_{\mathscr{A}} \psi$. Thus $\mathscr{A} \models \phi \leqslant \psi$ and so by our opening remark, $\mathscr{A}$ is not $\mathscr{L}$-complete.

(ii) $\Rightarrow$ (iv) ($\mathscr{A}$ approximable).

Clearly $\mathsf{Im}\, t_{\mathscr{A}} \supseteq \mathscr{K}(D)$, by 5.14(ii). Also, since $\mathscr{A}$ is approximable, we can apply the Characterisation Theorem to deduce that $t_{\mathscr{A}}$ is injective (modulo bisimulation). To show that $t_{\mathscr{A}}$ is a combinatory morphism, we argue as in 5.4.8. Application is preserved by $t_{\mathscr{A}}$ using (2) from the proof of (ii) $\Rightarrow$ (iii) and 5.4.8. The proof is completed by showing that $t_{\mathscr{A}}$ preserves denotations of λ-terms; i.e.,

$$\forall M \in \Lambda,\ \rho \in \mathsf{Env}(\mathscr{A}).\ t_{\mathscr{A}}([\![M]\!]^{\mathscr{A}}_{\rho}) = [\![M]\!]^{D}_{t_{\mathscr{A}} \cdot \rho}.$$

The proof is by induction on M. Since it is very similar to the corresponding part of the proof of 5.4.8, we omit it. The only non-trivial point is that in the case for abstraction we need

$$\forall a \in A.\, a \models_{\mathscr{A}} \phi \Rightarrow M,\ \rho[x \mapsto a] \models_{\mathscr{A}} \psi$$

if and only if

$$M,\ \rho[x \mapsto a_\phi] \models_{\mathscr{A}} \psi,$$

which is proved similarly to (3) in (ii) $\Rightarrow$ (iii).

(iv) $\Rightarrow$ (v)

Assuming (iv), $\mathscr{A}$ is isomorphic (modulo bisimulation) to a substructure of D. Since formulas in CF are (equivalent to) universal (Π^0_1) sentences, this yields $\mathfrak{Th}^{\mathscr{A}}(D) \subseteq \mathfrak{Th}^{\mathscr{A}}(\mathscr{A})$. Since $\mathscr{K}(D) \subseteq \mathsf{Im}\, t_{\mathscr{A}}$, to prove the converse it is sufficient to show, for $H \in \mathsf{CF}$, that

$$D,\ \rho \nvDash H \Rightarrow \exists \rho_0 : \mathsf{Var} \to \mathscr{K}(D).\, D,\ \rho_0 \nvDash H.$$

Let $H \equiv P \Rightarrow F$, where $P \equiv \bigwedge_{i \in I} M_i\Downarrow$. There are three cases, corresponding to the form of F.

Case 1. $F \equiv M \sqsubseteq N$. $D, \rho \not\models P \Rightarrow F$ implies $D, \rho \models P$ and $D, \rho \not\models M \sqsubseteq N$. Since D is algebraic, $D, \rho \not\models M \sqsubseteq N$ implies that for some $b \in \mathscr{K}(D)$, $b \sqsubseteq [\![M]\!]^D_\rho$ and $b \not\sqsubseteq [\![N]\!]^D_\rho$. Since the expression $[\![M]\!]^D_\rho$ is continuous in ρ, $b \sqsubseteq [\![M]\!]^D_\rho$ implies that for some $\rho_1 : \mathsf{Var} \to \mathscr{K}(D)$, $\rho_1 \sqsubseteq \rho$ and $b \sqsubseteq [\![M]\!]^D_{\rho_1}$. For all ρ' with $\rho_1 \sqsubseteq \rho' \sqsubseteq \rho$, $[\![N]\!]^D_{\rho'} \sqsubseteq [\![N]\!]^D_\rho$, and hence $b \not\sqsubseteq [\![N]\!]^D_{\rho'}$. Again, since D is algebraic,

$$D, \rho \models M_i\Downarrow \Rightarrow \exists \rho_i : \mathsf{Var} \to \mathscr{K}(D).\, \rho_i \sqsubseteq \rho \;\&\; D, \rho_i \models M_i\Downarrow.$$

Now let $\rho_0 \equiv \bigsqcup_{i \in I} \rho_i \sqcup \rho_1$. This is well-defined since D is a lattice. Moreover, $\rho_0 \sqsubseteq \rho$, and $\rho_0 : \mathsf{Var} \to \mathscr{K}(D)$. Since $\rho_0 \sqsupseteq \rho_i (i \in I)$, $D, \rho_0 \models M_i\Downarrow$; and since $\rho_1 \sqsubseteq \rho_0 \sqsubseteq \rho$, $b \sqsubseteq [\![M]\!]^D_{\rho_0}$ and $b \not\sqsubseteq [\![N]\!]^D_{\rho_0}$, and so $D, \rho_0 \not\models M \sqsubseteq N$. Thus $D, \rho_0 \not\models P \Rightarrow F$, as required.

The remaining cases are proved similarly.

(v) $\Rightarrow$ (i) ($\mathscr{A}$ adequate).

Consider the formula

$$H \equiv x\Omega(\mathbf{K}\Omega)\Downarrow \wedge x(\mathbf{K}\Omega)\,\Omega\Downarrow \Rightarrow x\Omega\Omega\Downarrow.$$

It is easy to see that $\mathscr{A} \models H$ iff P is not definable in $\mathscr{A}$. Since P is definable in D, the result follows. ∎

7.2. *Operational Extensionality Results*

We now turn to the question of when the bisimulation preorder on an lts can be characterized by means of a contextual preorder, as in (Bar84, Plo77, Mil77).

LEMMA 7.2.1. *Let $\mathscr{K} = \langle K, \cdot^{\mathscr{K}}, \Downarrow_{\mathscr{K}}, [\![-]\!] \rangle$ be a lazy λ-model. (Recall that $\mathscr{K}$ is a q-ats). Let M, N, P, Q range over $\Lambda(K)^o$. Then the following are equivalent:*

(i) $M \lesssim^{\mathscr{K}} N \Rightarrow \forall C[\,] \in \Lambda(K)^o.\, C[M]\Downarrow_{\mathscr{K}} \Rightarrow C[N]\Downarrow_{\mathscr{K}}$.
contextual preorder

(ii) $M \lesssim^{\mathscr{K}} N \Rightarrow \forall P \in \Lambda(K)^o.\, PM \lesssim^{\mathscr{K}} PN$.
right monotonicity of "·"

(iii) $M \lesssim^{\mathscr{K}} N \;\&\; P \lesssim^{\mathscr{K}} Q \Rightarrow MP \lesssim^{\mathscr{K}} NQ$.
$\lesssim^{\mathscr{K}}$ is a logical relation

(iv) $\mathscr{K} \models (ats)$ *$\mathscr{K}$ is an ats.*

Hence, $\langle K/\sim^{\mathscr{K}}, \cdot^{\mathscr{K}}, \Downarrow_{\mathscr{K}}, [\![-]\!]^{\mathscr{K}} \rangle$ is an lts.

Proof. It is clear that (iii) implies (ii). To show the converse, since $\precsim^{\mathscr{K}}$ is transitive, it suffices to show that

$$M \precsim^{\mathscr{K}} N \Rightarrow \forall P \in \Lambda(K)^o . MP \precsim^{\mathscr{K}} NP$$

which follows immediately from the definition of $\precsim^{\mathscr{K}}$.

Suppose (i) holds and let $M \precsim^{B} N$. For any $P \in \Lambda(K)^o$, consider contexts $C[\ ]$ of the form $(P[\ ])\,\vec{Q}$. Applying (i), we have

$$\forall \vec{Q} \subseteq \Lambda(K)^o . PM\vec{Q} \Downarrow_{\mathscr{K}} \Rightarrow PN\vec{Q} \Downarrow_{\mathscr{K}},$$

which means that $PM \precsim^{\mathscr{K}} PN$. Hence, (ii) is valid.

Suppose (iii) holds. To show (i), it only remains to show that

$$M \precsim^{\mathscr{K}} N \Rightarrow \lambda x . M \precsim^{\mathscr{K}} \lambda x . N$$

which follows from

$$\forall M, N \in \Lambda(K) . M \precsim^{\mathscr{K}} N \Rightarrow \forall P \in \Lambda(K) . M[P/x] \precsim^{\mathscr{K}} N[P/x].$$

By definition of ats, (iv) holds iff (ii) holds; hence, (iv) is equivalent to (i), (ii), and (iii). The last statement is valid by definition of $\precsim^{\mathscr{K}}$ over possibly open terms. ∎

DEFINITION 7.2.2. With reference to the above lemma, $\mathscr{K}$ is said to be *operationally extensional* if any one of the equivalent conditions holds.

DEFINITION 7.2.3. Let $\mathscr{A}$ be a lazy λ-model, $X \subseteq A$. Then $\mathscr{A}$ is *X-adequate* if

$$\forall M \in \Lambda(X)^o . \mathscr{A} \models M\Downarrow \Rightarrow D \models M\Downarrow.$$

Here $[\![M]\!]^D$ is the denotation in D obtained by mapping each $a \in X$ to $t_{\mathscr{A}}(a)$. Note that if we extend our endogenous program logic to terms in $\Lambda(X)^o$, with axioms

$$a, \Gamma \vdash \phi \qquad (\phi \in \mathscr{L}(a)),$$

then the Soundness and Completeness Theorems for D still hold, by a straightforward extension of the arguments used above.

PROPOSITION 7.2.4. *Let $\mathscr{A}$ be an X-adequate lazy λ-model. Then $\mathscr{A}$ is X-approximable; i.e.,*

$$\forall M, N_1, \ldots, N_k \in \Lambda(X)^o . \mathscr{A} \models MN_1 \cdots N_k\Downarrow$$
$$\Rightarrow \exists \phi_1, \ldots, \phi_k . M \models_{\mathscr{A}} (\phi_1 \to \cdots (\phi_k \to \lambda)_\perp \cdots)_\perp \;\&\; \forall i . 1 \leqslant i \leqslant k . N_i \models_{\mathscr{A}} \phi_i.$$

Proof.

$$
\begin{aligned}
\bullet \quad & \mathscr{A} \models MN_1 \cdots N_k \Downarrow \\
\Rightarrow \quad & D \models MN_1 \cdots N_k \Downarrow \\
\Rightarrow \quad & \exists \phi_1, \ldots, \phi_k . M \models_D (\phi_1 \to \cdots (\phi_k \to \lambda)_\perp \cdots)_\perp \\
& \& \; N_i \models_D \phi_i, \; 1 \leqslant i \leqslant k, \text{ since } D \text{ is approximable} \\
\Rightarrow \quad & \exists \phi_1, \ldots, \phi_k . M \vdash (\phi_1 \to \cdots (\phi_k \to \lambda)_\perp \cdots)_\perp \\
& \& \; N_i \vdash \phi_i, \; 1 \leqslant i \leqslant k, \text{ by extended completeness} \\
\Rightarrow \quad & \exists \phi_1, \ldots, \phi_k . M \models_{\mathscr{A}} (\phi_1 \to \cdots (\phi_k \to \lambda)_\perp \cdots)_\perp \\
& \& \; N_i \models_{\mathscr{A}} \phi_i, \; 1 \leqslant i \leqslant k, \text{ by extended soundness.} \quad \blacksquare
\end{aligned}
$$

We now turn to a number of applications of these ideas to syntactically presented lazy λ-models, i.e., "programming languages," and state a general operational extensionality result.

THEOREM 7.2.5 (Operational Extensionality). *Let* $\mathscr{K} = \langle \Lambda(K)^o, \cdot^{\mathscr{K}}, \Downarrow_{\mathscr{K}}, [\![-]\!] \rangle$ *be a lazy* λ*-model which is* K*-adequate. Then,* $\mathscr{K}$ *is operationally extensional.*

Proof. By Proposition 7.2.4, $\mathscr{K}$ is approximable. Then, by Proposition 5.2.6, $\mathscr{K}$ satisfies the axiom (ats); i.e., it is operationally extensional. $\blacksquare$

Since λI is trivially $\varnothing$-adequate, we have

COROLLARY 7.2.6. *λI is operationally extensional.*

Lambda Transition Systems: λI_c, λI_p

Next, we consider extensions of λI.

DEFINITION 7.2.7. (i) λI_c is the extension of λI defined by

$$\lambda I_c \stackrel{\text{def}}{=} \langle \Lambda(\mathsf{C})^o, \Downarrow_c \rangle$$

where $\Downarrow_c$ is the extension of the relation $\Downarrow$ defined in Definition 2.1.1 with the following rules:

$$\frac{}{\mathsf{C} \Downarrow_c \mathsf{C}} \qquad \frac{M \Downarrow_c \mathsf{C} \quad N \Downarrow_c}{MN \Downarrow_c \mathbf{I}}$$

(ii) λI_p is the extension $\langle \Lambda(\mathsf{P})^o, \Downarrow_p \rangle$ of λI with the rules

$$\frac{}{\mathsf{P} \Downarrow_p \mathsf{P}} \qquad \frac{}{\mathsf{P}M \Downarrow_p \mathsf{P}M} \qquad \frac{L \Downarrow_p \mathsf{P} \quad M \Downarrow_p}{LMN \Downarrow_p \mathbf{I}} \qquad \frac{L \Downarrow_p \mathsf{P} \quad N \Downarrow_p}{LMN \Downarrow_p \mathbf{I}}.$$

It is easy to see that the relations $\Downarrow_c$ and $\Downarrow_p$ as defined in λI_c and λI_p respectively are partial functions. Moreover, with these definitions the C and P combinators are convergence testing and parallel convergence testing constants, respectively, while C is definable in λI_p by

$$\mathsf{C}M \stackrel{\text{def}}{=} \mathsf{P}MM.$$

We shall see later that λI_c and λI_p are applicative transition systems (indeed lts's) and we denote the associated bisimulation preorders as $\precsim^c$ and $\precsim^p$, respectively. The respective bisimulation equivalences are denoted as $\sim^c$ and $\sim^p$.

LEMMA 7.2.8. *λI_c is C-adequate. Hence, by Theorem 7.2.5, λI_c is operational extensional.*

Proof. Define

$$c \equiv \{(\lambda \to (\phi \to \phi)_\perp)_\perp \mid \phi \in \mathscr{L}\}^\dagger \in \mathsf{Filt}\,\mathscr{L}.$$

Then it is easy to see that $c \subseteq t_{\mathscr{A}}(\mathsf{C})$, and by monotonicity and the Soundness Theorem,

$$[\![M[c/\mathsf{C}]]\!]^D \subseteq [\![M]\!]^D$$

for $M \in \Lambda^0(\{\mathsf{C}\})$. Thus

$$D \models M[c/\mathsf{C}]\Downarrow \Rightarrow D \models M\Downarrow. \qquad (*)$$

Now we prove that

$$\forall M, N \in \Lambda^0(\{\mathsf{C}\}).$$
$$M \Downarrow N \Rightarrow [\![M[c/\mathsf{C}]]\!]^D = [\![N[c/\mathsf{C}]]\!]^D \;\&\; D \models N[c/\mathsf{C}]\Downarrow, \qquad (**)$$

which by $(*)$ yields $I_C \models M\Downarrow \Rightarrow D \models M\Downarrow$, as required. $(**)$ is proved by a straightforward induction on the length of the proof that $M \Downarrow N$. ∎

LEMMA 7.2.9. *λI_p is P-adequate. Hence, λI_p is operational extensional.*

Proof. The argument for I_P is similar, using

$$p \equiv \{(\lambda \to (t \to (\phi \to \phi)_\perp)_\perp)_\perp \wedge (t \to (\lambda \to (\psi \to \psi)_\perp)_\perp)_\perp : \phi, \psi \in \mathscr{L}\}^\dagger. \quad ∎$$

We now turn to the question of full abstraction for these languages. Since, as we have seen, λI_p is P-adequate, and hence adequate and approximable, and C and P are definable, we can apply the Expansive Full Abstraction Theorem to obtain

PROPOSITION 7.2.10. *D is fully abstract for λI_p.*

8. Non-Full-Abstraction Results

We present two non-full-abstraction results in this section: namely, λI and λI_c are *not* equationally fully abstract with respect to D. Any *lazy reflexive object* in the category of cpo's and continuous functions expands to a lazy λ-model which we call a *topological lazy λ-model.* A corollary of the first non-full-abstraction result is that *adequate* lazy topological λ-models are *incomplete* with respect to fully lazy λ-theories. The second non-full-abstraction result is a consequence of the non-definability of parallel convergence testing in λI_c, which we prove by a syntactic case analysis.

8.1. *λI Is Not Equationally Fully Abstract w.r.t. D*

We state the first negative full-abstraction result.

THEOREM 8.1.1 (Non-Full-Abstraction 1). $\exists M, N \in \Lambda. M \sim^B N \& D \not\models M = N$.

Proof. Let $M \equiv x(\lambda y. x\Xi\Omega y)\,\Xi$, $N \equiv x(x\Xi\Omega)\,\Xi$, where Ω and Ξ represent any strongly unsolvable and $\mathbf{PO}_\perp$ terms, respectively. We show:

I. $M \sim^B N$,

II. $D \not\models M = N$.

I. It is easy to see that $A \in \mathscr{F} \Rightarrow A = \lambda y. Ay$ for $y \notin \mathrm{FV}(A)$. Consequently, to show I, it suffices to show that

$$\forall \sigma : \mathrm{Var} \to \Lambda^o. \sigma(x) = L. L\Xi\Omega\Uparrow \Rightarrow M_\sigma \sim^B N_\sigma,$$

since $\sim^B$ is a congruence relation. We consider the various cases for L such that $L\Xi\Omega\Uparrow$. Now, it should be clear that $L \notin \mathscr{F}_3$ (see Definition 2.3.1). We list the possible cases, modulo β-convertibility, for L such that $L\Xi\Omega\Uparrow$ as follows:

(a) X such that $X \in \mathbf{PO}_0$,

(b) $\lambda x. X$ such that $X \in \mathbf{PO}_0$,

(c) $\lambda x_1 x_2. X$ such that $X \in \mathbf{PO}_0$,

(d) $\lambda x_1 x_2. x_2 \vec{P}$.

We tabulate the respective proper order classes to which M and N belong when instantiated with L corresponding to the above cases:

$\sigma(x)=L$	M_σ	N_σ
(a)	$\mathbf{PO}_0$	$\mathbf{PO}_0$
(b)	$\mathbf{PO}_0$	$\mathbf{PO}_0$
(c)	$\mathbf{PO}_0$	$\mathbf{PO}_0$
(d)	$\mathbf{PO}_\infty$	$\mathbf{PO}_\infty$

Hence, we conclude that $M \sim^B N$.

II. For any environment ρ such that $\rho(x)=c$ where c is the convergence testing constant in D, we have

$$[\![M]\!]_\rho = c(\lambda y.c\top\bot y)\top = \top,$$

$$[\![N]\!]_\rho = c(c\top\bot)\top = c\bot\top = \bot.$$

Hence, $D \not\models M=N$.

Another pair of λ-terms that satisfies I and II above is

$$x(x(\mathbf{K}\Omega)\,\Omega)(\mathbf{K}\Omega), \qquad x(\lambda y.x(\mathbf{K}\Omega)\,\Omega y)(\mathbf{K}\Omega);$$

the proof is entirely similar. ∎

A fortriori, we have the following corollary:

COROLLARY 8.1.2. *λl is not equationally fully abstract with respect to D.*

Topological Incompleteness

It is well known that any reflexive object in the Cartesian closed category of cpo's and Scott continuous functions is a λ-model which we refer to as a *topological λ-model*, following Honsell and Ronchi della Rocca (HdR88). Let E be an object of the same category. We say that E is *lazy reflexive* if $[E \to E]_\perp \lhd E$; i.e., the *lifted* function space of E is a retract into E. It is straightforward to see, by essentially the same arguments as for D, that a lazy reflexive object is a lazy λ-model. We call such a λ-model a *topological lazy λ-model.*

DEFINITION 8.1.3. A lazy λ-model $\mathscr{A} = \langle A, \cdot, \Uparrow, [\![-]\!] \rangle$ is *fully lazy* if

$$\forall m, n \in \omega+1.\forall M \in \mathbf{PO}_m.\forall N \in \mathbf{PO}_n.\mathscr{A} \models M=N \Leftrightarrow m=n.$$

Open Question 8.1.4. It is obvious that a fully lazy topological λ-model is adequate. Is the converse true? That is to say, if all strongly unsolvable terms have the same denotation in a lazy topological λ-model, does it follow that all $\mathbf{PO}_\infty$-terms have the same denotation? (We can infer from the premise that for any fixed $n \in \omega$, all $\mathbf{PO}_n$-terms have the same denotation.)

Are all fully lazy λ-theories induced by fully lazy topological λ-models; i.e., are fully lazy topological λ-models *complete* with respect to fully lazy λ-theories? The answer is no.

Let E be a fully lazy topological λ-model. Then M and N in the proof of the theorem have different denotations in E because convergence testing is definable in E, as is the case in D. We therefore have a simple proof of the following incompleteness result:

COROLLARY 8.1.5 (Topological Incompleteness). *Fully lazy topological λ-models are incomplete with respect to fully lazy λ-theories.*

Compare the corollary with the topological incompleteness result in (HdR88). Another corollary of the Non-Full-Abstraction Theorem is the following:

COROLLARY 8.1.6. *Convergence testing is not definable in λI.*

Proof. If convergence testing were definable in λI by C, say, then with reference to the same M and N in the proof of Theorem 8.1.1, for σ such that $\sigma(x) = C$, we have $M_\sigma \Downarrow$ and $N_\sigma \Uparrow$, contradicting I in the proof. ∎

8.2. λI_c *Is Not Equationally Fully Abstract w.r.t. D*

Given that convergence testing is definable in D and that in the construction of our previous counterexample for the non-full-abstraction of λI with respect to D, convergence testing features so pivotally, it is at least plausible that λI_c might be fully abstract with respect to D. This turns out *not* to be the case.

We now use the sequential nature of λI_c to obtain a negative full abstraction result. This will require a few preliminary notions.

DEFINITION 8.2.1. The *one-step lazy reduction* relation $>$ over terms in $\Lambda(\mathbf{C})$ is defined by extending $>$ as defined in Definition 2.3.4 with the additional rules

$$\frac{}{\mathsf{C}(\lambda x.M) > \mathbf{I}} \qquad \frac{}{\mathsf{C}\mathsf{C} > \mathbf{I}} \qquad \frac{M > M'}{\mathsf{C}M > \mathsf{C}M'}.$$

We then define

$$\gg \stackrel{\text{def}}{=} \text{the reflexive, transitive closure of } >,$$

$$M\uparrow \stackrel{\text{def}}{=} \exists \{M_n : n \in \omega\}.\, M \equiv M_0 \,\&\, \forall n.\, M_n > M_{n+1},$$

$$M\not> \stackrel{\text{def}}{=} M \notin \mathrm{dom}(>),$$

$$M \downarrow N \stackrel{\text{def}}{=} M \gg N \,\&\, N\not>.$$

It is clear that $>$ is a partial function. Note that these relations are being defined over *all* terms, not just closed ones. For closed terms, these new notions are related to the evaluation predicate $\Downarrow_c$ as follows:

PROPOSITION 8.2.2. *For* $M, N \in \Lambda(C)^o$:

(i) $M \Downarrow_c N \Leftrightarrow M \downarrow N$,

(i) $M \Uparrow_c \Leftrightarrow M\uparrow$.

We omit the straightforward proof. The following proposition is basic; it says that "reduction commutes with substitution."

PROPOSITION 8.2.3. $M \gg N \Rightarrow M[P/x] \gg N[P/x]$.

Proof. Straightforward. ▮

Now we come to the basic sequentially property of λI_c from which various non-definability results can be deduced. We call a context of $\Lambda(C)$ of the following shape a *lazy* context:

$$C(\cdots(C([\ ]\vec{P})\vec{Q})\cdots)\vec{R}, n \geqslant 0.$$

n

PROPOSITION 8.2.4. *For* $M \in \Lambda(C)$, *exactly one of the following conditions holds*:

(i) $M\uparrow$,

(ii) $M \downarrow \lambda x.N$,

(iii) $M \downarrow C$,

(iv) $M \downarrow C[x]$ *for some lazy context* $C[\ ] \in \Lambda(C)$.

Proof. Straightforward. ▮

THEOREM 8.2.5 (Non-Full-Abstraction 2). P *is not definable in* λI_c; *hence D is not fully abstract for* λI_c.

Proof. We show that λI_c satisfies

$$x(\mathbf{K}\Omega)\,\Omega \Downarrow_c \ \&\ x\Omega(\mathbf{K}\Omega) \Downarrow_c \Rightarrow x\Omega\Omega \Downarrow_c,$$

and hence, as in the proof of the Expansive Full Abstraction Theorem, P is not definable in λI_c.

Without loss of generality, we consider closed terms of the form $\lambda y_1.\lambda y_2.M$. Assume that

$$(\lambda y_1\, y_2.M)(\mathbf{K}\Omega)\,\Omega \Downarrow_c \ \&\ (\lambda y_1\, y_2.M)\,\Omega(\mathbf{K}\Omega) \Downarrow_c.$$

Applying Proposition 8.2.4, we see that case (i) is impossible; cases (ii) and (iii) imply that $(\lambda y_1\, y_2.M)\,\Omega\Omega\Downarrow_c$; while in case (iv), if $x = y_1$, then $(\lambda y_1\, y_2.M)\,\Omega(\mathbf{K}\Omega)\Uparrow_c$, *contra hypothesis*; and if $x = y_2$, $(\lambda y_1\, y_2.M)(\mathbf{K}\Omega)\,\Omega\Uparrow_c$, also *contra hypothesis*. Thus case (iv) is impossible, and the proof is complete. ∎

The above proof appeals to the Expansive Full Abstraction Theorem. We present a direct proof as a consequence of the following:

PROPOSITION 8.2.6. $\exists M, N \in \Lambda(\mathsf{C}).\, M \sim^c N \,\&\, D \not\models M = N$.

Proof. Consider the terms

$$M \equiv \lambda x.\mathsf{C}(\mathsf{C}(x(\mathbf{K}\Omega)\,\Omega)\,A), \qquad N \equiv \lambda x.\mathsf{C}(\mathsf{C}(x(\mathbf{K}\Omega)\,\Omega)\,B),$$

where $A \equiv (x\Omega\Omega)$, $B \equiv (x\Omega(\mathbf{K}\Omega))$.

We show:

I. $M \precsim^c N \,\&\, N \precsim^c M$,

II. $D \not\models M = N$.

I. Let $C[\ ] \equiv \lambda x.\mathsf{C}(\mathsf{C}(x(\mathbf{K}\Omega)\,\Omega)(x\Omega[\ ]))$. Then, note that $M \equiv C[\Omega]$, $N \equiv C[\mathbf{K}\Omega]$. Since $\Omega \precsim^c \mathbf{K}\Omega$, by the precongruence of $\precsim^c$, we deduce that $M \precsim^c N$. To show $N \precsim^c M$, we only need to show that

$$\forall P \in \Lambda(\mathsf{C})^o.\, NP\Downarrow_c \Rightarrow MP\Downarrow_c. \tag{$*$}$$

This is because, first, we have $N\Downarrow_c$ *and* $M\Downarrow_c$. Next, assume $(*)$ is valid. Then, observe that

$$\begin{aligned} NP\vec{Q}\Downarrow_c &\Rightarrow NP\Downarrow_c \mathbf{I} \,\&\, \vec{Q}\Downarrow_c \\ &\Rightarrow MP\Downarrow_c \mathbf{I} \,\&\, \vec{Q}\Downarrow_c \\ &\Rightarrow MP\vec{Q}\Downarrow_c. \end{aligned}$$

To verify $(*)$, we claim that it suffices to consider those P such that (let σ be the substitution mapping x to P) $B_\sigma\Downarrow_c \,\&\, A_\sigma\Uparrow_c$. This is because, if $B_\sigma\Uparrow_c$, then $NP\Uparrow_c$; if $A_\sigma\Downarrow_c$, then (note that $A_\sigma \equiv P\Omega\Omega$, by $\Omega \precsim^c \mathbf{K}\Omega$ and the precongruence of $\precsim^c$) $P(\mathbf{K}\Omega)\,\Omega\Downarrow_c$, whence $MP\Downarrow_c$. Now, suppose $B_\sigma \equiv P\Omega(\mathbf{K}\Omega)\Downarrow_c$ and $A_\sigma \equiv P\Omega\Omega\Uparrow_c$. By the non-$\lambda$C-definability of P, we must have $P(\mathbf{K}\Omega)\,\Omega\Uparrow_c$, and so, $NP\Uparrow_c$, and we are done.

II. Let parallel convergence testing be definable in D by p, say. Then $[\![M]\!]\, p\Uparrow$ and $[\![N]\!]\, p\Downarrow$ in D. ∎

We refer the reader to another non-full-abstraction result in Section 9, Proposition 9.3.4.

9. Full Abstraction II: A Restrictive Approach

9.1. *Overview of the Section*

We summarize the main results of this section as follows:

The Problem

Given an lts $\mathscr{K} = \langle K, \cdot^{\mathscr{K}}, \Downarrow_{\mathscr{K}}, [\![-]\!]^{\mathscr{K}} \rangle$ with a combinatory interpretation in D, we aim to construct a lazy λ-model $Q^{\mathscr{K}}$ which is a *retract* of D such that $Q^{\mathscr{K}}$ is *fully abstract* with respect to $\mathscr{K}$ by the *restrictive* approach. That is to say, $Q^{\mathscr{K}} \overset{\phi^{\mathscr{K}}}{\rightarrowtail} D \overset{\psi^{\mathscr{K}}}{\twoheadrightarrow} Q^{\mathscr{K}}$ with $\psi^{\mathscr{K}} \circ \phi^{\mathscr{K}} = \mathsf{id}$ and

$$\forall M \in \Lambda(K)^o . [\![M]\!]^{Q^{\mathscr{K}}} \overset{\text{def}}{=} \psi^{\mathscr{K}}([\![M]\!]),$$

satisfying $\forall M, N \in \Lambda(K)^o$,

- $[\![MN]\!]^{Q^{\mathscr{K}}} = [\![M]\!]^{Q^{\mathscr{K}}} \cdot^{\mathscr{K}} [\![N]\!]^{Q^{\mathscr{K}}}$,
- $M \precsim^{\mathscr{K}} N \Leftrightarrow Q^{\mathscr{K}} \models M \sqsubseteq N$.

In the following, we present a sketch of the general strategy to construct a retract $Q^{\mathscr{K}}$ for any such lts $\mathscr{K}$. However, we are only able to *prove* that $Q^{\mathscr{K}}$ is fully abstract for $\mathscr{K}$ for a restricted class of lts's, which includes λI_c and λI_ω, a "labelled" version of λI (to be introduced later).

Construction of $Q^{\mathscr{K}}$

The construction relies on a *bisimulation logical relation*, $\lessdot^{\mathscr{K}}$, between D and $\mathscr{K}$ which captures the extent to which an element d of D *bisimulates* an element M of $\Lambda(K)^o$ with respect to a suite of *tests* consisting of elements of $\Lambda(K)^o$. $\lessdot^{\mathscr{K}} \subseteq D \times \Lambda(K)^o$ satisfies the following recursive specification: $d \lessdot^{\mathscr{K}} M$ iff

- $\forall \vec{P} \subseteq \Lambda(K)^o . D \models d\vec{P}\Downarrow \Rightarrow \mathscr{K} \models M\vec{P}\Downarrow_{\mathscr{K}}$ &
- $\forall e \in D . \forall N \in \Lambda(K)^o . [e \lessdot^{\mathscr{K}} N \Rightarrow de \lessdot^{\mathscr{K}} MN]$.

Thus, $\lessdot^{\mathscr{K}}$ may be seen as a natural extension of the by now familiar notion of bisimulation to one between two *different* lts's. Intuitively, $d \lessdot^{\mathscr{K}} M$ if "all that can be observed about d in D by applying it to terms in $\Lambda(K)^o$ can equally be observed about M in $\mathscr{K}$." $\lessdot^{\mathscr{K}}$ satisfies the property of *arbitrary join inclusiveness*; i.e., for any $X \subseteq D$

$$[\forall x \in X . x \lessdot^{\mathscr{K}} M] \Rightarrow \left(\bigsqcup X\right) \lessdot^{\mathscr{K}} M.$$

Define a preorder $\lesssim^{\mathscr{K}}$ on D as

$$d \lesssim^{\mathscr{K}} e \overset{\text{def}}{=} \forall M \in \Lambda(K)^o . e \lessdot^{\mathscr{K}} M \Rightarrow d \lessdot^{\mathscr{K}} M.$$

$\lesssim^{\mathscr{K}}$ compares the extent to which any two elements in D *bisimulate* elements of $\Lambda(K)^o$. Finally, $Q^{\mathscr{K}}$ is obtained by taking the respective supremums of the equivalence classes induced by the preorder $\lesssim^{\mathscr{K}}$.

Proof of Full Abstraction

We secure a proof of full abstraction by a technique employed in (Mil77, Mul86). Construct for the "model" $Q^{\mathscr{K}}$ and the "language" $\mathscr{K}$, respectively, chains of *approximants* such that, roughly speaking, both the model $Q^{\mathscr{K}}$ and the language $\mathscr{K}$ are appropriate *completions* of their respective chains.

We assume that the *canonical projections* $\langle \psi_n \rangle_{n \in \omega}$ *are definable in the language* $\mathscr{K}$ by $\langle \Psi_n \rangle_{n \in \omega}$ in the sense that $\forall M, N \in \Lambda(K)^o$, and $n \in \omega$, $\mathscr{K}$ satisfies

- $\Psi_0 \sim^{\mathscr{K}} \lambda x.\Omega$,
- $\Psi_{n+1} M \Downarrow_{\mathscr{K}} \Leftrightarrow M \Downarrow_{\mathscr{K}}$,
- $\Psi_{n+1} MN \sim^{\mathscr{K}} \Psi_n(M(\Psi_n N))$,

where Ω represents any divergent element in $\mathscr{K}$, i.e., $\Omega \Uparrow_{\mathscr{K}}$; and that $\mathscr{K}$ is *reflexive*, i.e.,

$$\forall M \in \Lambda(K)^o. [\![M]\!] \lessdot^{\mathscr{K}} M.$$

- For each $i \in \omega$, define $\Lambda(K)_i^o$ as the smallest subset of $\Lambda(K)^o$ containing

$$\{\Psi_i M : M \in \Lambda(K)^o\}$$

closed under application and $\sim^{\mathscr{K}}$. $\mathscr{K}_i =^{\text{def}} \langle \Lambda(K)_i^o, \Downarrow_{\mathscr{K}} \rangle$, the *ith approximant of the language* $\mathscr{K}$, is a well-defined q-aswd and denote the associated bisimulation ordering as $\precsim_i^{\mathscr{K}}$.

- Define for each $i \in \omega$, $Q_i^{\mathscr{K}}$, the *ith approximant of the model*, consisting of the respective supremums of the intersection of D_i and the equivalence classes induced by $\lesssim^{\mathscr{K}}$.

Full abstraction of the "completion," i.e., $\forall M, N \in \Lambda(K)^o$,

$$M \precsim^{\mathscr{K}} N \Leftrightarrow Q^{\mathscr{K}} \models M \sqsubseteq N;$$

then follows from the full abstraction of the approximants, i.e., $\forall M, N \in \Lambda(K)_i^o$,

$$M \precsim_i^{\mathscr{K}} N \Leftrightarrow Q_i^{\mathscr{K}} \models M \sqsubseteq N,$$

by a continuity argument. To summarize, we solve the above-mentioned problem for a class of lts's as follows:

THEOREM 9.1.1 (Restrictive Full Abstraction). *Suppose $\mathscr{K}$ has a combinatory interpretation in D and that it is a reflexive lts in which the projection functions are internally definable. Then, $Q^{\mathscr{K}}$, a lazy λ-model which is a* retract *of D may be constructed such that $Q^{\mathscr{K}}$ is fully abstract with respect to $\mathscr{K}$. That is to say, $Q^{\mathscr{K}} \overset{\phi^{\mathscr{K}}}{\rightarrowtail} D \overset{\psi^{\mathscr{K}}}{\twoheadrightarrow} Q^{\mathscr{K}}$ with $\psi^{\mathscr{K}} \circ \phi^{\mathscr{K}} = \mathsf{id}$ and*

$$\forall M \in \Lambda(K)^o . [\![M]\!]^{Q^{\mathscr{K}}} \overset{\text{def}}{=} \psi^{\mathscr{K}}([\![M]\!]),$$

satisfying $\forall M, N \in \Lambda(K)^o$,

- $[\![MN]\!]^{Q^{\mathscr{K}}} = [\![M]\!]^{Q^{\mathscr{K}}} \cdot^{\mathscr{K}} [\![N]\!]^{Q^{\mathscr{K}}}$,
- $M \lesssim^{\mathscr{K}} N \Leftrightarrow Q^{\mathscr{K}} \models M \sqsubseteq N$. ∎

λI_c satisfies the premises of the theorem, hence a fully abstract model which is a retract of D exists (and can be constructed) for it. Similarly for λI_ω. The same method reduces the full abstraction problem for λI to an open question about the *conservativity* of λI_ω over λI.

Summary of Results

We summarize the full abstraction results established in this paper in the following table:

Languages	Fully abstract models	Fully abstract w.r.t. D
λI_p	D (7.1.3)	Yes (7.2.10)
λI_c	$Q^{\lambda I_c}$ (9.6.5)	No (8.1.2)
λI_ω	$Q^{\lambda I_\omega}$ (9.6.5)	No (9.3.4)
λI	? (9.6.6)	No (8.2.5)

In this section, we also prove the following theorem about the relationships between the λ-theories induced by λI_p, λI_c, λI_ω and λI.

Notation 9.1.2. Let L range over λI_c, λI_ω, λI_p. Define

$$L \restriction \lambda \overset{\text{def}}{=} \{M = N : M, N \in \Lambda^o \ \& \ L \models M = N\}.$$

THEOREM 9.1.3.

$$\lambda I_p \restriction \lambda \subset \lambda I_c \restriction \lambda \subset \lambda I_\omega \restriction \lambda \subseteq \lambda I.$$
$$(9.3.3) \quad (8.2.6)$$

9.2. *Approximants of the Language $\mathscr{K}_i = \langle \Lambda(K)_i^o, \cdot^{\mathscr{K}}, \Downarrow_{\mathscr{K}} \rangle$*

In this section, we define the notion of the *approximants* of languages (which are lts's) given the assumption that *projection maps* are internally

definable. The projection maps in question are essentially appropriate *syntactic* representations of the canonical projection maps from D to D_n. For this class of languages which includes λI_c and λI_ω (to be introduced), we prove an *Operational Approximation Lemma* which relates bisimulation preorders of the *sub-languages* to the *full* language.

Let $\mathscr{K} = \langle K, \cdot^{\mathscr{K}}, \Downarrow_{\mathscr{K}}, [\![-]\!]^{\mathscr{K}} \rangle$ range over lts's for the rest of this subsection. Denote the associated bisimulation preorder and equivalence by $\lesssim^{\mathscr{K}}$ and $\sim^{\mathscr{K}}$, respectively.

DEFINITION 9.2.1. (i) We say that *projection maps are definable* in $\mathscr{K}$ if there exists $\langle \Psi_n \in \Lambda(K)^o : n \in \omega \rangle$ such that for all $M, N \in \Lambda(K)^o$ and $n \in \omega$, $\mathscr{K}$ satisfies

- $\Psi_0 \sim^{\mathscr{K}} \lambda x.\Omega$,
- $\Psi_{n+1} M \Downarrow_{\mathscr{K}} \Leftrightarrow M \Downarrow_{\mathscr{K}}$,
- $\Psi_{n+1} MN \sim^{\mathscr{K}} \Psi_n(M(\Psi_n N))$,

where Ω represents any divergent element in $\mathscr{K}$; i.e., $\Omega \Uparrow_{\mathscr{K}}$. Note that it follows, by an inductive argument, that $\mathscr{K}$ satisfies

$$\Psi_m(\Psi_n M) \sim^{\mathscr{K}} \Psi_{\min(m,n)} M,$$

where $\min(m, n)$ is the minimum of m and n.

(ii) For $i \in \omega$, define $\Lambda(K)_i^o$ inductively as follows:

$$\frac{M \in \Lambda(K)^o}{\Psi_i M \in \Lambda(K)_i^o} \qquad \frac{M_1, \ldots, M_m \in \Lambda(K)_i^o \quad m \geqslant 1}{(M_1 \cdots M_m) \in \Lambda(K)_i^o} \qquad \frac{M \in \Lambda(K)_i^o \quad M \sim^{\mathscr{K}} N}{N \in \Lambda(K)_i^o}.$$

In other words, $\Lambda(K)_i^o$ is the least subset of $\Lambda(K)^o$ containing

$$\{\Psi_i M : M \in \Lambda(K)^o\}$$

which is closed under application and $\sim^{\mathscr{K}}$.

Notation 9.2.2. We abbreviate $\Psi_i M$ as M^i.

Clearly, $\mathscr{K}_i \stackrel{\text{def}}{=} \langle \Lambda(K)_i^o, \cdot^{\mathscr{K}}, \Downarrow_{\mathscr{K}} \rangle$, the *ith approximant of* $\mathscr{K}$ with $\cdot^{\mathscr{K}}$ and $\Downarrow_{\mathscr{K}}$ subject to the obvious restriction is a q-aswd. Denote the associated bisimulation preorder and equivalence by $\lesssim_i^{\mathscr{K}}$ and $\sim_i^{\mathscr{K}}$, respectively.

Remark 9.2.3. In the above definition, we have assumed that $\mathscr{K}$ is an lts. However, for the respective notions of the internal definability of the projection maps and approximants of the language to make sense, it

suffices for $\mathscr{K}$ to be a lazy λ-model. Specifically, $\mathscr{K}$ does not need to be operationally extensional.

Remark 9.2.4. We have $\forall M \in \Lambda(K)_i^o . M^i \sim^{\mathscr{K}} M$. Also, $\forall M \in \Lambda(K)_i^o . M \sim^{\mathscr{K}} M^{i+1}$ which implies that

$$\forall i \in \omega . \Lambda(K)_i^o \subseteq \Lambda(K)_{i+1}^o .$$

LEMMA 9.2.5 (Operational Approximation). *Let* $\mathscr{K} = \langle K, \cdot^{\mathscr{K}}, \Downarrow_{\mathscr{K}}, [\![-]\!]^{\mathscr{K}} \rangle$ *be an lts in which projection maps are definable and let* $i \in \omega$.

(i) *Let* $M, N \in \Lambda(K)^o$. *Then* $M \precsim^{\mathscr{K}} N \Rightarrow \forall i \in \omega . M^i \precsim_i^{\mathscr{K}} N^i$.

(ii) *Let* $M, N \in \Lambda(K)_i^o$. *Then* $M \precsim_i^{\mathscr{K}} N \Rightarrow M \precsim^{\mathscr{K}} N$.

Proof. (i) By an appeal to the fact that $\precsim^{\mathscr{K}}$ is a logical relation. For (ii), let $M, N \in \Lambda(K)_i^o$ such that $M \precsim_i^{\mathscr{K}} N$. Let $X_1, \ldots, X_l \subseteq \Lambda(K)^o$ for $l \in \omega$. Then, for $l \leqslant i$, and by Remark 9.2.4,

$$\begin{aligned} MX_1 \cdots X_l &\sim^{\mathscr{K}} M^i X_1 \cdots X_l && \text{Definition 9.2.1} \\ &\sim^{\mathscr{K}} (MX_1^{i-1} \cdots X_l^{i-1})^{i-l} . \end{aligned}$$

Observe that $X_j^{i-j} \in \Lambda(K)_i^o$. Now, by Definition 9.2.1

$$\begin{aligned} MX_1 \cdots X_l \Downarrow_{\mathscr{K}} &\Rightarrow MX_1^{i-1} \cdots X_l^{i-l} \Downarrow_{\mathscr{K}} && \text{by premise} \\ &\Rightarrow NX_1^{i-1} \cdots X_l^{i-l} \Downarrow_{\mathscr{K}} \\ &\Rightarrow NX_1 \cdots X_l \Downarrow_{\mathscr{K}} . \end{aligned}$$

We have thus shown that $M \precsim^{\mathscr{K}} N$. ∎

COROLLARY 9.2.6. *Let* $M, N \in \Lambda(K)_i^o$. *Then*

$$M \sim^{\mathscr{K}} N \Leftrightarrow M \sim_i^{\mathscr{K}} N .$$

Hence, for $M \in \Lambda(K)_i^o$, *by Remark* 9.2.4, $M \sim_i^{\mathscr{K}} M^i$. *Note also, that as a consequence of the Corollary,* $\mathscr{K}_i$ *is an aswd.*

Projection Maps Are Definable in λl_c

Recall the sequence of λC-terms $\langle \Psi_n : n \in \omega \rangle$ (see Definition 4.4.6) defining the respective canonical projections of D to D_n.

LEMMA 9.2.7. *Let* $M, N, P \in \Lambda(\mathsf{C})^o$ *and* $n, m \in \omega$.

(i) $M\Downarrow_{\mathsf{c}} \Leftrightarrow \Psi_{n+1} M\Downarrow_{\mathsf{c}}$.

(ii) $\Psi_n(\Psi_m P) \sim^{\mathsf{c}} \Psi_{\min(m,n)} P$.

(iii) $(\Psi_{n+1} M)\, N \sim^{\mathsf{c}} \Psi_n(M(\Psi_n N)) \sim^{\mathsf{c}} (\Psi_{n+1} M)(\Psi_n N)$.

Hence, projection maps are definable in λI_{c}.

Proof. (i) is immediate from the definition of Ψ_{n+1}. We prove the first equivalence of (iii) by induction on n. The base case of $n=0$ is trivial. We have

$$\begin{aligned}(\Psi_{n+1} M)\, N &\sim^{\mathsf{c}} \mathsf{C}M(\lambda y.\, \Psi_n(M(\Psi_n y)))\, N \\ &\sim^{\mathsf{c}} \mathsf{C}M(\Psi_n(M(\Psi_n N))).\end{aligned}$$

The last step is justified because $\mathsf{C}MPN \sim^{\mathsf{c}} \mathsf{C}M(PN)$ which may be seen to hold by a simple case analysis according to whether $M\Downarrow_{\mathsf{c}}$ or $M\Uparrow_{\mathsf{c}}$. By the same argument, by definition of Ψ_n, we have $\mathsf{C}M(\Psi_n(M(\Psi_n N)) \sim^{\mathsf{c}} (\Psi_n(M(\Psi_n N))$. Hence, $(\Psi_{n+1} M)\, N \sim^{\mathsf{c}} \Psi_n(M(\Psi_n N))$.

Next, we prove (ii) by induction on $n+m=k$. The base case of $k=0$, and the cases of $n, m \leqslant 1$ are trivial. Suppose true for $n+m=k$. Now, let $m+n=k+1$. Without loss of generality, assume $n, m \geqslant 1$. Then, by (i),

$$\begin{aligned}\Psi_n(\Psi_m P) &\sim^{\mathsf{c}} \mathsf{C}(\Psi_m P)(\lambda y.\, \Psi_{n-1}((\Psi_m P)(\Psi_{n-1} y))) && \text{(i) and (iii)} \\ &\sim^{\mathsf{c}} \mathsf{C}P(\lambda y.\, \Psi_{n-1}(\Psi_{m-1}(P(\Psi_{m-1}(\Psi_{n-1} y))))) && \text{ind. hyp.} \\ &\sim^{\mathsf{c}} \mathsf{C}P(\lambda y.\, \Psi_{\min(n-1,m-1)}(P(\Psi_{\min(n-1,m-1)}))) \\ &\sim^{\mathsf{c}} \Psi_{\min(n,m)} P.\end{aligned}$$

Finally, we apply (ii) and the first equivalence of (iii) to prove the second equivalence of (iii).

$$\begin{aligned}(\Psi_{n+1} M)(\Psi_n N) &\sim^{\mathsf{c}} \Psi_n(M(\Psi_n(\Psi_n N))) && \text{(ii)} \\ &\sim^{\mathsf{c}} \Psi_n(M(\Psi_n N)) \\ &\sim^{\mathsf{c}} (\Psi_{n+1} M)\, N. \quad \blacksquare\end{aligned}$$

The Lambda Transition System $\lambda l_\omega = \langle (\Lambda^\omega)^o, \Downarrow_\omega \rangle$

LEMMA 9.2.8. *Projection maps are not definable in* λI.

Proof. Suppose for a contradiction that a λ-term X defines a projection map; then it *discriminates* between convergent and divergent elements in the following elementary way:

$$\forall M \in \Lambda^o . \begin{cases} XM\Downarrow & \text{if } M\Downarrow, \\ XM\Uparrow & \text{if } M\Uparrow. \end{cases}$$

By an argument similar to that in the proof of Lemma 4.4.1, we see that the only such term X is the identity **I**. ∎

We define a new q-aswd $\lambda l_\omega = \langle (\Lambda^\omega)^o, \Downarrow_\omega \rangle$ with $\Lambda^\omega =^{\text{def}} \Lambda(\langle \Psi_n : n \in \omega \rangle)$ which is essentially λl augmented with the *formal constants* that simulate the projection maps. The preceding lemma shows that it is not superfluous to introduce λl_ω. Note that the constants Ψ_n here are not to be confused with the λC-terms called by the same name which were introduced previously: the context in which they occur should clarify which of the two is meant.

DEFINITION 9.2.9. We write $\Lambda^\omega =^{\text{def}} \Lambda(\langle \Psi_n : n \in \omega \rangle)$. The binary reduction relation $\Downarrow_\omega \subseteq (\Lambda^\omega)^o \times (\Lambda^\omega)^o$ ($M \Downarrow_\omega N @ n$ means "M reduces to N in n steps") is defined inductively as follows:

$$\frac{}{\Psi_n \Downarrow_\omega \Psi_n @ 0} \qquad \frac{}{\lambda x.P \Downarrow_\omega \lambda x.P @ 0}$$

$$\frac{M \Downarrow_\omega \Psi_{n+1} @ m \quad N \Downarrow_\omega @ l}{MN \Downarrow_\omega \lambda y.\, \Psi_n(N(\Psi_n\, y)) @ m+l+1} \qquad \frac{M \Downarrow_\omega \lambda x.P @ m \quad P[Q/x] \Downarrow_\omega N @ l}{MQ \Downarrow_\omega N @ m+l+1}$$

As usual, when there is no need to keep track of the number of proof steps, we simply write $M \Downarrow_\omega N$. Define the bisimulation preorder $\precsim^\omega$ and equivalence $\sim^\omega$ accordingly.

We provide a *relative interpretation* of $\Downarrow_\omega$ in $\Downarrow_{\mathsf{C}}$ via a syntactic translation

$$(\cdots)' : \Lambda^\omega \to \Lambda(\mathbf{C}),$$

where $M' =^{\text{def}}$ "M with all occurrences of the *formal constants* Ψ_i replaced by the λC-*terms* Ψ_i for each $i \in \omega$."

LEMMA 9.2.10. *Let* $M \in (\Lambda^\omega)^o$. $M \Downarrow_\omega N \Leftrightarrow M' \Downarrow_{\mathsf{C}} N'$.

Proof. Straightforward induction according to the rules in Definitions 9.2.9 and 7.2.7, respectively. ∎

As a helpful consequence, we have the following:

COROLLARY 9.2.11. *Let* $M, N \in (\Lambda^\omega)^o$. *Then* $M' \precsim^{\mathsf{C}} N' \Rightarrow M \precsim^\omega N$. *The converse is not true* (*see Proposition* 9.3.3).

Since projection maps are definable in λI_c, by an appeal to the preceding corollary, we conclude that

LEMMA 9.2.12. *Projection maps are definable in* λI_ω.

PROPOSITION 9.2.13 (Operational Extensionality). λI_ω *is operationally extensional.*

We present two proofs. The first uses only first principles and is an extension of the method used in the proof of Proposition 2.2.4. The second proof appeals to the general operational extensionality result in Theorem 7.2.5.

Proof. It suffices to show the following: Let M, N range over $(\Lambda^\omega)^o$. We have

$$M \sqsubseteq^\omega N \Rightarrow \forall l \in \omega. \forall C[\ ] \in (\Lambda^\omega)^o. C[M] \Downarrow_\omega @\ l \Rightarrow C[N] \Downarrow_\omega.$$

We prove by induction on l. The base case is obvious. Without loss of generality, consider the following three syntactic classes of closed contexts:

(1) $C[\ ] \equiv (\lambda x. P[\ ])(Q[\ ])\ \vec{R}[\ ]$,

(2) $C[\ ] \equiv \Psi_{n+1}(P[\ ])\ \vec{R}[\ ]$,

(3) $C[\ ] \equiv [\ ](P[\ ])\ \vec{R}[\ ]$.

Now, suppose $C[M] \Downarrow_\omega @\ l+1$.

(1) The proof corresponding to this case is entirely similar to case (1) in the proof of Proposition 2.2.4.

(2) First, consider the case where $C[\ ] \equiv \Psi_{n+1}(P[\ ])$. Since $C[M] \Downarrow_\omega @\ l+1$, we have $P[M] \Downarrow_\omega @\ l$. By the induction hypothesis, $P[N] \Downarrow_\omega$, and so $C[N] \Downarrow_\omega$. Now, consider $C[\ ] \equiv \Psi_{n+1}(P[\ ])(Q[\ ])\ \vec{R}[\ ]$. Let $D[\ ] \equiv (\lambda x. \Psi_n(P[\ ](\Psi_n x)))(Q[\ ])\ \vec{R}[\ ]$. Since $C[M] \Downarrow_\omega @\ l+1$, we have

$$\Psi_{n+1} P[M]\ Q[M]\ \vec{R}[M] > D[M] \Downarrow_\omega @\ l,$$

where $>$ is the one-step leftmost reduction relation associated with $\Downarrow_\omega$. Applying the induction hypothesis to $D[M]$, we have $D[N] \Downarrow_\omega$. Now, it is easy to see that

$$\forall S, T, \vec{U} \in (\Lambda^\omega)^o. \Psi_{n+1} S T \vec{U} \sim^\omega (\lambda x. \Psi_n(S(\Psi_n x)))\ T \vec{U},$$

whence we conclude that $C[N] \Downarrow_\omega$.

(3) Without loss of generality, M has either of the two following syntactic shapes:

a. $M \equiv (\lambda x.S)\,\vec{T}$,

b. $M \equiv \Psi_{n+1}\vec{T}$.

a. Define $D[\ ] \equiv (\lambda x.S)\,\vec{T}(P[\ ])\,\vec{R}[\ ]$. Then $C[M] \equiv D[M]$. by (1), we have $D[N]\Downarrow_\omega$. But $D[N] \equiv MP[N]\,\vec{R}[N]$, and so by an appeal to $M \lesssim^\omega N$, we have $C[N] \equiv NP[N]\,\vec{R}[N]\Downarrow_\omega$.

b. Define $D[\ ] \equiv \Psi_{n+1}\vec{T}(P[\ ])\,\vec{R}[\ ]$. Note that $C[M] \equiv D[M]$. By (2), we have $D[N]\Downarrow_\omega$; by a similar argument to that for case 3a, we conclude that $C[N]\Downarrow_\omega$. ∎

Proof. By Theorem 7.2.5, it suffices to show that λI_ω is $\{\Psi_n : n \in \omega\}$-adequate.

Define for $n \in \omega$, $F_n : \mathscr{L} \to \mathscr{L}$ as follows:

$$F_0(\phi) \stackrel{\text{def}}{=} t,$$

$$F_{n+1}(t) \stackrel{\text{def}}{=} t,$$

$$F_{n+1}(\phi \wedge \psi) \stackrel{\text{def}}{=} F_{n+1}(\phi) \wedge F_{n+1}(\psi),$$

$$F_{n+1}((\phi \to \psi)_\perp) \stackrel{\text{def}}{=} (F_n(\phi) \to F_n(\psi))_\perp.$$

Note that $F_n(F_n\phi) = F_n(\phi)$. Define

$$\mathbf{P}_n \stackrel{\text{def}}{=} \{(F_n(\phi) \to F_n(\psi))_\perp : \phi \in \mathscr{L}\}^\dagger \in \mathsf{Filt}\,\mathscr{L}.$$

We claim that

$$\forall n \in \omega.\, \mathbf{P}_n \subseteq t_{\lambda I_\omega}(\Psi_n),$$

where $t_{\lambda I_\omega}$ is the unique morphism from λI_ω to D in the category **qATS**.

We prove the claim by induction on n. The base case is trivial. Suppose the claim is true for some $n \geqslant 0$. It suffices to show that

$$M \models_{\lambda I_\omega} F_{n+1}(\phi) \Rightarrow \Psi_{n+1} \models_{\lambda I_\omega} F_{n+1}(\phi)$$

for $F_{n+1}(\phi)$ of the shape $(\psi \to (\psi' \to \psi'')_\perp)_\perp$ in view of the axiom $((\to_\perp -\ \wedge)$ in Definition 5.3.1, the definition of $F_{n+1}(\phi_1 \wedge \phi_2)$, and the Soundness Theorem 5.3.2. Let $M \models_{\mathscr{L}} F_{n+1}(\phi) \equiv (\psi \to (\psi' \to \psi'')_\perp)_\perp$ for some $\phi \in \mathscr{L}$. Note that $(\psi' \to \psi'')_\perp \equiv F_n(\theta)$ and $\psi \equiv F_n(\theta')$ for some $\theta, \theta' \in \mathscr{L}$.

Then, we aim to prove that

$$\Psi_{n+1}M \models_{\mathscr{L}} (\psi \to (\psi' \to \psi'')_\perp)_\perp. \qquad (\dagger)$$

Clearly, $\Psi_{n+1} M \Downarrow_\omega$, and so, to prove (†), it suffices to prove that

$$N \models_{\mathscr{L}} \psi \Rightarrow \Psi_{n+1} MN \sim^\omega \Psi_n(M(\Psi_n N)) \models_{\mathscr{L}} (\psi' \to \psi'')_\perp ; \qquad (\ddagger)$$

the preceding equivalence $\sim^\omega$ is justified because projection maps are definable in λI_ω.

By induction hypothesis, $\forall \phi \in \mathscr{L}. \Psi_n \models_{\mathscr{L}} (F_n(\phi) \to F_n(\phi))_\perp$. It follows that $\Psi_n N \models_{\mathscr{L}} \psi \equiv F_n(\theta')$, and so $M(\Psi_n N) \models_{\mathscr{L}} (\psi' \to \psi'')_\perp \equiv F_n(\theta)$. (‡) then follows from another appeal to the induction hypothesis.

The rest of the proof follows that of Lemma 7.2.8. ▮

It follows from the Proposition that λI_ω (i.e., the associated quotient structure $\langle (\Lambda^\omega)^o / \sim^\omega, \Downarrow_\omega \rangle$) is an lts.

A Conjecture: λI_ω *Is a Conservative Extension over* λI

Λ^ω is reminiscent of the classical labelled λ-terms (see (Wad78, Hyl76, Bar84)): the only difference is that the *labels* themselves, i.e., Ψ_i, which belong to Λ^ω, are not labelled terms. Λ^o may be seen as the collection of *limits* of all "partial" terms (i.e., those with labels) in $(\Lambda^\omega)^o$ with respect to the obvious ordering induced by the labels. In other words, given any term $M \in (\Lambda^\omega)^o$, there is a term $|M| \in \Lambda^o$ defined by replacing all Ψ_i with $\mathbf{I}$, which M approximates. Now, it is obvious that for $M, N \in \Lambda^o$,

$$M \precsim^\omega N \Rightarrow M \precsim^B N, \qquad (*)$$

because $\Downarrow_\omega$ extends $\Downarrow$ conservatively. We conjecture the converse of (*).

Conjecture 9.2.14 (Conservativity). Let $M, N \in \Lambda^o$.

(i) $M \precsim^B N \Rightarrow M \precsim^\omega N$.

(ii) $M \sim^B N \Rightarrow M \sim^\omega N$.

The fact that $(\Lambda^\omega)^o$ is a superset of Λ^o gives us some grounds to suspect the *falsity* of the above implications because in general, with an enlarged suite of tests available in $(\Lambda^\omega)^o$, one expects to differentiate more finely between terms. However, the new tests consist in *approximants* of terms which *already exist* in Λ^o, from which one might perhaps surmise that nothing *really* new or of much consequence is introduced in $(\Lambda^\omega)^o$. We shall see that if Conjecture 9.2.14(i) (and respectively (ii)) is indeed true, then our argument for the construction of an inequationally (and respectively equationally) fully abstract model for λI is valid.

9.3. *Relationships between* λl, λl_ω, λl_c, *and* λl_p

LEMMA 9.3.1. *Convergence testing is not definable in* λl_ω.

Remark 9.3.2. A context $C[\ ]$ of Λ^ω is *lazy* if it has the shape

$$C[\ ] \equiv \Psi_{i_n}(\cdots(\Psi_{i_1}([\ ]\,\vec{P}^0)\,\vec{P}^1)\cdots)\,\vec{P}^n,$$

where $\vec{P}^i \equiv P^i_1, \ldots, P^i_{l_i}$ and $n \geqslant 0$.

Observe that for any lazy context $C[\ ]$,

$$C[\mathbf{K}\Omega] \sqsubseteq^\omega \mathbf{K}\Omega.$$

Proof. Suppose, for a contradiction, $X \in (\Lambda^\omega)^o$ defines convergence testing. It is easy to see that X must have the syntactic shape $\lambda x.C[x]$ for some lazy context $C[\ ]$. Now, since $\mathbf{K}\Omega\Downarrow_\omega$, $X(\mathbf{K}\Omega) \sim^\omega \mathbf{I}$, but this contradicts the preceding remark. ▮

PROPOSITION 9.3.3. $\exists M, N \in \Lambda^o.\, M \sim^\omega N \,\&\, M \not\sim^c N$.

Proof. Let $M \equiv x(x(\mathbf{K}\Omega)\,\Omega)(\mathbf{K}\Omega)$ and $N \equiv x(\lambda y.x(\mathbf{K}\Omega)\,\Omega y)(\mathbf{K}\Omega)$. It should be clear that $N \not\simeq^c M$. It then remains to show $M \sim^\omega N$. Since λl_ω is an lts, it satisfies the conditional-η axiom (Remark 3.3.5). For the same reason as in the proof of Theorem 8.1.1, it suffices to show that

$$\forall L \in (\Lambda^\omega)^o.\, L(\mathbf{K}\Omega)\,\Omega\Uparrow_\omega \Rightarrow [L(\lambda y.L(\mathbf{K}\Omega)\,\Omega y)(\mathbf{K}\Omega) \sim^\omega L(L(\mathbf{K}\Omega)\,\Omega)(\mathbf{K}\Omega)].$$

We tabulate the various cases of L such that $L(\mathbf{K}\Omega)\,\Omega\Uparrow_\omega$ and the corresponding values of M_σ and N_σ where $\sigma(x) = L$. In the following table,

- $\perp$ ranges over any $X \in (\Lambda^\omega)^o$ such that $X\Uparrow_\omega$;
- $C[\ ]$ ranges over all lazy contexts; and
- $D[\ ]$ ranges over lazy contexts (refer to Remark 9.3.2) that satisfy

$$n \geqslant 1 \,\&\, \max\{|\vec{P}^0|, \ldots, |\vec{P}^n|\} \geqslant 1.$$

$\sigma(x) = L$	$M_\sigma \sim^\omega L\Omega(\mathbf{K}\Omega)$	$N_\sigma \sim^\omega L(\mathbf{K}\Omega)(\mathbf{K}\Omega)$
$\perp$	$\perp$	$\perp$
$\lambda x.\perp$	$\perp$	$\perp$
Ψ_n	$\perp$	$\perp$
$\lambda x.C[x]$	$\perp$	$\perp$
$\lambda x_1 x_2.D[x_1]$	$\perp$	$\perp$
$\lambda x_1 x_2.C[x_2]$	X	Y

We claim that $X \sim^{\omega} Y$. To see this, observe that

$$\Psi_{i_n}(\cdots(\Psi_{i_1}((\mathbf{K}\Omega)\,\vec{P}^0)\,\vec{P}^1)\cdots)\,\vec{P}^n \sim^{\omega} \Psi_{i_n}(\cdots(\Psi_{i_1}((\mathbf{K}\Omega)\,\vec{Q}^0)\,\vec{Q}^1)\cdots)\,\vec{Q}^n$$
$$\Leftarrow \forall i.\, 1 \leqslant i \leqslant n.\, |\vec{P}^i| = |\vec{Q}^i|.$$

Hence, we are done. ∎

COROLLARY 9.3.4. *λI_{ω} is not fully abstract w.r.t. D.*

We compare the lazy λ-theories induced by the lts's λI_{ω}, λI_{c}, and λI_{p} and their expressiveness in terms of the definability of various functions as follows.

Notation 9.3.5. Let L range over λI_{c}, λI_{ω}, λI_{p}. Define

$$L \restriction \lambda \stackrel{\text{def}}{=} \{M = N\colon M, N \in \Lambda^o \,\&\, L \models M = N\}.$$

THEOREM 9.3.6. *$\lambda I_{\mathrm{p}} \restriction \lambda \subset \lambda I_{\mathrm{c}} \restriction \lambda \subset \lambda I_{\omega} \restriction \lambda \subseteq \lambda I$. If Conjecture 9.2.14 is true, then the last inclusion may be replaced by equality. Note that $\subset$ means strict inclusion.*

Proof. The first strict inclusion is proved by appealing to Theorem 8.2.6 and the fact that C may be simulated in λI_{p}. For the second strict inclusion, see Corollary 9.2.11 and Proposition 9.3.3. The last inclusion is obvious. ∎

We summarize the definability results obtained so far as follows:

Definability	λI	λI_{ω}	λI_{c}	λI_{p}
Projection maps		√	√	√
Convergence testing			√	√
Parallel convergence				√

9.4. *Bisimulation Logical Relations and Their Properties*

In this section, we study *bisimulation logical relations*. Hitherto, we have considered only how an element of an lts might *bisimulate* another element of the *same* lts, and this with respect to a suite of tests (which are again elements of the same lts) consisting of "inputs arguments" to be applied to. Bisimulation logical relations may be regarded as the natural extension of the notion of bisimulation to one between elements of *different* lts's. Given any lts $\mathscr{K}$, $\lessdot^{\mathscr{K}}$, the bisimulation logical relation between D and $\mathscr{K}$ is by construction a *logical relation*. Further, $\lessdot^{\mathscr{K}}$ is *arbitrary join inclusive*. We consider specifically $\lessdot$, $\lessdot^{\omega}$, and $\lessdot^{\mathrm{c}}$, bisimulation logical relations between D and λI, between D and λI_{ω} and between D and λI_{c}, respectively. We show that all three are *reflexive*; i.e., $\forall M \in \Lambda(K)^o.\, [\![M]\!] \lessdot^{\mathscr{K}} M$.

Introduction

In this subsection, we introduce a class of hereditarily defined relations known as *logical relations*. Logical relations were used by Gordon Plotkin in (Plo73) to tackle definability problems in lambda calculus and domain theory. We refer the reader to (Sta85) for further historical remarks and a survey of some powerful applications of logical relations. In our work, given an lts $\mathscr{K}$, logical relations are employed to construct retracts of D with accompanying *compositional* semantic functions with respect to the interpretation of λK-terms. This is needed specifically for the construction of fully abstract models for λI_c and λI_ω. The connection between logical relations and the existence of surjective homomorphic functions is crucial to our construction, as is the case in most applications—a point emphasized in (AL86).

Following (Plo73), we introduce the notion of a logical relation:

DEFINITION 9.4.1. A relation $R \subseteq \prod_{i \in \mathscr{I}} K_i$, where each $\mathscr{K}_i =^{\text{def}} \langle K_i, \Downarrow_i \rangle$ is an lts, is a *logical relation* if $\vec{d} \in R \Rightarrow \forall \vec{e} \in R. \vec{d} \cdot \vec{e} \in R$, where $\vec{d} \cdot \vec{e} =^{\text{def}} \langle d_i e_i : i \in \mathscr{I} \rangle$.

Notation 9.4.2. (i) In the following, let $\mathscr{K} =^{\text{def}} \langle K, \Downarrow_{\mathscr{K}} \rangle$ range over lambda transition sytems with a prescribed interpretation in D. The associated bisimulation preorder and equivalence are denoted $\precsim^{\mathscr{K}}$ and $\sim^{\mathscr{K}}$ respectively.

(ii) For simplicity, we omit the decorated brackets of the semantic function $[\![-]\!]$ in the following context: for $d \in D$ and $\vec{P} \subseteq \Lambda(K)^o$,

$$d\vec{P} \equiv dP_1 \cdots P_n \stackrel{\text{def}}{=} d[\![P_1]\!] \cdots [\![P_n]\!],$$

where $[\![P]\!]$ is the denotation in D obtained by mapping each $k \in K$ to D via the prescribed interpretation.

Specification

We prescribe a recursive specification of the *bisimulation logical relations* we seek: $\lessdot^{\mathscr{K}} \subseteq D \times \Lambda(K)^o$, where $d \lessdot^{\mathscr{K}} M$ iff

$$\forall \vec{P} \subseteq \Lambda(K)^o. [D \models d\vec{P}\Downarrow \Rightarrow \mathscr{K} \models M\vec{P}\Downarrow_{\mathscr{K}}]$$
$$\& \quad \forall e \in D. \forall N \in \Lambda(K)^o. [e \lessdot^{\mathscr{K}} N \Rightarrow de \lessdot^{\mathscr{K}} MN].$$

Remark 9.4.3. Let $a, b \in K$. Recall that $a \precsim^{\mathscr{K}} b \Leftrightarrow \forall \vec{d} \subseteq K. a\vec{d}\Downarrow_{\mathscr{K}} \Rightarrow b\vec{d}\Downarrow_{\mathscr{K}}$. The intended reading of the bisimulation ordering is "for all sequences of *tests*, $\vec{d}$, to which a and b are subjected, all that can be observed about a can equally be observed about b." Note that the tests are themselves elements of the *same* lts and that only convergence is observable.

The logical relation $d \lessdot^{\mathscr{K}} M$ may be seen as a generalization of the notion of bisimulation ordering to one between elements of *possibly different* lts's. We may read $d \lessdot^{\mathscr{K}} M$ as "for all sequences of elements of $\mathscr{K}$ (or equivalently, λK-definable elements of D) as tests, all that can be observed about d can equally be observed about M." In other words, $d \lessdot^{\mathscr{K}} M$ means that with respect to the elements of $\mathscr{K}$ as tests, M contains at least as much operational information as d.

The second implication in the above recursive specification (which is what qualifies $\lessdot^{\mathscr{K}}$ as a *logical* relation) may be seen as a generalized notion of precongruence between elements of two (or more) different lts's.

Existence of the Bisimulation Logical Relation $\lessdot^{\mathscr{K}}$

Following (MP87), we show that a non-trivial relation satisfying the above recursive specification does indeed exist and prove some salient properties satisfied by the relation. Note that the existence proof is general enough to apply to bisimulation logical relations between any two lts's, provided the first lts has *approximable application* as defined in (Lon83) and (Ong88b, Chap. 3).

We begin by defining inductively a family of relations $\lessdot_n^{\mathscr{K}}$ on $D_n \times \Lambda(K)^o$ which are intended to be successive approximations to the targeted logical relation $\lessdot^{\mathscr{K}}$. Recall the following embedding–projection pairs:

$$D_n \overset{\phi_n}{\rightarrowtail} D \overset{\psi_n}{\twoheadrightarrow} D_n, \qquad D_n \overset{i_n}{\rightarrowtail} D_{n+1} \overset{j_n}{\twoheadrightarrow} D_n.$$

DEFINITION 9.4.4. For $n \in \omega$, define inductively $\lessdot_n^{\mathscr{K}} \subseteq D_n \times \Lambda(K)^o$ as follows:

$$d \lessdot_0^{\mathscr{K}} M \overset{\text{def}}{=} t;$$

$$d \lessdot_{n+1}^{\mathscr{K}} M \overset{\text{def}}{=} \forall \vec{P} \subseteq \Lambda(K)^o.\, \phi_{n+1}(d)\, \vec{P}\Downarrow \Rightarrow M\vec{P}\Downarrow_{\mathscr{K}}$$
$$\& \quad \forall e \in D_n.\, \forall N \in \Lambda(K)^o.\, e \lessdot_n^{\mathscr{K}} N \Rightarrow de \lessdot_n^{\mathscr{K}} MN].$$

D_1 is a two-element set with the top element equal to $\perp_1$—the least convergent element of D. It is easy to see that $\forall M\Downarrow_{\mathscr{K}}.\, \perp_1 \lessdot_1^{\mathscr{K}} M$. The targeted relation, $\lessdot^{\mathscr{K}}$, is obtained as the "conjunction" of all $\lessdot_n^{\mathscr{K}}$, as n ranges over ω, which we make precise later in the subsection.

We remark the following useful fact.

LEMMA 9.4.5. *Let* $D \overset{F}{\rightarrowtail} E \overset{G}{\twoheadrightarrow} D$ *be an embedding of complete lattices* D *into* E; *i.e.*, F, G *are continuous maps satisfying* $F \circ G \sqsubseteq \mathsf{id}$ *and* $G \circ F = \mathsf{id}$. *Then* F *preserves arbitrary joins.*

Proof. By the Adjoint Functor Theorem. ∎

Applying the above lemma, we observe that the embeddings $\phi_n : D_n \rightarrowtail D$ preserve arbitrary joins. To all intents and purposes, we may regard the embeddings, ϕ_n, simply as set-theoretic inclusions.

Let us establish some properties satisfied by the family of relations $\lessdot_n^{\mathscr{H}}$. Note that all the results for the rest of this subsection culminating in Lemma 9.4.12 apply, in particular, to $\lessdot_n^{c}$, $\lessdot_n$, and $\lessdot_n^{\omega}$.

LEMMA 9.4.6. $\forall n \in \omega . \forall d, d' \in D_n . \forall M \in \Lambda(K)^o . d' \sqsubseteq d \lessdot_n^{\mathscr{H}} M \Rightarrow d' \lessdot_n^{\mathscr{H}} M.$

Proof. We prove by induction on n. The base case is trivial. Suppose true for n. Let $d' \sqsubseteq d \lessdot_{n+1}^{\mathscr{H}} M$. Applying the monotonicity of "$\cdot$" repeatedly,

$$\phi_{n+1}(d')\,\vec{P}\Downarrow \Rightarrow \phi_{n+1}(d)\,\vec{P}\Downarrow \Rightarrow M\vec{P}\Downarrow_{\mathscr{H}}.$$

Suppose $e \lessdot_n^{\mathscr{H}} N$. Then, $d'e \sqsubseteq de \lessdot_n^{\mathscr{H}} MN$. By the induction hypothesis, $d'e \lessdot_n^{\mathscr{H}} MN$ and we are done. ∎

LEMMA 9.4.7. *For each $n \in \omega$, $\lessdot_n^{\mathscr{H}}$ is arbitrary join inclusive in its first argument; i.e.,* $\forall X \subseteq D_n . \forall x \in X . x \lessdot_n^{\mathscr{H}} M \Rightarrow \bigsqcup X \lessdot_n^{\mathscr{H}} M]$.

Proof. By induction on n. Again, the base case is trivial. Suppose true for n. Let $X \subseteq D_{n+1}$ and $\forall x \in X . x \lessdot_{n+1}^{\mathscr{H}} M$. We will show $\bigsqcup X \lessdot_{n+1}^{\mathscr{H}} M$. Suppose $\phi_{n+1}(\bigsqcup X)\,\vec{P}\Downarrow$. Recall that the projection ϕ_{n+1} preserves arbitrary joins. Since application left-preserves arbitrary joins (Lemma 4.5.2), by applying it repeatedly, we deduce that $\phi_{n+1}(x)\,\vec{P}\Downarrow$ for some x. By supposition, $M\vec{P}\Downarrow_{\mathscr{H}}$.

It remains to show $e \lessdot_n^{\mathscr{H}} N \Rightarrow (\bigsqcup X) \cdot e \lessdot_n^{\mathscr{H}} MN$. Since application left-preserves arbitrary joins, $(\bigsqcup X) \cdot e = \bigsqcup_{x \in X} x \cdot e$. Observe that $\forall x \in X . xe \lessdot_n^{\mathscr{H}} MN$ by supposition. Invoking the induction hypothesis, we have $\bigsqcup_{x \in X} x \cdot e \lessdot_n^{\mathscr{H}} MN$ and we are done. ∎

The preceding lemmas establish that $\{d \in D_n : d \lessdot_n^{\mathscr{H}} M\} \subseteq D_n$ is a downward-closed set which is also closed under *arbitrary* (not just directed) joins. It is therefore, a fortiori, Scott-closed.

LEMMA 9.4.8. (i) *If $d \lessdot_n^{\mathscr{H}} M$ then $i_n(d) \lessdot_{n+1}^{\mathscr{H}} M$.*

(ii) *If $d \lessdot_{n+1}^{\mathscr{H}} M$ then $j_n(d) \lessdot_n^{\mathscr{H}} M$.*

Proof. We prove by simultaneous induction on n. The base cases for (i) and (ii) are trivial. Suppose assertions (i) and (ii) are true for n. For (i): Suppose $d \lessdot_{n+1}^{\mathscr{H}} M$, we want to prove $i_{n+1}(d) \lessdot_{n+2}^{\mathscr{H}} M$. Observe that $\phi_{n+2}(i_{n+1}(d)) = \phi_{n+1}(d)$. Hence,

$$\phi_{n+2}(i_{n+1}(d))\,\vec{P}\Downarrow \Rightarrow M\vec{P}\Downarrow_{\mathscr{H}}.$$

It remains to prove that $e \lessdot^{\mathscr{K}}_{n+1} N \Rightarrow i_{n+1}(d) \cdot e \lessdot^{\mathscr{K}}_{n+1} MN$. By the induction hypothesis in (ii), $j_n(e) \lessdot^{\mathscr{K}}_n N$. Hence, $d(j_n(e)) \lessdot^{\mathscr{K}}_n MN$. Now, by the induction hypothesis in (i), $i_n(d(j_n(e))) \lessdot^{\mathscr{K}}_{n+1} MN$, which is just $i_{n+1}(d) \cdot e \lessdot^{\mathscr{K}}_{n+1} MN$. The proof for (ii) is similar and we omit it. ∎

We are now in a position to define the relation $\lessdot^{\mathscr{K}}$ for which we have hitherto been labouring in this subsection.

DEFINITION 9.4.9. Define $\lessdot^{\mathscr{K}} \subseteq D \times \Lambda(K)^o$ as follows:

$$d \lessdot^{\mathscr{K}} M \stackrel{\text{def}}{=} \forall n \in \omega . d_n \lessdot^{\mathscr{K}}_n M.$$

Defined in this way, $\lessdot^{\mathscr{K}}$ inherits properties satisfied by its "approximants," $\lessdot^{\mathscr{K}}_n$. A first such inherited property is what we loosely term as the Scott-closedness (in the first argument) of $\lessdot^{\mathscr{K}}$. In fact, $\lessdot^{\mathscr{K}}$ satisfies a stronger property: it is *arbitrary join inclusive*, which we show later.

LEMMA 9.4.10 (Scott-Closedness). *For each $M \in \Lambda(K)^o$, the set*

$$\lessdot^{\mathscr{K}-1}(M) \stackrel{\text{def}}{=} \{d \in D : d \lessdot^{\mathscr{K}} M\}$$

is Scott-closed.

Proof. Straightforward induction. ∎

Remark 9.4.11. For each $d \in D$, there is a canonical choice of $\langle d_n \rangle_{n \in \omega}$, where each $d_n \in D$, such that $d = \bigsqcup \phi_n(d_n)$; each d_n is uniquely determined by the n-projection function $\psi_n : D \to D_n$. However, for any sequence $\langle e_n \rangle_{n \in \omega}$ where $e_n \in D_n$ such that

$$d = \bigsqcup_{n \in \omega} \phi_n(e_n),$$

we claim that $d \lessdot^{\mathscr{K}} M$ iff $\forall n \in \omega . e_n \lessdot^{\mathscr{K}}_n M$.

"$\Rightarrow$" We first note that by Lemma 9.4.8, for $d \in D_n$

$$d \lessdot^{\mathscr{K}}_n M \Leftrightarrow \phi_n(d) \lessdot^{\mathscr{K}} M.$$

Now, for each n, $\phi_n(e_n) \sqsubseteq d \lessdot^{\mathscr{K}} M$. By $\lessdot^{\mathscr{K}}$ being left-closed in the first argument, we have $\phi_n(e_n) \lessdot^{\mathscr{K}} M$. Hence, $e_n \lessdot^{\mathscr{K}}_n M$.

"$\Leftarrow$" $e_n \lessdot^{\mathscr{K}}_n M \Rightarrow \phi_n(e_n) \lessdot^{\mathscr{K}} M$. Then, by directed-join inclusiveness of $\lessdot^{\mathscr{K}}$.

We are now in a position to show that the relation $\lessdot^{\mathscr{K}}$ thus defined does indeed satisfy the recursive specification we stated at the outset.

LEMMA 9.4.12 (Recursive Specification). *$d \lessdot^{\mathscr{K}} M$ iff*

(A) $\forall \vec{P} \subseteq \Lambda(K)^o.[D \models d\vec{P}\Downarrow \Rightarrow \mathscr{K} \models M\vec{P}\Downarrow_{\mathscr{K}}]$ *and*

(B) $\forall e \in D. \forall N \in \Lambda(K)^o. e \lessdot^{\mathscr{K}} N \Rightarrow de \lessdot^{\mathscr{K}} MN]$.

Proof. "$\Rightarrow$": Suppose $d\vec{P}\Downarrow$. By the continuity of "$\cdot$", we have $d\vec{P} = \bigsqcup (d_n\vec{P})$. Hence, for some n, $d_n\vec{P}\Downarrow$, and so by $d_n \lessdot_n^{\mathscr{K}} M$, we have $M\vec{P}\Downarrow_{\mathscr{K}}$. Let $e \lessdot^{\mathscr{K}} N$. Then $e_n \lessdot_n^{\mathscr{K}} N$ and so, as $d_{n+1} \lessdot_{n+1}^{\mathscr{K}} M$, we have $d_{n+1}e_n \lessdot_n^{\mathscr{K}} MN$ and so $\phi_n(d_{n+1}e_n) \lessdot^{\mathscr{K}} MN$. Now, since $de = \bigsqcup_{n\in\omega} \phi_n(d_{n+1} \cdot e_n)$, by Remark 9.4.11, we conclude that $de \lessdot^{\mathscr{K}} MN$.

"$\Leftarrow$": We show by induction that $\forall n \in \omega. d_n \lessdot_n^{\mathscr{K}} M$. The base case is trivial. Suppose $d_n \lessdot_n^{\mathscr{K}} M$ and we want to establish $d_{n+1} \lessdot_{n+1}^{\mathscr{K}} M$. $\phi_{n+1}(d_{n+1})\,\vec{P}\Downarrow$ clearly implies, by monotonicity of "$\cdot$", that $d\vec{P}\Downarrow$. $M\vec{P}\Downarrow_{\mathscr{K}}$ then follows by supposition. Let $e \lessdot_n^{\mathscr{K}} N$. By hypothesis, $d\phi_n(e) \lessdot^{\mathscr{K}} MN$. But, $\phi_n(d_{n+1}e) \sqsubseteq d\phi_n(e)$. Since $\lessdot^{\mathscr{K}}$ is left-closed (in the first argument), we have $\phi_n(d_{n+1}e) \lessdot^{\mathscr{K}} MN$ which is equivalent to $d_{n+1}e \lessdot_n^{\mathscr{K}} MN$. ∎

Bisimulation Logical Relations are Arbitrary Join Inclusive

We can now prove an important property satisfied by the logical relation $\lessdot^{\mathscr{K}}$.

LEMMA 9.4.13 (Arbitrary Join Inclusiveness). *$\lessdot^{\mathscr{K}}$ is arbitrary join inclusive in the first argument; i.e., for any $X \subseteq D$*

$$[\forall x \in X. x \lessdot^{\mathscr{K}} M] \Rightarrow \left(\bigsqcup X\right) \lessdot^{\mathscr{K}} M.$$

Proof. By Fact 4.5.1, we have

$$\bigsqcup X = \bigsqcup_{x\in X} \bigsqcup_{n\in\omega} \phi_n(x_n) = \bigsqcup_{n\in\omega} \bigsqcup_{x\in X} \phi_n(x_n) = \bigsqcup_{n\in\omega} \phi_n\left(\bigsqcup_{x\in X} x_n\right);$$

the last is justified because the embedding ϕ_n preserves arbitrary joins. By Remark 9.4.11, it suffices to show inductively that

$$\forall n \in \omega. \bigsqcup_{x\in X} x_n \lessdot_n^{\mathscr{K}} M.$$

Base case is again trivial. Suppose $\bigsqcup_{x\in X} x_n \lessdot_n^{\mathscr{K}} M$. Since

$$\left(\bigsqcup_{x\in X} \phi_{n+1}(x_{n+1})\right)\vec{P} = \bigsqcup_{x\in X} (\phi_{n+1}(x_{n+1})\,\vec{P}),$$

$\phi_{n+1}(\bigsqcup_{x\in X} x_{n+1})\,\vec{P}\Downarrow$ then implies that for some $x \in X$, $\phi_{n+1}(x_{n+1})\,\vec{P}\Downarrow$. Then, by definition of $x \lessdot^{\mathscr{K}} M$, we deduce that $M\vec{P}\Downarrow_{\mathscr{K}}$.

Suppose $e \lessdot_n^{\mathscr{K}} N$. By definition of $x \lessdot^{\mathscr{K}} M$, $\forall x \in X. x_{n+1} \cdot e \lessdot_n^{\mathscr{K}} MN$. Since $\lessdot_n^{\mathscr{K}}$ is arbitrary join inclusive, we have

$$\left(\bigsqcup_{x \in X} x_{n+1} \right) \cdot e = \bigsqcup_{x \in X} (x_{n+1} \cdot e) \lessdot_n^{\mathscr{K}} MN,$$

and we are done. ∎

LEMMA 9.4.14. *For $d \in D$ and $M, N \in \Lambda(K)^o$, $d \lessdot^{\mathscr{K}} M \lesssim^{\mathscr{K}} N \Rightarrow d \lessdot^{\mathscr{K}} N$.*

Proof. The lemma may be established by proving $\forall n \in \omega. \mathbf{L}_n$ inductively, where

$$\mathbf{L}_n : \forall d \in D_n. \forall M, N \in \Lambda(K)^o. d \lessdot_n^{\mathscr{K}} M \lesssim^{\mathscr{K}} N \Rightarrow d \lessdot_n^{\mathscr{K}} N.$$

The details should be routine by now and we omit them. ∎

As examples of $\lessdot^{\mathscr{K}}$, we consider $\lessdot$, $\lessdot^{\omega}$, and $\lessdot^{\mathsf{c}}$. In the case of λI_{ω}, the formal constants Ψ_i are given the natural interpretations in D, i.e., $\mathsf{Gr}(\mathsf{up}\, \psi_i)$); and for λI_{c}, C is interpreted as $\mathsf{Gr}(\mathsf{up}(\perp_1 \searrow i))$ as before.

Reflexive Bisimulation Logical Relations

DEFINITION 9.4.15. Let $\mathscr{K}$ be an lts with a prescribed interpretation in D. The bisimulation logical relation $\lessdot^{\mathscr{K}}$ is *reflexive* if $\forall M \in \Lambda(K)^o. [\![M]\!] \lessdot^{\mathscr{K}} M$. We say that $\mathscr{K}$ is reflexive if $\lessdot^{\mathscr{K}}$ is.

Open Question 9.4.16. Compare the above notion with the Fundamental Theorem of Logical Relation (see, e.g., (Sta85)). Can a similar general result be proved? More specifically, characterize the class of lts's $\mathscr{K} = \langle K, \Downarrow_{\mathscr{K}} \rangle$ such that $\forall M \in \Lambda(K)^o. [\![M]\!] \lessdot^{\mathscr{K}} M$.

For the rest of this subsection, we show that $\lessdot$, $\lessdot^{\mathsf{c}}$, and $\lessdot^{\omega}$ are reflexive.

PROPOSITION 9.4.17. *For $\mathscr{K} = \lambda I$, λI_{c}, λI_{ω}, we have $\forall M \in \Lambda(K)^o. [\![M]\!] \lessdot^{\mathscr{K}} M$.*

Proof. We prove only the case of λI_{c}. The rest is similar. Owing to the well-known translation between $\Lambda^o(C)$ and $\mathbf{CL}(C)$, where C is a set of constants and $\mathbf{CL}(C)$ the class of **SK**-combinators over C (see, e.g., (Bar84)), and because $\lessdot^{\mathsf{c}}$ is a logical relation, it suffices to prove that

(a) $c =^{\mathrm{def}} [\![\mathsf{C}]\!] \lessdot^{\mathsf{c}} \mathsf{C}$

(b) $k =^{\mathrm{def}} [\![\mathbf{K}]\!] \lessdot^{\mathsf{c}} \mathbf{K}$

(c) $s =^{\mathrm{def}} [\![\mathbf{S}]\!] \lessdot^{\mathsf{c}} \mathbf{S}$.

We omit the proof of (b) since it is similar to and easier than (c). With reference to the Recursive Specification Lemma, observe that in each case, condition (A) is trivially satisfied.

(a) We show that $e \lessdot^{c} N \Rightarrow ce \lessdot^{c} CN$. Suppose the antecedent. First, we establish that $\forall \vec{P}.ce\vec{P}\Downarrow \Rightarrow CN\vec{P}\Downarrow_{c}$. Without loss of generality, we may assume $e\Downarrow$, hence $N\Downarrow_{c}$; then the implication is trivially true since $ce = [\![\mathbf{I}]\!]$ and $CN\Downarrow_{c}\mathbf{I}$. Now, suppose $f \lessdot^{c} P$; then $cef = f \lessdot^{c} P \sim^{c} CNP$, and so by Lemma 9.4.14 we have $cef \lessdot^{c} CNP$.

(c) We show $e \lessdot^{c} N \Rightarrow se \lessdot^{c} SN$. We first show that

$$\forall P_1, \ldots, P_n.seP_1 \cdots P_n\Downarrow \Rightarrow SNP_1 \cdots P_n\Downarrow_{c}. \qquad (*)$$

$(*)$ holds trivially for $n \leqslant 1$. For $n \geqslant 2$,

$$seP_1 \cdots P_n = eP_2(P_1P_2)\,P_3 \cdots P_n;$$

similarly for $SN\vec{P}$. $(*)$ is then seen to hold because $e \lessdot^{c} N$. Next, we show that $f \lessdot^{c} P \Rightarrow sef \lessdot^{c} SNP$. Suppose the antecedent. By similar argument, we see that $\forall \vec{Q}.sef\vec{Q}\Downarrow_{c} \Rightarrow SNP\vec{Q}\Downarrow_{c}$. Suppose $g \lessdot^{c} Q$. Then $sefg = eg(fg) \lessdot^{c} NQ(PQ) \sim^{c} SNPQ$ and so we are done. ▮

Operationally Tailored Preorders on D

Given an lts $\mathscr{K} = \langle K, \Downarrow_{\mathscr{K}} \rangle$, we define a preorder $\lesssim^{\mathscr{K}}$ on D which, roughly speaking, compares the extent to which any two elements in D bisimulate elements of K with respect to tests consisting of elements of K. The quotient of D by $\simeq^{\mathscr{K}}$ identifies any two elements of D which are *not* observably distinguishable with respect to elements of $\mathscr{K}$ as tests. In certain cases to be specified in the sequel, this quotient structure turns out to be a fully abstract model for the language $\mathscr{K}$.

DEFINITION 9.4.18. For $d, e \in D$,

$$d \lesssim^{\mathscr{K}} e \stackrel{\text{def}}{=} \forall M \in \Lambda(K)^{o}.[e \lessdot^{\mathscr{K}} M \Rightarrow d \lessdot^{\mathscr{K}} M].$$

We abbreviate $d \lesssim^{\mathscr{K}} e \,\&\, e \lesssim^{\mathscr{K}} d$ as $d \simeq^{\mathscr{K}} e$. Denote the $\simeq^{\mathscr{K}}$-equivalence class of d as $[d]^{\mathscr{K}}$.

We show that $\lesssim^{\mathscr{K}}$ simulates in D the bisimulation relation $\sqsubseteq\!\!\!\simeq^{\mathscr{K}}$ of $\mathscr{K}$; i.e.,

$$\forall M, N \in \Lambda(K)^{o}.M \sqsubseteq\!\!\!\simeq^{\mathscr{K}} N \Leftrightarrow [\![M]\!] \lesssim^{\mathscr{K}} [\![N]\!].$$

Since $\lessdot^{\mathscr{K}}$ is left-closed in its first argument, it follows that $\sqsubseteq$, the domain order of D, is a *refinement* of $\lesssim^{\mathscr{K}}$; i.e., $\sqsubseteq \subseteq \lesssim^{\mathscr{K}}$. The following result says that the relation $\lesssim^{\mathscr{K}}$ is componentwise *arbitrary join inclusive.*

LEMMA 9.4.19. *Let* $X \subseteq D$. *Then*

(i) $[\forall x \in X. x \lesssim^{\mathscr{K}} d] \Rightarrow \bigsqcup X \lesssim^{\mathscr{K}} d$.

(ii) $[\forall x \in X. d \lesssim^{\mathscr{K}} x] \Rightarrow d \lesssim^{\mathscr{K}} \bigsqcup X$.

(iii) $X \subseteq [d]^{\mathscr{K}} \Rightarrow \bigsqcup X \in [d]^{\mathscr{K}}$.

Proof. (i) is a straightforward corollary of the join inclusiveness of $\lessdot^{\mathscr{K}}$. (ii) follows directly from $\sqsubseteq \subseteq \lesssim^{\mathscr{K}}$ and the transitivity of $\lesssim^{\mathscr{K}}$. (iii) follows from (i) and (ii). ∎

COROLLARY 9.4.20. *Let* $d, e \in D$. *Then,* $d \lesssim^{\mathscr{K}} e \Rightarrow \bigsqcup [d]^{\mathscr{K}} \sqsubseteq \bigsqcup [e]^{\mathscr{K}}$.

Proof. Suppose $d \lesssim^{\mathscr{K}} e$. By (i) of the lemma, $\bigsqcup [d]^{\mathscr{K}} \lesssim^{\mathscr{K}} e$; and so, by (i) again, $\bigsqcup [d]^{\mathscr{K}} \sqcup e \lesssim^{\mathscr{K}} e$. The other direction follows from $\sqsubseteq \subseteq \lesssim^{\mathscr{K}}$. Hence, $\bigsqcup [d]^{\mathscr{K}} \sqcup e \simeq^{\mathscr{K}} e$. It then follows that $\bigsqcup [d]^{\mathscr{K}} \sqsubseteq \bigsqcup [d]^{\mathscr{K}} \sqcup e \sqsubseteq \bigsqcup [e]^{\mathscr{K}}$. ∎

Next, we establish a crucial relationship between $\lesssim^{\mathscr{K}}$ and $\precsim^{\mathscr{K}}$.

COROLLARY 9.4.22. *Suppose* $\mathscr{K}$ *has a combinatory interpretation in* D *and that it is reflexive. Let* $M, N \in \Lambda(K)^o$. *Then,* $M \precsim^{\mathscr{K}} N \Leftrightarrow [\![M]\!] \lessdot^{\mathscr{K}} N$.

Proof. "⇒": By reflexivity of $\mathscr{K}$; i.e., $[\![M]\!] \lessdot^{\mathscr{K}} M$ and an appeal to Lemma 9.4.14.

"⇐": Immediate from adequacy of $\mathscr{K}$ and condition (A) of the Recursive Specification Lemma. ∎

LEMMA 9.4.23. *Suppose* $\mathscr{K}$ *has a combinatory interpretation in* D *and that it is reflexive;* $d \in D$ *and* $M \in \Lambda(K)^o$. *Then* $d \lessdot^{\mathscr{K}} M \Leftrightarrow d \lesssim^{\mathscr{K}} [\![M]\!]$.

Proof. "⇒": Let $d \lessdot^{\mathscr{K}} M$. Suppose $[\![M]\!] \lessdot^{\mathscr{K}} N$ and so, by the above corollary, $M \precsim^{\mathscr{K}} N$, which by Lemma 9.4.14 implies $d \lessdot^{\mathscr{K}} N$. Hence, $d \lesssim^{\mathscr{K}} [\![M]\!]$.

"⇐": Suppose $d \lesssim^{\mathscr{K}} [\![M]\!]$. Reflexivity of $\lessdot^{\mathscr{K}}$ and adequacy then imply $d \lessdot^{\mathscr{K}} M$. ∎

PROPOSITION 9.4.24 (Crucial). *Suppose that $\mathscr{K}$ has a combinatory interpretation in D and that it is reflexive; and let $M, N \in \Lambda(K)^o$. Then*

$$[\![M]\!] \lesssim^{\mathscr{K}} [\![N]\!] \Leftrightarrow M \sqsubseteq^{\mathscr{K}} N.$$

Proof.

$$\begin{aligned} M \sqsubseteq^{\mathscr{K}} N &\Leftrightarrow [\![M]\!] \lessdot^{\mathscr{K}} N && \text{Corollary 9.4.22} \\ &\Leftrightarrow [\![M]\!] \lesssim^{\mathscr{K}} [\![N]\!]. && \text{Lemma 9.4.23} \ \blacksquare \end{aligned}$$

As usual, we define $[d]^{\mathscr{K}} \lesssim^{\mathscr{K}} [e]^{\mathscr{K}}$ if $d \lesssim^{\mathscr{K}} e$, turning $\langle D/\simeq^{\mathscr{K}}, \lesssim^{\mathscr{K}}/\simeq^{\mathscr{K}} \rangle$ into a partial order. In fact, we can say rather more than that:

LEMMA 9.4.25. $D_{\lesssim^{\mathscr{K}}} = \langle D/\simeq^{\mathscr{K}}, \lesssim^{\mathscr{K}}/\simeq^{\mathscr{K}} \rangle$ *is a complete lattice.*

Proof. It is instructive, at this juncture, to identify the $\lesssim^{\mathscr{K}}$-least element. Clearly, $\forall d. \bot \lesssim^{\mathscr{K}} d$. We claim that

$$d \lesssim^{\mathscr{K}} \bot \Rightarrow d = \bot,$$

which implies that $[\bot]^{\mathscr{K}} = \{\bot\}$. To see this, recall that $\forall M. \bot \lessdot^{\mathscr{K}} M$; and so, for any $d \lesssim^{\mathscr{K}} \bot$, we have that

$$\forall M \in \Lambda(K)^o. \forall \vec{P} \in \Lambda(K)^o. d\vec{P}\Downarrow \Rightarrow M\vec{P}\Downarrow_{\mathscr{K}}.$$

In particular, $\forall M. d\Downarrow \Rightarrow M\Downarrow_{\mathscr{K}}$. Now, $d \neq \bot$; i.e., $d\Downarrow$ would lead to a contradiction.

The $\lesssim^{\mathscr{K}}$-largest element is $[\top]^{\mathscr{K}}$.

It remains (and suffices) to show that every subset has a lub. Let $X = \{[d_i]^{\mathscr{K}} : i \in \mathscr{I}\} \subseteq D_{\lesssim^{\mathscr{K}}}$. Observe that $\bar{d} =^{\text{def}} \bigsqcup \{d_i'\}$ where $\forall i \in \mathscr{I}. d_i' \in [d_i]^{\mathscr{K}}$ exists because D is a complete lattice. We immediately have $\forall i. [d_i]^{\mathscr{K}} \lesssim^{\mathscr{K}} [\bar{d}]^{\mathscr{K}}$ because $d_i \lesssim^{\mathscr{K}} d_i' \sqsubseteq \bar{d}$. Hence, $[\bar{d}]^{\mathscr{K}}$ is an $\lesssim^{\mathscr{K}}$-upper bound of X. Now suppose that for some d, $\forall i \in \mathscr{I}. [d_i]^{\mathscr{K}} \lesssim^{\mathscr{K}} [d]^{\mathscr{K}}$. Suppose that $d \lessdot^{\mathscr{K}} M$. The previous supposition then implies that

$$\forall i \in \mathscr{I}. d_i' \lessdot^{\mathscr{K}} M.$$

By arbitrary join inclusiveness of $\lessdot^{\mathscr{K}}$, $\bar{d} \lessdot^{\mathscr{K}} M$. Hence, $[\bar{d}]^{\mathscr{K}} \lesssim^{\mathscr{K}} [d]^{\mathscr{K}}$. $\blacksquare$

Note that the $\lesssim^{\mathscr{K}}$-lub, $[\bar{d}]^{\mathscr{K}}$, where $\bar{d} =^{\text{def}} \bigsqcup d_i'$, is independent of the choice of representatives from the respective $\simeq^{\mathscr{K}}$-equivalence classes $[d_i]^{\mathscr{K}}$.

LEMMA 9.4.26. (i) *For* $d \in D$, $\bigsqcup_{\sqsubseteq} [d]^{\mathscr{K}} \simeq^{\mathscr{K}} d$; *equivalently,* $\bigsqcup_{\sqsubseteq} [d]^{\mathscr{K}} \in [d]^{\mathscr{K}}$.

(ii) *The map* $[\![-]\!]^{\mathscr{A}} : D \to D_{\lesssim^{\mathscr{A}}}$ *defined by* $d \mapsto [\![d]\!]^{\mathscr{A}}$ *is continuous; i.e., for* $X \equiv \{d_i : i \in \mathscr{I}\} \subseteq D$, *a* $\sqsubseteq$*-directed set,*

$$\bigsqcup_{\lesssim^{\mathscr{A}}} \{[\![d_i]\!]^{\mathscr{A}} : i \in \mathscr{I}\} = \left[\!\!\left[\bigsqcup_{\sqsubseteq} \{d_i : i \in \mathscr{I}\} \right]\!\!\right]^{\mathscr{A}}.$$

Proof. (i) Direct from Lemma 9.4.19(iii).

(ii) $[\![-]\!]$ is monotinic because $\sqsubseteq \subseteq \lesssim^{\mathscr{A}}$. For the same reason, if $\{d_i : i \in \mathscr{I}\}$ is a $\sqsubseteq$-directed subset of D, then $\{[\![d_i]\!] : i \in \mathscr{I}\}$ is a $\lesssim^{\mathscr{A}}$-directed set. Since $D_{\lesssim^{\mathscr{A}}}$ is a complete lattice, $\bigsqcup_{\lesssim^{\mathscr{A}}} [\![d_i]\!]$ exists and by the preceding remark,

$$\bigsqcup_{\lesssim^{\mathscr{A}}} \{[\![d_i]\!] : i \in \mathscr{I}\} = \left[\!\!\left[\bigsqcup_{\sqsubseteq} \{d_i : i \in \mathscr{I}\} \right]\!\!\right],$$

which is precisely what the continuity of $[\![-]\!]$ entails. ∎

Definition of the Model $Q^{\mathscr{A}}$

We are now in a position to define a substructure $Q^{\mathscr{A}}$ of D which will turn out to be fully abstract with respect to $\mathscr{A}$ subject to certain conditions which will be specified in the sequel.

DEFINITION 9.4.27. Define a map $F_{\mathscr{A}} : \kappa(D) \to D$, where $\kappa(D)$ is the set of compact elements of D, by

$$F_{\mathscr{A}} : d \mapsto \bigsqcup_{\sqsubseteq} [\![d]\!]^{\mathscr{A}}.$$

Monotonicity of $F_{\mathscr{A}}$ follows from $\sqsubseteq \subseteq \lesssim^{\mathscr{A}}$ and Corollary 9.4.20.

Now, let $Q^{\mathscr{A}}$ be the unique continuous extension of $F_{\mathscr{A}}$ to D. Observe that for $d \in D$, $\bigsqcup_{\sqsubseteq} [\![d]\!]^{\mathscr{A}} \in [\![d]\!]^{\mathscr{A}}$ implies $[\![\bigsqcup_{\sqsubseteq} [\![d]\!]^{\mathscr{A}}]\!]^{\mathscr{A}} = [\![d]\!]^{\mathscr{A}}$. Hence,

$$F_{\mathscr{A}}(F_{\mathscr{A}}(d)) = \bigsqcup \left[\!\!\left[\bigsqcup [\![d]\!]^{\mathscr{A}} \right]\!\!\right]^{\mathscr{A}} = \bigsqcup [\![d]\!]^{\mathscr{A}} = F_{\mathscr{A}}(d).$$

Thus, we have $Q^{\mathscr{A}} \circ Q^{\mathscr{A}} = Q^{\mathscr{A}}$, i.e., $Q^{\mathscr{A}}$ is a projection. This implies that $Q^{\mathscr{A}} \lhd D$; i.e., $Q^{\mathscr{A}}$ is a *retract* of D. Note that $\mathrm{id}_D \sqsubseteq Q^{\mathscr{A}}$. We write the image of the projection map $Q^{\mathscr{A}}$ as $Q^{\mathscr{A}}$—no confusion need arise.

9.5. *Approximants of the Model:* $Q_i^{\mathscr{A}}$

In this subsection, we define $Q_i^{\mathscr{A}}$, the ith approximant of $Q^{\mathscr{A}}$, which, we show, is *fully abstract* with respect to $\mathscr{A}_i$, the ith approximant of the language $\mathscr{A} = \langle K, \Downarrow_{\mathscr{A}} \rangle$. This subsection is devoted to the proof of the

"approximant" full abstraction results at each finite level $i \in \omega$. Roughly speaking, the strategy that we adopt is to prove the main full abstraction result by establishing it as the "limit" of the approximant full abstraction results at each finite level. Recall that $D_i =^{\text{def}} \psi_i D$ and $[\![\Psi_i]\!] = \psi_i$. For each $d \in D_i$, define $[d]_i^{\mathscr{K}} =^{\text{def}} D_i \cap [d]^{\mathscr{K}}$.

LEMMA 9.5.1. (i) *For* $i \in \omega$, $d \in D_i$, $\bigsqcup_{\sqsubseteq} [d]_i^{\mathscr{K}} = \psi_i(\bigsqcup_{\sqsubseteq} [d]^{\mathscr{K}}) \in [d]_i^{\mathscr{K}}$.

(ii) *Define* $\max_i : D_i \to D_i$ *by* $\max_i(d) = \bigsqcup [d]_i^{\mathscr{K}}$. *Then* $\max_i$ *is monotonic, hence continuous since* D_i *is finite.*

(iii) *For* $d, e \in D$. $d \lesssim^{\mathscr{K}} e \Rightarrow \bigsqcup [d]_i^{\mathscr{K}} \sqsubseteq \bigsqcup [e]_i^{\mathscr{K}}$.

Proof. (i) Let $\bar{d} =^{\text{def}} \bigsqcup [d]_i^{\mathscr{K}}$. We first observe that $\bar{d} \in D_i$, because D_i is a complete lattice. Next, we show that $\bar{d} \simeq^{\mathscr{K}} d$; equivalently, $\bar{d} \in [d]^{\mathscr{K}}$. Now, $d \lesssim^{\mathscr{K}} \bar{d}$ follows immediately from $\sqsubseteq \subseteq \lesssim^{\mathscr{K}}$. To show $\bar{d} \lesssim^{\mathscr{K}} d$, suppose $d \lessdot^{\mathscr{K}} M$. By definition of $[d]_i^{\mathscr{K}}$

$$\forall e \in [d]_i^{\mathscr{K}} . e \lessdot^{\mathscr{K}} M.$$

By arbitrary join inclusiveness of $\lessdot^{\mathscr{K}}$, $\bar{d} = \bigsqcup [d]_i^{\mathscr{K}} \lessdot^{\mathscr{K}} M$. Since $[d]_i^{\mathscr{K}} = [d]^{\mathscr{K}} \cap D_i$, we have $\bigsqcup [d]_i^{\mathscr{K}} \sqsubseteq \bigsqcup [d]^{\mathscr{K}}$. Hence,

$$\bigsqcup [d]_i^{\mathscr{K}} = \psi_i\left(\bigsqcup [d]_i^{\mathscr{K}}\right) \sqsubseteq \psi_i\left(\bigsqcup [d]^{\mathscr{K}}\right)$$

because $\bigsqcup [d]_i^{\mathscr{K}} \in D_i$. To show that $\psi_i(\bigsqcup [d]^{\mathscr{K}}) \sqsubseteq \bigsqcup [d]_i^{\mathscr{K}}$, it suffices to show that

$$\psi_i\left(\bigsqcup [d]^{\mathscr{K}}\right) \simeq^{\mathscr{K}} d.$$

$d \sqsubseteq \bigsqcup [d]^{\mathscr{K}}$ implies $d = \psi_i(d) \sqsubseteq \psi_i(\bigsqcup [d]^{\mathscr{K}})$ which implies $d \lesssim^{\mathscr{K}} \psi_i(\bigsqcup [d]^{\mathscr{K}})$. Now, suppose $d \lessdot^{\mathscr{K}} M$. Then, by inclusiveness of $\lessdot^{\mathscr{K}}$, $\bigsqcup [d]^{\mathscr{K}} \lessdot^{\mathscr{K}} M$. By definition of $\lessdot^{\mathscr{K}}$, $\forall i \in \omega . \psi_i(\bigsqcup [d]^{\mathscr{K}}) \lessdot_i^{\mathscr{K}} M$, which is equivalent to $\psi_i(\bigsqcup [d]^{\mathscr{K}}) \lessdot^{\mathscr{K}} M$, and we are done.

(ii) Let $d, e \in D_i$ such that $d \sqsubseteq e$. Then we have $d \lesssim^{\mathscr{K}} e$ and so, by Corollary 9.4.20, $\bigsqcup [d]^{\mathscr{K}} \sqsubseteq \bigsqcup [e]^{\mathscr{K}}$. Monotonicity of ψ_i and (i) together imply that $\max_i(d) \sqsubseteq \max_i(e)$.

(iii) Similar to (ii). ∎

DEFINITION 9.5.2. We define $Q_i^{\mathscr{K}} : D \to D_i$ as $Q_i^{\mathscr{K}}(d) = \max_i \circ \psi_i(d) = \bigsqcup [d_i]_i^{\mathscr{K}}$.

Then, for $d \in D_i$,

$$\begin{aligned} Q_i^{\mathscr{K}} \circ Q_i^{\mathscr{K}}(d) &= Q_i^{\mathscr{K}}\left(\bigsqcup [d]_i^{\mathscr{K}}\right) \\ &= \bigsqcup \left[\bigsqcup [d]_i^{\mathscr{K}}\right]_i^{\mathscr{K}} \\ &= \bigsqcup [d]_i^{\mathscr{K}} \\ &= Q_i^{\mathscr{K}}(d). \end{aligned}$$

The penultimate step is justified because

$$\bigsqcup [d]_i^{\mathscr{K}} \in [d]^{\mathscr{K}} \cap D_i \Rightarrow \left[\bigsqcup [d]_i^{\mathscr{K}}\right]_i^{\mathscr{K}} = [d]_i^{\mathscr{K}}.$$

Hence, $Q_i^{\mathscr{K}}$ is a retract of D_i; and hence, of D.

DEFINITION 9.5.3. We define, for each $i \in \omega$, a structure $\langle Q_i^{\mathscr{K}}, \cdot, [\![-]\!]^{Q_i^{\mathscr{K}}} \rangle$, where

- "$\cdot$" is just the restriction of "$\cdot$": $D \times D \to D$ to $Q_i^{\mathscr{K}} \times Q_i^{\mathscr{K}}$.
- $[\![-]\!]^{Q_i^{\mathscr{K}}} : \Lambda(K)^o \to Q_i^{\mathscr{K}}$ is defined by $[\![M]\!]^{Q_i^{\mathscr{K}}} =^{\text{def}} Q_i^{\mathscr{K}} \circ \psi_i([\![M]\!])$.

Clearly, we need to show that application is well defined; i.e., $\forall d, e \in Q_i^{\mathscr{K}}. d \cdot e \in Q_i^{\mathscr{K}}$. To do this, we first prove a useful lemma. Let $x, y \in D$ and define $x \searrow y$ to be the element $\mathsf{Gr}(\mathsf{up}(x \searrow y))$ in D. Note that for $d \in D$,

$$(x \searrow y) \cdot d = \begin{cases} y & \text{if } x \sqsubseteq d, \\ \bot & \text{else.} \end{cases}$$

LEMMA 9.5.4. *Let $\mathscr{K}$ be reflexive. Suppose $d, c \in D$ such that $dc\Downarrow$. Then,*

$$b \lesssim^{\mathscr{K}} dc \Rightarrow (c \searrow b) \lesssim^{\mathscr{K}} d.$$

Proof. Suppose $d \lessdot^{\mathscr{K}} M$. We will establish $(c \searrow b) \lessdot^{\mathscr{K}} M$ according to the Recursive Specification Lemma. Let $P_1, \ldots, P_n \in \Lambda(K)^o$ such that $(c \searrow b)\, \vec{P}\Downarrow_{\mathscr{K}}$. We consider three cases:

- $n = 0$: Since by premise, $dc\Downarrow$, and so, $d\Downarrow$ which implies $M\Downarrow_{\mathscr{K}}$ by supposition.

- $n = 1$: Then $(c \searrow b)\, P_1 = b$ and $c \sqsubseteq [\![P_1]\!]$, the latter implies that $c \lessdot^{\mathscr{K}} P$ by reflexivity of $\lessdot^{\mathscr{K}}$ and left-closedness of $\lessdot^{\mathscr{K}}$. Since $\lessdot^{\mathscr{K}}$ is a logical relation, $b \lesssim^{\mathscr{K}} dc \lessdot^{\mathscr{K}} MP_1$. Hence, $(c \searrow b)\, P_1\Downarrow \Rightarrow MP_1\Downarrow_{\mathscr{K}}$.

• $n \geqslant 2$: It should be clear that $(c \searrow b)\,\vec{P}\Downarrow \Rightarrow (c \searrow b)\,\vec{P} = bP_2 \cdots P_n$ and $c \sqsubseteq P_1$. By the same argument as before $b \lesssim^{\mathscr{K}} MP_1$; then $bP_2 \cdots P_n \Downarrow \Rightarrow MP_1 \cdots P_n \Downarrow_{\mathscr{K}}$.

Next, we show that $e \lesssim^{\mathscr{K}} N \Rightarrow (c \searrow b)\,e \lesssim^{\mathscr{K}} MN$. Without loss of generality, suppose that $(c \searrow b)\,e\Downarrow$; i.e., $(c \searrow b)\,e = b$ and $c \sqsubseteq e$; the latter implies that $c \lesssim^{\mathscr{K}} N$ by left-closedness of $\lesssim^{\mathscr{K}}$. Hence, $dc \lesssim^{\mathscr{K}} MN$, and so, by assumption, $b \lesssim^{\mathscr{K}} MN$. ▮

Remark 9.5.5. If $c, b \in D_{i-1}$, then observe that $\psi_i(c \searrow b) = (c \searrow b)$, and so $(c \searrow b) \in D_i$.

COROLLARY 9.5.6. *The application in* $Q_i^{\mathscr{K}} = \langle Q_i^{\mathscr{K}}, \cdot, [\![-]\!]^{Q_i^{\mathscr{K}}} \rangle$ *is well defined; i.e., for* $d, e \in Q_i^{\mathscr{K}}$, $d \cdot e \in Q_i^{\mathscr{K}}$.

Proof. Let $f =^{\text{def}} \bigsqcup [de]_{i-1}^{\mathscr{K}}$ $(= \bigsqcup [de]_i^{\mathscr{K}})$, then $f \in D_{i-1} \subseteq D_i$. To show that $d \cdot e \in Q_i^{\mathscr{K}}$, it suffices to show that $f = de$. Because $de \in [de]_{i-1}^{\mathscr{K}}$, we clearly have $de \sqsubseteq \bigsqcup [de]_{i-1}^{\mathscr{K}} = f$. It remains to show that $f \sqsubseteq de$. Now $f \lesssim^{\mathscr{K}} \psi_{i-1}(de) = de_{i-1}$, by Lemma 9.5.1(i). Without loss of generality, assume $de_{i-1}\Downarrow$; otherwise $f = \bot$ since $[\bot]^{\mathscr{K}}$ is a singleton set. By the lemma, we have $(e_{i-1} \searrow f) \lesssim^{\mathscr{K}} d$. By Lemma 9.5.1 and the above remark,

$$(e_{i-1} \searrow f) \sqsubseteq \bigsqcup [(e_{i-1} \searrow f)]_i^{\mathscr{K}} \sqsubseteq \bigsqcup [d]_i^{\mathscr{K}} = d.$$

Hence, $f = (e_{i-1} \searrow f)\,e \sqsubseteq de$. ▮

$Q_i^{\mathscr{K}} = \langle Q_i^{\mathscr{K}}, \cdot, \Uparrow \rangle$ is a q-aswd where $\cdot$ is restricted to $Q_i^{\mathscr{K}}$.

LEMMA 9.5.7. *Suppose that* $\mathscr{K}$ *has a combinatory interpretation in* D *and that it is reflexive.*

(i) *For* $d \in D_i$ *and* $M \in \Lambda(K)_i^o$, *then* $d \lesssim^{\mathscr{K}} M \Rightarrow Q_i^{\mathscr{K}}(d) = \bigsqcup [d]_i^{\mathscr{K}} \sqsubseteq [\![M]\!]^{Q_i^{\mathscr{K}}}$.

(ii) $\forall M \in \Lambda(K)_i^o . [\![M]\!]^{Q_i^{\mathscr{K}}} \lesssim^{\mathscr{K}} M$.

Proof. (i) Suppose $d \lesssim^{\mathscr{K}} M$, then $d \lesssim^{\mathscr{K}} [\![M]\!]$ by Lemma 9.4.23. This implies that $Q_i^{\mathscr{K}}(d) \sqsubseteq [\![M]\!]^{Q_i^{\mathscr{K}}}$ by Lemma 9.5.1(iii).

(ii) Let $M \in \Lambda(K)_i^o$. Now $[\![M]\!]^{Q_i^{\mathscr{K}}} \approx^{\mathscr{K}} [\![M]\!]_i = [\![M]\!] \lesssim^{\mathscr{K}} M$, because $\lesssim^{\mathscr{K}}$ is reflexive. Hence, $[\![M]\!]^{Q_i^{\mathscr{K}}} \lesssim^{\mathscr{K}} M$. ▮

PROPOSITION 9.5.8. *Suppose* $\mathscr{K}$ *has a combinatory interpretation in* D *and that it is reflexive. Then* $[\![-]\!]^{Q_i^{\mathscr{K}}}$ *preserve application; i.e.,* $\forall M, N \in \Lambda(K)_i^o . [\![MN]\!]^{Q_i^{\mathscr{K}}} = [\![M]\!]^{Q_i^{\mathscr{K}}} \cdot [\![N]\!]^{Q_i^{\mathscr{K}}}$.

Proof. Let $M, N \in \Lambda(K)^o_i$. Then

$$\begin{aligned}[\![MN]\!]^{Q_i^{\mathscr{N}}} &= Q_i^{\mathscr{N}}([\![M^i N^i]\!]_i) \\ &\sqsubseteq Q_i^{\mathscr{N}}([\![M]\!]_i \cdot [\![N]\!]_i) \\ &\sqsubseteq Q_i^{\mathscr{N}}([\![M]\!]^{Q_i^{\mathscr{N}}} \cdot [\![N]\!]^{Q_i^{\mathscr{N}}}).\end{aligned}$$

Since $[\![M]\!]^{Q_i^{\mathscr{N}}}$, $[\![N]\!]^{Q_i^{\mathscr{N}}} \in Q_i^{\mathscr{N}}$ and application in $Q_i^{\mathscr{N}}$ is well defined, $[\![M]\!]^{Q_i^{\mathscr{N}}} \cdot [\![N]\!]^{Q_i^{\mathscr{N}}} \in Q_i^{\mathscr{N}}$. Hence, $[\![MN]\!]^{Q_i^{\mathscr{N}}} \sqsubseteq Q_i^{\mathscr{N}}([\![M]\!]^{Q_i^{\mathscr{N}}} [\![N]\!]^{Q_i^{\mathscr{N}}}) = [\![M]\!]^{Q_i^{\mathscr{N}}} \cdot [\![N]\!]^{Q_i^{\mathscr{N}}}$.

To prove the other inequality, observe that, since $\lessdot^{\mathscr{N}}$ is a logical relation (this property is crucial at this point), and since by Lemma 9.5.7(ii), $[\![M]\!]^{Q_i^{\mathscr{N}}} \cdot [\![N]\!]^{Q_i^{\mathscr{N}}} \lessdot^{\mathscr{N}} MN$, by Lemma 9.5.7(i),

$$[\![M]\!]^{Q_i^{\mathscr{N}}} [\![N]\!]^{Q_i^{\mathscr{N}}} = Q_i([\![M]\!]^{Q_i^{\mathscr{N}}} [\![N]\!]^{Q_i^{\mathscr{N}}}) \sqsubseteq [\![MN]\!]^{Q_i^{\mathscr{N}}}. \quad \blacksquare$$

9.6. *Proof of Full Abstraction*

PROPOSITION 9.6.1 (Approximant Full Abstraction). *Suppose $\mathscr{N}$ has a combinatory interpretation in D; it is reflexive and projection maps are definable in it. Then, $\forall i \in \omega.\mathscr{N}_i$ is fully abstract with respect to $Q_i^{\mathscr{N}}$; i.e.,*

$$\forall M, N \in \Lambda(K)^o_i. M \sqsubseteq_i^{\mathscr{N}} N \Leftrightarrow [\![M]\!]^{Q_i^{\mathscr{N}}} \sqsubseteq [\![N]\!]^{Q_i^{\mathscr{N}}}.$$

Proof. "$\Rightarrow$": Let $M \sqsubseteq_i^{\mathscr{N}} N$. By Lemma 9.2.5(ii), we have $M \sqsubseteq^{\mathscr{N}} N$. Hence, by Proposition 9.4.24, $[\![M]\!] \lesssim^{\mathscr{N}} [\![N]\!]$. Then, by Lemma 9.5.1(iii),

$$[\![M]\!]^{Q_i^{\mathscr{N}}} = \bigsqcup [\![[\![M]\!]]\!]_i^{\mathscr{N}} \sqsubseteq \bigsqcup [\![[\![N]\!]]\!]_i^{\mathscr{N}} = [\![N]\!]^{Q_i^{\mathscr{N}}}.$$

Thus, indeed $[\![M]\!]^{Q_i^{\mathscr{N}}} \sqsubseteq [\![N]\!]^{Q_i^{\mathscr{N}}}$.

"$\Leftarrow$": Conversely, suppose that $[\![M]\!]^{Q_i^{\mathscr{N}}} \sqsubseteq [\![N]\!]^{Q_i^{\mathscr{N}}}$. Then

$$[\![M]\!] = ([\![M]\!])_i \simeq^{\mathscr{N}} [\![M]\!]^{Q_i^{\mathscr{N}}} \sqsubseteq [\![N]\!]^{Q_i^{\mathscr{N}}} \simeq^{\mathscr{N}} ([\![N]\!])_i = [\![N]\!];$$

i.e., $[\![M]\!] \lesssim^{\mathscr{N}} [\![N]\!]$. By Proposition 9.4.24, $M \sqsubseteq^{\mathscr{N}} N$ and so, by Lemma 9.2.5(i), $M^i \sqsubseteq_i^{\mathscr{N}} N^i$. Since $M \sim^{\mathscr{N}_i} M^i$ and $N \sim^{\mathscr{N}_i} N^i$ by Remark 9.2.6, $M \sqsubseteq_i^{\mathscr{N}} N$. $\blacksquare$

LEMMA 9.6.2. (i) *For $i \leqslant j$, $Q_i^{\mathscr{N}} \xrightarrow{Q_j^{\mathscr{N}}} Q_j^{\mathscr{N}} \xrightarrow{Q_i^{\mathscr{N}}} Q_i^{\mathscr{N}}$ is an embedding. Hence, we have $Q_0^{\mathscr{N}} \trianglelefteq Q_1^{\mathscr{N}} \trianglelefteq \cdots Q_i^{\mathscr{N}} \trianglelefteq \cdots \trianglelefteq Q_j^{\mathscr{N}} \cdots$.*

(ii) $Q^{\mathscr{N}} = \bigsqcup_{i \in \omega} Q_i^{\mathscr{N}}$.

Proof. (i) Let $d \in Q_i^{\mathscr{N}} \subseteq D_i \subseteq D_j$. Then

$$
\begin{aligned}
Q_i^{\mathscr{K}} \circ Q_j^{\mathscr{K}}(d) &= Q_i^{\mathscr{K}}\left(\bigsqcup [d]_j^{\mathscr{K}}\right) \\
&= \bigsqcup \left[\bigsqcup [d]_j^{\mathscr{K}}\right]_i^{\mathscr{K}} \\
&= \bigsqcup [d_i]_i^{\mathscr{K}} \qquad \therefore \bigsqcup [d]_j^{\mathscr{K}} \in [d]_j^{\mathscr{K}} \\
&= \bigsqcup [d]_i^{\mathscr{K}} \\
&= Q_i^{\mathscr{K}}(d) = d.
\end{aligned}
$$

Let $e \in Q_j^{\mathscr{K}}$. Then $[e]_i^{\mathscr{K}} \subseteq [e]_j^{\mathscr{K}} \Rightarrow \bigsqcup [e]_i^{\mathscr{K}} \sqsubseteq \bigsqcup [e]_j^{\mathscr{K}} \Rightarrow \bigsqcup [e]_i^{\mathscr{K}} \lesssim \bigsqcup [e]_j^{\mathscr{K}}$. Hence, by Lemma 9.5.1(iii), $Q_j^{\mathscr{K}} \circ Q_i^{\mathscr{K}}(e) = \bigsqcup [\bigsqcup [e]_i^{\mathscr{K}}]_j^{\mathscr{K}} \sqsubseteq \bigsqcup [\bigsqcup [e]_j^{\mathscr{K}}]_j^{\mathscr{K}} = e$.

(ii) For each $d \in D_i$, $F_{\mathscr{K}}(d) = \bigsqcup [d]^{\mathscr{K}}$. But $[d]^{\mathscr{K}} = \bigcup_{j \in \omega} [d]_j^{\mathscr{K}} \Rightarrow \bigsqcup [d]^{\mathscr{K}} = \bigsqcup_{j \in \omega} (\bigsqcup [d]_j^{\mathscr{K}})$. Therefore, $F_{\mathscr{K}}(d) = \bigsqcup_{j \in \omega} (\bigsqcup [d]_j^{\mathscr{K}}) = \bigsqcup_{j \in \omega} Q_j^{\mathscr{K}}(d)$. Since $F_{\mathscr{K}}$ is the unique extension of $Q^{\mathscr{K}}$, we have $Q^{\mathscr{K}} = \bigsqcup_i Q_i^{\mathscr{K}}$. ∎

LEMMA 9.6.3. *The application in* $\langle Q^{\mathscr{K}}, \cdot \rangle$ *is well defined.*

Proof. Let $d, e \in Q^{\mathscr{K}}$. Then

$$
\begin{aligned}
d \cdot e &= Q^{\mathscr{K}}(d) \cdot Q^{\mathscr{K}}(e) \\
&= \bigsqcup_{i \in \omega} Q_i^{\mathscr{K}}(d) \cdot \bigsqcup_{i \in \omega} Q_i^{\mathscr{K}}(e) \\
&= \bigsqcup_{i \in \omega} (Q_i^{\mathscr{K}}(d) \cdot Q_i^{\mathscr{K}}(e)) \\
&= \bigsqcup_{i \in \omega} Q_i^{\mathscr{K}}(d \cdot e) \\
&= Q^{\mathscr{K}}(d \cdot e). \quad \blacksquare
\end{aligned}
$$

Let $M \in \Lambda(K)^o$. Then

$$
\begin{aligned}
[\![M]\!]^Q &= Q^{\mathscr{K}}([\![M]\!]) \\
&= \left(\bigsqcup_i Q_i^{\mathscr{K}}\right)\left(\bigsqcup_i [\![M]\!]_i\right) \\
&= \bigsqcup_i Q_i^{\mathscr{K}}([\![M]\!]_i) \\
&= \bigsqcup_i Q_i^{\mathscr{K}}([\![M']\!]_i) \\
&= \bigsqcup_i [\![M']\!]^{Q_i^{\mathscr{K}}}.
\end{aligned}
$$

Also, $[\![M]\!] = \bigsqcup [\![M^i]\!]_i \sqsubseteq \bigsqcup [\![M^i]\!]^{Q_i^{\mathscr{N}}} = [\![M]\!]^{Q}$. Hence, $\forall M \in \Lambda(K)^o . [\![M]\!] \sqsubseteq [\![M]\!]^{Q}$.

PROPOSITION 9.6.4. $[\![-]\!]^{Q}$ *preserves application; i.e.,*

$$\forall M, N \in \Lambda(K)^o . [\![MN]\!]^{Q} = [\![M]\!]^{Q}\, [\![N]\!]^{Q}.$$

Proof.

$$\begin{aligned}
[\![MN]\!]^{Q} &= Q^{\mathscr{N}}([\![MN]\!]) \\
&= \left(\bigsqcup_i Q_i^{\mathscr{N}}\right)\left(\bigsqcup_i [\![M^i N^i]\!]_i\right) \\
&= \bigsqcup_i Q_i^{\mathscr{N}}([\![M^i N^i]\!]_i) \\
&= \bigsqcup_i [\![M^i N^i]\!]^{Q_i^{\mathscr{N}}} \\
&= \bigsqcup_i [\![M^i]\!]^{Q_i^{\mathscr{N}}}\, [\![N^i]\!]^{Q_i^{\mathscr{N}}} \qquad [\![-]\!]^{Q_i^{\mathscr{N}}} \text{ preserves application} \\
&= \left(\bigsqcup_i [\![M^i]\!]^{Q_i^{\mathscr{N}}}\right)\left(\bigsqcup_i [\![N^i]\!]^{Q_i^{\mathscr{N}}}\right) \\
&= [\![M]\!]^{Q}\, [\![N]\!]^{Q}. \quad \blacksquare
\end{aligned}$$

We are now able to prove the main theorem (9.1.1) of this section and solve the full abstraction problem which we posed in the beginning of the section for a class of lts's.

Proof of Theorem 9.1.1. "⇒": Suppose that $M \precsim^{\mathscr{N}} N$. Then, by Lemma 9.2.5, $\forall i \in \omega . M^i \precsim_i^{\mathscr{N}} N^i$. Full abstraction of $\mathscr{N}_i$ w.r.t. $Q_i^{\mathscr{N}}$ implies that $\forall i \in \omega . [\![M^i]\!]^{Q_i^{\mathscr{N}}} \sqsubseteq [\![N^i]\!]^{Q_i^{\mathscr{N}}}$. Hence,

$$[\![M]\!]^{Q} = \bigsqcup_{i \in \omega} [\![M^i]\!]^{Q_i^{\mathscr{N}}} \sqsubseteq \bigsqcup_{i \in \omega} [\![N^i]\!]^{Q_i^{\mathscr{N}}} = [\![N]\!]^{Q}.$$

"⇐": Suppose that $[\![M]\!]^{Q} \sqsubseteq [\![N]\!]^{Q}$. Then $[\![M]\!] \sqsubseteq [\![M]\!]^{Q} \sqsubseteq [\![N]\!]^{Q} = Q^{\mathscr{N}}([\![N]\!]) \simeq^{\mathscr{N}} [\![N]\!]$. Thus, $[\![M]\!] \lesssim^{\mathscr{N}} [\![N]\!]$, and so, by Proposition 9.4.24, we have $M \precsim^{\mathscr{N}} N$. $\blacksquare$

COROLLARY 9.6.5. *Since λI_c and λI_ω have combinatory interpretations in D, are reflexive, and projection maps are definable in them, fully abstract models which are retracts of the canonical model D exist and can be constructed for them.*

COROLLARY 9.6.6 (Full Abstraction Conjecture). *If Conjecture* 9.2.14(i) *is true, then* λI *is fully abstract with respect to* $Q^{\lambda I_\omega}$.

10. VARIATIONS

10.1. *Lazy Lambda Calculus with Ground Data*

For our final non-definability result, we consider a different style of extension of λI, to incorporate *ground data*. We consider the simplest possible such extension, where a single atom is added. This corresponds to the domain equation

$$D_* = \mathbf{1} + [D_* \to D_*]$$

(where $+$ is separated sum), which is indeed an extension of our original domain, in the sense that D is a retract of D_*. D_* is still a Scott domain (indeed, a consistently complete algebraic cpo), but it is no longer a lattice; we have introduced *inconsistency* via the sum.

This extension is reflected on the syntactic level by two constants, $*$ and C. We define

$$\lambda I_* \stackrel{\text{def}}{=} \langle \Lambda(\{*, \mathsf{C}\})^o, \Downarrow \rangle$$

with $\Downarrow$ extending the definition for λI as follows,

$$\frac{}{\lambda x.P \Downarrow \lambda x.P} \qquad \frac{M \Downarrow \lambda x.P \quad P[Q/x] \Downarrow N}{MQ \Downarrow N} \qquad \frac{}{* \Downarrow *}$$

$$\frac{}{\mathsf{C} \Downarrow \mathsf{C}} \qquad \frac{M \Downarrow \mathsf{C} \quad N \Downarrow \lambda x.P}{MN \Downarrow \mathbf{T}} \qquad \frac{M \Downarrow \mathsf{C} \quad N \Downarrow \mathsf{C}}{MN \Downarrow \mathbf{T}} \qquad \frac{M \Downarrow \mathsf{C} \quad N \Downarrow *}{MN \Downarrow \mathbf{F}}$$

where $\mathbf{T} \equiv \lambda x.\lambda y.x$, $\mathbf{F} \equiv \lambda x.\lambda y.y$.

Remark 10.1.1. (i) We see that the C combinator introduced here is a natural generalization (not strictly an extension) of the C defined previously in the pure case. Of course, C corresponds to *case selection*, which in the unary case—lifting being unary separated sum—is just convergence testing.

(ii) Let $M \in \Lambda(\{*, \mathsf{C}\})^o$. If $M \Downarrow N$ then N must be one of the following: C, $*$, or $\lambda x.P$. We remark that $*P\vec{R}\Uparrow$.

A theory can be developed for λI_* which runs parallel to what we have done for the pure lazy λ-calculus. Some of the technical details are more complicated because of the presence of inconsistency, but the ideas and

results are essentially the same. Our reasons for mentioning this extension are twofold:

1. To show how the ideas we have developed can be put in a broader context. In particular, with the extension to λI_* the reader should be able to see, at least in outline, how our work can be applied to systems such as Martin-Löf's Type Theory under its Domain Interpretation (DNP83), and (the analogues of) our results in this section can be used to settle most of the questions and conjectures raised in (DNP83).

2. To prove an interesting result which clarifies a point about which there seems to be some confusion in the literature; namely, *what is parallel or?*

The *locus classicus* for parallel or in the setting of typed λ-calculus is (Plo77) (see also (Sco69)). But what of untyped λ-calculus? In (Bar84, p. 375), we find the definition

$$FMN = \begin{cases} \mathbf{I} & \text{if } M \text{ or } N \text{ is solvable,} \\ \text{unsolvable} & \text{otherwise,} \end{cases}$$

which (modulo the difference between the standard and lazy theories) corresponds to our parallel convergence combinator P. The point we wish to make is this: in the pure λ-calculus, where (in domain terms) there are no inconsistent data values (since everything is a function), i.e., we have a lattice, parallel convergence does indeed play the role of parallel or, as the Expansive Full Abstraction Theorem shows. However, when we introduce ground data, and hence inconsistency, a distinction reappears between parallel convergence and parallel or, and it is definitely *wrong* to conflate them. To substantiate this claim, we prove the following result:

> *Even if parallel convergence is added to λI_*, parallel or is still not definable.*

This result is also of interest from the point of view of the fine structure of definability; it shows that parallelism is not all or nothing even in the simple, deterministic setting of λI_*.

DEFINITION 10.1.2. $\lambda I_{*\mathsf{P}}$ is the extension of λI_* with a constant P and the rules

$$\frac{M \Downarrow \mathsf{C} \quad N \Downarrow \mathsf{P}}{MN \Downarrow \mathbf{T}} \qquad \frac{M \Downarrow \mathsf{C} \quad N \Downarrow \mathsf{P}L}{MN \Downarrow \mathbf{T}}$$

$$\frac{}{\mathsf{P} \Downarrow \mathsf{P}} \qquad \frac{M \Downarrow \mathsf{P}}{MN \Downarrow \mathsf{P}N} \qquad \frac{M \Downarrow \mathsf{P} \quad N\Downarrow}{MNL \Downarrow \mathbf{I}} \qquad \frac{M \Downarrow \mathsf{P} \quad L\Downarrow}{MNL \Downarrow \mathbf{I}}.$$

DEFINITION 10.1.3. Let $\lambda I'_*$ be an extension of λI_*. We say that *parallel or is definable in* $\lambda I'_*$ if for some term M

- $M(\mathbf{K}\Omega)\,\Omega$, $M\Omega(\mathbf{K}\Omega)$ converge to abstractions
- $M{*}{*} \Downarrow *$.

THEOREM 10.1.4. *Parallel or is not definable in* $\lambda I_{*\mathrm{p}}$.

We prove the theorem by proceeding along similar lines to our previous non-definability results in λI_c.

Remark 10.1.5. Define

$$\mathsf{positive}(M) \stackrel{\text{def}}{=} M \equiv (\lambda x.N) \text{ or } \mathsf{P} \text{ or } \mathsf{C} \text{ or } \mathsf{P}N,$$

$$\mathsf{negative}(M) \stackrel{\text{def}}{=} M \equiv *,$$

$$\mathsf{canonical}(M) \stackrel{\text{def}}{=} \mathsf{positive}(M) \text{ or } \mathsf{negative}(M).$$

Let $M \in \Lambda(\{\mathsf{P}, \mathsf{C}, *\})^o$. Note that

$$M\Downarrow \Leftrightarrow \mathsf{C}M\Downarrow;$$

also, $\exists M.M \Downarrow N \Leftrightarrow \mathsf{canonical}(N)$.

DEFINITION 10.1.6. Define a *one-step reduction relation* over $\Lambda(\{\mathsf{P}, \mathsf{C}, *\})$ which is an extension of $>$ as defined in Definition 8.2.1 as follows:

$$\frac{}{(\lambda x.P)\,Q > P[Q/x]} \qquad \frac{M > M'}{MN > M'N}$$

$$\frac{}{*M > *M}$$

$$\frac{\mathsf{positive}(M)}{\mathsf{C}M > \mathbf{T}} \qquad \frac{\mathsf{negative}(M)}{\mathsf{C}M > \mathbf{F}} \qquad \frac{M > M'}{\mathsf{C}M > \mathsf{C}M'}$$

$$\frac{\mathsf{canonical}(M) \text{ or } \mathsf{canonical}(N)}{\mathsf{P}MN > \mathbf{I}} \qquad \frac{M > M' \quad N > N'}{\mathsf{P}MN > \mathsf{P}M'N'}.$$

$M \gg N$, $M\uparrow$, and $M \downarrow N$ are defined similarly. In addition, we define

$$M \downarrow M \mathbin{@} 0 \stackrel{\text{def}}{=} M \not>.$$

$$M \downarrow N \mathbin{@} n+1 \stackrel{\text{def}}{=} M \equiv M_0 > M_1 > \cdots > M_{n+1} \,\&\, M_{n+1} \not>.$$

Clearly, $M \downarrow N$ iff there is a unique $n \in \omega$ such $M \downarrow N \mathbin{@} n$.

Remark 10.1.7. The two syntactic classes of the "normal forms" of the one-step leftmost reduction relation $>$ are

$$\mathsf{nf}(>) \stackrel{\text{def}}{=} \{M \in \Lambda(\{\mathsf{P}, \mathsf{C}, *\}) : M \not> \} = \{M : \mathsf{canonical}(M)\} \cup \mathbf{U},$$

where $\mathbf{U}$ is defined by the following rules, with L ranging over $\Lambda(\{\mathsf{P}, \mathsf{C}, *\})$:

$$\frac{}{x \in \mathbf{U}} \qquad \frac{M \in \mathbf{U}}{\mathsf{C}M, ML \in \mathbf{U}} \qquad \frac{M \in \mathbf{U} \quad \neg[\mathsf{canonical}(L)]}{\mathsf{P}ML, \mathsf{P}LM \in \mathbf{U}}.$$

With these extensions, $>$ is still a partial function, and the following properties still hold.

LEMMA 10.1.8. *Let* $M \in \Lambda(\{\mathsf{P}, \mathsf{C}, *\})^o$.

(i) $M\Uparrow \Leftrightarrow M\uparrow$.

(ii) $M \Downarrow N \Leftrightarrow M \downarrow N$.

Proof. By induction on the number of proof steps in $M \Downarrow N$ and $M \downarrow N$. ∎

LEMMA 10.1.9. *Let* $M, P \in \Lambda(\{\mathsf{P}, \mathsf{C}, *\})$. *Then* $M \gg N \Rightarrow M[P/x] \gg N[P/x]$. *In particular,* $M\uparrow \Rightarrow \forall \sigma : \mathsf{Var} \to \Lambda(\{\mathsf{P}, \mathsf{C}, *\})^o . M_\sigma\uparrow$.

Proof. Straightforward analysis according to the rules of $>$. ∎

COROLLARY 10.1.10. *Let* $M \in \Lambda(\{\mathsf{P}, \mathsf{C}, *\})$ *and* $\sigma : \mathsf{Var} \to \Lambda(\{\mathsf{P}, \mathsf{C}, *\})^o$. *Then*

$$M_\sigma \downarrow N \Leftrightarrow \exists M' . M \downarrow M' \;\&\; M'_\sigma \downarrow N.$$

Proof. Suppose $M_\sigma \downarrow N$. Then, by the lemma, for some $M' \in \Lambda(\{\mathsf{P}, \mathsf{C}, *\})$, we have $M \downarrow M'$ and $M_\sigma \gg M'_\sigma$. Since $>$ is a partial function, we must have $M'_\sigma \downarrow N$. The converse is obvious. ∎

We now define $\mathscr{T} =^{\text{def}} \{M \in \Lambda(\{\mathsf{P}, \mathsf{C}, *\}) : \mathsf{FV}(M) \subseteq \{y_1, y_2\} \;\&\; M \not> \}$.

PROPOSITION 10.1.11. *Let* $M \in \mathscr{T}$ *and define* $f_i : \Lambda(\{\mathsf{P}, \mathsf{C}, *\}) \to \Lambda(\{\mathsf{P}, \mathsf{C}, *\})$ *for* $i = 1, 2, 3$ *as follows*:

$$f_1(M) \stackrel{\text{def}}{=} M[\mathbf{K}\Omega/y_1, \Omega/y_2],$$

$$f_2(M) \stackrel{\text{def}}{=} M[\Omega/y_1, \mathbf{K}\Omega/y_2],$$

$$f_3(M) \stackrel{\text{def}}{=} M[*/y_1, */y_2].$$

If $f_1(M)\downarrow a_1 @ n_1$, $f_2(M)\downarrow a_2 @ n_2$, *and* $f_3(M)\downarrow a_3 @ n_3$ *then either*

(a) $\mathsf{negative}(a_i)$ *for* $i = 1, 2, 3$ *i.e.,* $a_1 \equiv a_2 \equiv a_3 \equiv *$, *or*

(b) $\mathsf{positive}(a_i)$ *for* $i = 1, 2, 3$.

Proof. Observe that terms M in $\mathscr{T}$ must have one of the following forms:

(i) $xN_1 \cdots N_k$ $(x \in \{y_1, y_2\}, k \geqslant 0)$

(ii) $*$

(iii) $\lambda x.N$

(iv) C (v) P (vi) $\mathsf{P}N$

(vii) $\mathsf{C}NN_1 \cdots N_k$ $(k \geqslant 0)$

(viii) $\mathsf{P}M_1M_2N_1 \cdots N_k$ $(k \geqslant 0)$.

We prove the proposition by induction on $\langle n_1, n_2, n_3\rangle$ (where $\langle n_1, n_2, n_3\rangle \leqslant \langle n_1', n_2', n_3'\rangle$ if $n_i \leqslant n_i'$ for $i = 1$, 2, and 3) and by case analysis according to the above classification. The base case is clearly valid. Most of these cases can be disposed of directly; we deal with the two which use the induction hypothesis.

(vii) Note that $N \in \mathscr{T}$. We must have $f_1(N)\downarrow b_1 @ m_1$, $f_2(N)\downarrow b_2 @ m_2$ and $f_3(N)\downarrow b_3 @ m_3$ and $\langle m_1, m_2, m_3\rangle < \langle n_1, n_2, n_3\rangle$. Applying the induction hypothesis, we infer that either $f_i(\mathsf{C}N)\downarrow \mathbf{T}$ for $i = 1$, 2, and 3; or $f_i(\mathsf{C}N)\downarrow \mathbf{F}$. By an appeal to Corollary 10.1.10, we conclude the argument by applying the hypothesis to $f_i(X)$ or $f_i(Y)$, respectively, where $N_1N_3 \cdots N_k \downarrow X$ and $N_2N_3 \cdots N_k \downarrow Y$.

(viii) Under the hypothesis of the lemma, we must have $f_i(\mathsf{P}M_1M_2) \Downarrow \mathbf{I}$ for $i = 1$, 2, and 3. Hence, by Corollary 10.1.10, we can apply the induction hypothesis to $f_i(X)$, where $N_1 \cdots N_k \downarrow X$. ∎

Proof of Theorem 10.1.4. Now suppose that we are given a putative term in $\Lambda(\{\mathsf{P}, \mathsf{C}, *\})^o$ defining parallel or. Without loss of generality, we may take this term to have the form $\lambda y_1.\lambda y_2.M$. Applying Corollary 10.1.10, we can obtain $M' \in \mathscr{T}$ from M; but then applying Proposition 10.1.11, we see that $\lambda y_1.\lambda y_2.M'$ cannot define parallel or. Applying Corollary 10.1.10 again, we conclude that $\lambda y_1.\lambda y_2.M$ cannot define parallel or either. ∎

10.2. *Varieties of Function Space*

We round off our treatment by briefly considering the varieties of function space.

1. *The Scott Function Space*

$[D \to E]$, the standard function space of all continuous functions from D to E. In terms of our domain logic $\mathscr{L}$, we can obtain this construction by adding the axiom

$$(1) \quad t \leqslant (t \to t).$$

Note that with (1), $\mathscr{L}$ collapses to a single equivalence class (corresponding to the trivial one-point solution of $D = [D \to D]$). For this reason, Coppo *et al.*, have to introduce atoms in their work on Extended Applicative Type Structures (CDC84).

2. *The Strict Function Space*

$[D \to_\perp E]$, all *strict* continuous functions. This satisfies (1), and also

$$(2) \quad (t \to_\perp \phi) \leqslant f \qquad (\phi \neq t).$$

3. *The Lazy Function Space*

$[D \to E]_\perp$, which satisfies neither (1) nor (2). This has of course been our object of study in this paper.

4. *The Landin–Plotkin Function Space*

$[D \to_\perp E]_\perp$, the lifted strict function space. This satisfies (2) but not (1). The reason for our nomenclature is that this construction in the category of domains and strict continuous functions corresponds to Plotkin's $[D \rightharpoonup E]$ construction in his (equivalent) category of predomains and partial functions (Plo85). Moreover, this may be regarded as the formalization of Landin's applicative-order λ-calculus, with abstraction used to protect expressions from evaluation, as illustrated extensively in (Lan64, Lan65, Bur75).

The intriguing point about these four constructions is that (1) and (2) are *mathematically* natural, yielding cartesian closure and monoidal closure in, e.g., **CPO** and $\mathbf{CPO}_\perp$, respectively (the latter being analogous to partial functions over sets); while (3) and (4) are *computationally* natural, as argued extensively for (3) in this section, and as demonstrated convincingly for (4) by Plotkin in his work on predomains (Plo85). Much current work is aimed at providing good categorical descriptions of generalizations of (4) (Ros86, RR88, Mog86, Mog88b, Mog88a); a similar program is carried out for (3) in (Ong88a, Chap. 5).

RECEIVED June 14, 1989; FINAL MANUSCRIPT RECEIVED May 15, 1991

REFERENCES

[Abr87b] ABRAMSKY, S. (1987), "Domain Theory and the Logic of Observable Properties," Ph.D. thesis, University of London.

[Abr87c] ABRAMSKY, S. (1987), Domain theory in logical form, *in* "Proceedings, Conference on Logic in Computer Science," Comput. Soc. Press.

[Abr91a] ABRAMSKY, S. (1991), "A Domain Equation for Bisimulation," *Inform. and Comput.* **92**, 161–218.

[Abr91b] ABRAMSKY, S. (1991), Domain theory in logical form, *Ann. Pure Appl. Logic* **51**, 1–77.

[AH87] ABRAMSKY, S., AND HANKIN, C. L. (Eds.) (1987), "Abstract Interpretation for Declarative Languages," Horwood.

[AL86] ASPERTI, A., AND LONGO, G. (1986), "Categories of Partial Morphisms and the Relation between Type Structures," Nota Scientifica S-7-85, Dipartimento di Informatica, Università di Pisa; invited lecture, Banach Semester, 1985.

[Aug84] AUGUSTSSON, L. (1984), A compiler for lazy ML, *in* "ACM Symposium on Lazy and Functional Programming," pp. 218–227.

[Böh68] BÖHM, C. (1968), "Alcune proprieta delle forme $\beta\eta$-normali nel λK-calculus," Pubblicazioni 696, Instituto per le Applicazioni del Calcolo, Rome, Italy.

[Bar84] BARENDREGT, H. (1984), "The Lambda Calculus: Its Syntax and Semantics," rev. ed., North-Holland, Amsterdam.

[BCL85] BERRY, G., CURIEN, P.-L., AND LÉVY, J.-J. (1985), Full abstraction for sequential languages: The state of the art, *in* "Algebraic Semantics" (M. Nivat and J. Reynolds, Eds.), pp. 89–132, Cambridge Univ. Press, London/New York.

[Ber81] BERRY, G. (1981), "Some Syntactic and Categorical Constructions of Lambda Calculus Models," Rapport de Recherche 80, Institute National de Recherche en Informatique et en Automatique (INRIA).

[BHA86] BURN, G., HANKIN, C., AND ABRAMSKY, S. (1986), Strictness analysis for higher order functions, *Sci. Comput. Programming* **7**, 249–278.

[Blo88] BLOOM, B. (1988), Can LCF be topped? Flat lattice models of typed lambda calculus, *in* "Proceedings of the Third Symposium on LICS," Comput. Soc. Press.

[Bur75] BURGE, W. H. (1975), "Recursive Programming Techniques," Addison–Wesley, Reading, Ma.

[BvL86] BARENDREGT, H., AND VAN LEEUWEN, M. (1986), "Functional Programming and the Language TALE." Technical Report 412, Dept. of Mathematics. University of Utrecht.

[CDC84] COPPO, M., DEZANI-CIANCAGLINI, M., HONSELL, F., AND LONGO, G. (1984), Extended type structure and filter lambda models, *in* "Logic Colloquium '82" (G. Lolli, G. Longo, and A. Marcja, Eds.), pp. 241–262. Elsevier, Amsterdam.

[Cur86] CURIEN, P.-L. (1986), "Categorical Combinators, Sequential Algorithms, and Functional Programming," Pitman, New York.

[DNP83] DYBJER, P., NORDSTRÖM, B., PETERSSON, K., AND SMITH, J., Eds. (1983), "Workshop on Semantics of Programming Languages, Göteborg, Sweden, August 1983," Programming Methodology Group, Chalmers University of Technology.

[Fai85] FAIRBAIRN, J. (1985), "Design and Implementation of a Simple Typed Language Based on the Lambda Calculus," Ph.D. Thesis, Univ. of Cambridge.

[FW76] FRIEDMAN, D. P., AND WISE, D. S. (1976), CONS should not evaluate its arguments, *in* "Automata, Languages and Programming" (Michaelson and Milner, Eds.), Edinburgh University Press.

[GMW79] GORDON, M. J. C., MILNER, A. J. R., AND WADSWORTH, C. P. (1979), "Edinburgh LCF," Lecture Notes in Computer Science, Vol. 78, Springer-Verlag, Berlin/New York.

[HdR88] HONSELL, F., AND RONCHI DELLA ROCCA, S. (1988), An approximation

theorem for Scott's topological lambda models and the topological incompleteness of lambda calculus, *J. Comput. System Sci.*, to appear.

[Hen80] HENDERSON, P. (1980), "Functional Programming: Applications and Implementation," Prentice-Hall, Englewood Cliffs, NJ.

[HM76] HENDERSON, P., AND MORRIS, J. H. (1976), A Lazy Evaluator, *in* "Third ACM Symposium on the Principles of Programming Languages, Atlanta, GA."

[HM85] HENNESSY, M. C. B., AND MILNER, R. (1985), Algebraic laws for nondeterminism and concurrency, *J. Assoc. Comput. Mach.* **32**, 137–161.

[Hyl76] HYLAND, M. (1976), A syntactic characterization of the equality in some models of the lambda calculus, *J. London Math. Soc. (2)* **12**, 361–370.

[Koy82] KOYMANS, C. P. J. (1982), Models of the lambda calculus, *Inform. and Control* **52**, 206–332.

[Koy84] KOYMANS, C. P. J. (1984), "Models of the Lambda Calculus," Ph.D. thesis, University of Utrecht.

[Lan64] LANDIN, P. J. (1964), The mechanical evaluation of expressions, *Comput. J.* **6**, 308–320.

[Lan65] LANDIN, P. J. (1965), A correspondence between ALGOL 60 and Church's lambda notation, *Comm. ACM* **8**, 89–101, 158–165.

[Lon83] LONGO, G. (1983), Set-theoretical models of lambda calculus: Theories, expansions and isomorphisms, *Ann. Pure Appl. Logic* **24**, 153–188.

[LS86] LAMBEK, J., AND SCOTT, P. J. (1986), "Introduction to Higher Categorical Logic," Cambridge Studies in Advanced Mathematics, No. 7, Cambridge Univ. Press, New York/London.

[MC88] MEYER, A. R., AND COSMADAKIS, S. C. (1988), Semantical paradigms: Notes for an invited lecture, *in* "Proceedings, 3rd Annual IEEE Symposium, Logic in Computer Science," Comput. Soc. Press.

[Mey82] MEYER, A. (1982), What is a model of the lambda calculus? *Inform. and Control* **52**, 87–122.

[Mil75] MILNER, R. (1975), Processes, a mathematical model of computing agents, *in* "Logic Colloquium, Bristol 1973," pp. 157–174, North-Holland, Amsterdam.

[Mil77] MILNER, R. (1977), Fully abstract models of typed lambda-calculus, *Theoret. Comput. Sci.* **4**, 1–22.

[Mil80] MILNER, R. (1980), "A Calculus for Communicating Systems," Lecture Notes in Computer Science, Vol. 92, Springer-Verlag, Berlin.

[Mog86] MOGGI, E. (1986), Categories of partial morphisms and the λ_p-calculus, *in* "Category Theory and Computer Programming" (D. Pitt, S. Abramsky, A. Poigné, and D. Rydeheard, Eds.), pp. 242–251, Lecture Notes in Computer Science, Vol. 240, Springer-Verlag, Berlin/New York.

[Mog88a] MOGGI, E. (1988), Partial Cartesian closed categories of effective objects, *Inform. and Comput.*

[Mog88b] MOGGI, E. (1988), "The Partial Lambda Calculus," Ph.D. thesis, University of Edinburgh.

[Mos74] MOSCHOVAKIS, Y. (1974), "Elementary Induction on Abstract Structures," North-Holland, Amsterdam.

[MP87] MOSES, P. D., AND PLOTKIN, G. D. (1987), On proving limiting completeness, *SIAM J. Comput.* **16**, 179–194.

[Mul86] MULMULEY, K. (1986), Fully abstract submodels of typed lambda calculus, *J. Comput. System Sci.* **33**, 2–46.

[Ong88a] ONG, C.-H. L. (1988), Fully abstract models of the lazy lambda calculus, *in* "Proceedings, 29th Conference on Foundations of Computer Science," Comput. Soc. Press.

[Ong88b] ONG, C.-H. L. (1988), "The Lazy Lambda Calculus: An Investigation into the Foundations of Functional Programming," Ph.D. thesis, University of London.

[Par81] PARK, D. M. (1981), Concurrency on automata and infinite sequences, *in* "Conference on Theoretical Computer Science, Berlin, 1981" (P. Deussen, Ed.), Lecture Notes in Computer Science Vol. 104, Springer-Verlag, Berlin/New York.

[Pau87] PAULSON, L. C. (1987), "Logic and Computation: Interactive Proof with Cambridge LCF," Cambridge Univ. Press, London/New York.

[PJ87] PEYTON JONES, S. L. (1987), "The Implementation of Functional Programming Languages," Prentice–Hall, Englewood Cliffs, NJ.

[Plo73] PLOTKIN, G. D. (1973), "Lambda-Definability and Logical Relations," Technical Report SAI-RM-4, School of Artificial Intelligence, Univ. of Edinburgh.

[Plo75] PLOTKIN, G. D. (1975), Call-by-name, call-by-value and the lambda calculus, *Theoret. Comput. Sci.* **1**, 125–159.

[Plo77] PLOTKIN, G. D. (1977), LCF as a programming language, *Theoret. Comput. Sci.* **5**, 223–255.

[Plo81] PLOTKIN, G. D. (1981), "Post-Graduate Lecture Notes in Advanced Domain Theory," Department of Computer Science, Univ. of Edinburgh.

[Plo85] PLOTKIN, G. D. (1985), "Types and Partial Functions," Post-Graduate Lecture Notes Department of Computer Science, University of Edinburgh.

[Ros86] ROSOLINI, G. (1986), "Continuity and Effectiveness in Topoi," Ph.D. Thesis, Carnegie–Mellon University.

[RR88] ROBINSON, E., AND ROSOLINI, G. (1988), Categories of partial maps, *J. Symbolic Logic*, to appear.

[Sco69] SCOTT, D. S. (1969), A type theoretical alternative to cuch, iswim and owhy.

[Sco72] SCOTT, D. S. (1972), Continuous lattices, *in* "Toposes, Algebraic Geometry and Logic" (E. Lawvere, Ed.), pp. 97–136. Lecture Notes in Mathematics, Vol. 274, Springer-Verlag, Berlin.

[Sco80] SCOTT, D. S. (1980), Relating theories of lambda calculus, *in* "To H. B. Curry: Essays in Combinatory Logic, Lambda Calculus and Formalism" (J. R. Hindley and J. P. Seldin, Eds.), pp. 403–450, Academic Press, New York.

[Sta85] STATMAN, R. (1985), Logical relations and the typed λ-calculus, *Inform. and Control* **65**, 85–97.

[Sto79] STOY, J. E. (1979), "Denotational Semantics: The Scott–Strachey Approach to Programming Language Theory," The MIT Press Series in Computer Science, MIT Press, Cambridge, MA.

[Sto88a] STOUGHTON, A. (1988), "Fully Abstract Models of Programming Languages," Research Notes in Theoretical Computer Science, Pitman, New York.

[Sto88b] STOUGHTON, A. (1988), Private communication via electronic mail.

[Tar55] TARSKI, A. (1955), A lattice-theoretical fixpoint theorem and its applications, *Pacific J. Math.* **5**, 285–309.

[Tur85] TURNER, D. A. (1985), Miranda—A non-strict functional language with polymorphic types, *in* "Functional Programming Languages and Computer Architectures" (J. P. Jouannaud, Ed.), Lecture Notes in Computer Science, Vol. 201, Springer-Verlag, Berlin.

[Wad76] WADSWORTH, C. P. (1976), The relation between computational and denotational properties for Scott's D_∞-models of the lambda calculus, *SIAM J. Comput.* **5**, 488–521.

[Wad78] WADSWORTH, C. P. (1978), Approximate reduction and lambda calculus models, *SIAM J. Comput.* **7**, 337–356.

[Wad85] WADLER, P. (1985), "Introduction to ORWELL," Technical Report, Oxford University Programming Research Group.